



ISTIO SERVICE MESH

COURSE SUMMARY



Comprehensive Summary of
Concepts, Traffic Management,
Security, and Observability
in Istio



Youssef ElZarka



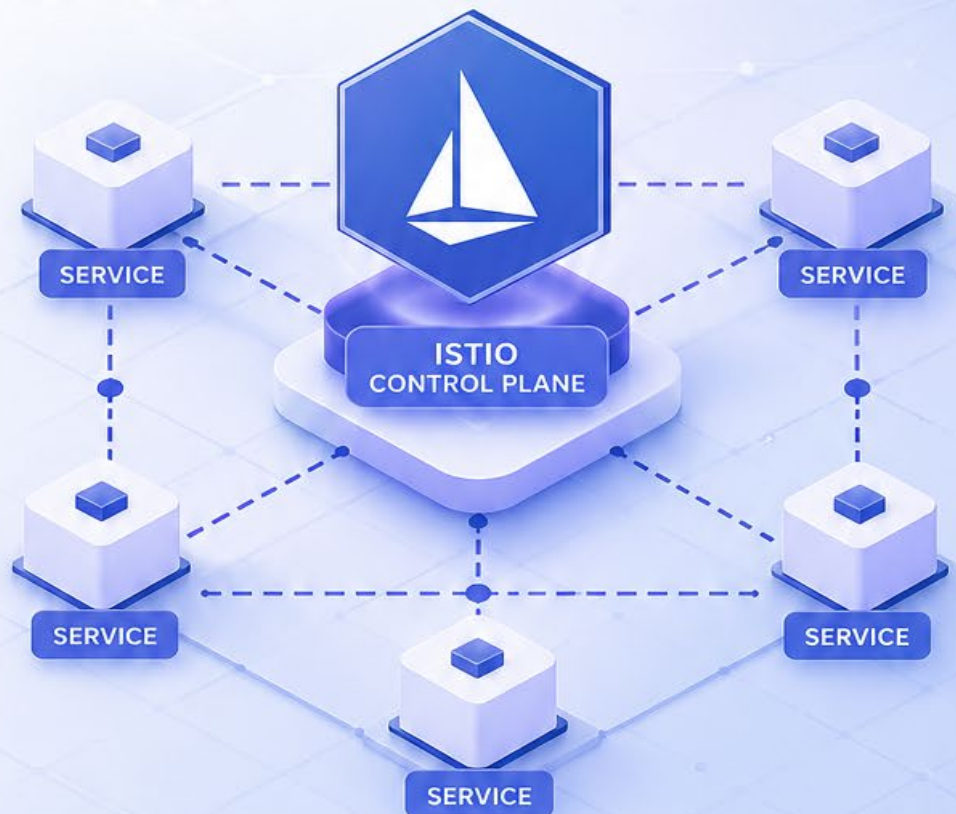
Traffic
Management



Security



Observability



Instructor:
Sevi Karakose



CLOUD-NATIVE



MICROSERVICES



SERVICE MESH

Introduction

Course Introduction	3
Pre-Requests.....	6

Istio Introduction

Monolithic vs. Microservice	8
Service Mesh	10
Istio	12
Installing Istio – Introduction.....	14
Demo - Installing Istioctl	16
Installing Istio on your Cluster.....	18
Deploying Our First Application on Istio.....	20
Demo Deploying Our First Application on Istio	22
Visualizing Service Mesh with Kiali	25
Demo Installing Kiali	27
Demo Create Traffic Into Your Mesh	30

Pre-Requisites

Kubernetes Services	33
Sidecars.....	34
Envoy	36
Gateways.....	38

Traffic Management

Virtual Services	41
Destination Rules	45
Demo – Gateways.....	48
Demo – Virtual Services	51
Demo – Destination Rules	54
Fault Injection.....	58
Demo – Fault Injection.....	60
Timeouts.....	62
Demo – Timeouts.....	64
Retries	66
Circuit Breaking	68

Demo – Circuit Breaking.....	69
A/B Testing.....	71

Security

Security in Istio.....	73
Istio Security Architecture	74
Authentication.....	76
Demo Authentication	79
Authorization.....	81
Demo Authorization	84
Certificate Management.....	87
Demo – Certificate Management.....	89

Observability

Demo Visualizing Metrics with Prometheus and Grafana.....	95
Demo Distributed Tracing with Jaeger.....	98
Demo Kiali in Details	102
Service Mesh Interface (SMI)	106

Course Introduction

مقدمة: الكورس بيتكلم عن إيه؟ 🚀

- ❑ مع تطور الأنظمة الحديثة، بقينا نتحرك من فكرة الـ Monolithic Applications (برنامج واحد كبير) إلى Microservices Architecture (مجموعة خدمات صغيرة مستقلة).
- ❑ التحول ده حل مشاكل كتير زي scalability وسهولة التطوير، لكن في المقابل خلق تحدي جديد:
 - ❖ إزاي الخدمات دي تتواصل مع بعض بشكل منظم، آمن، وقابل للمراقبة؟
- ❑ هنا ظهر مفهوم مهم جدًا اسمه: Service Mesh (فيما معناه طريقة حل مشاكل الـ Microservices Communication)
- ❖ والـ Tool الأشهر لتطبيق المفهوم ده على Kubernetes هو: Istio

أولاً: المشكلة الأساسية

❖ في نظام Microservices، عندك Services كتير:

- تخيل عندك Microservices : DB → Payment → Auth → Backend → Frontend
- وكل Service محتاجة:
 - تكلم Services تانية
 - تعمل Retry لو في فشل
 - تحدد Timeout
 - تأمن الاتصال
 - تسجل Logs وتعمل Monitoring
- ولو كل Service هتكتب الكلام ده بنفسها:
 - الكود هيكبر ويتعقد
 - هيبقى فيه تكرار
 - صعب تتحكم في النظام كله

ثانيًا: الحل Service Mesh

- ❖ Service Mesh هو Layer إضافية بنتحط بين الخدمات، مسؤولة عن:
 - إدارة الترافيك
 - تطبيق السياسات (Policies)
 - الأمان
 - المراقبة
- 🚩 بدل ما كل Service تعمل ده بنفسها

ثالثًا: دور Istio

- ❖ Istio هو Implementation للـ Service Mesh على Kubernetes
- ❖ بيخليك:
 - تتحكم في الترافيك بين الخدمات
 - تضيف Security
 - تراقب الأداء
 - تختبر السيناريوهات المختلفة
- 🚩 وكل ده من غير ما تغير كود التطبيق

1 الأساسيات

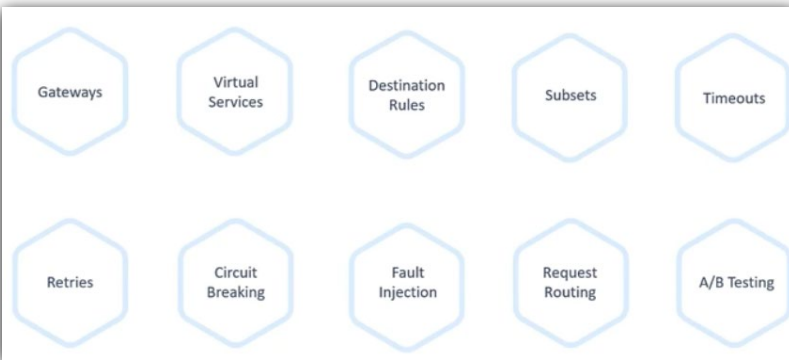
- الفرق بين Monolith و Microservices
- ليه نحتاج Service Mesh

2 Istio Basics

- تثبيت Istio
- تشغيله على Kubernetes

3 Visualization (مهم جداً)

- ❖ باستخدام أدوات زي: Kiali
- ❖ تقدر تشوف:
- مين بيكلم مين
- الترافيك ماشي إزاي
- فين المشاكل



خامساً: أهم مفاهيم Istio

- ❖ Gateways ← مدخل الترافيك من خارج الكلاستر
- ❖ Virtual Services ← تتحكم في الترافيك (Routing) مثلاً:
- 80% من المستخدمين يروحوا لـ Version 1
- 20% ← Version 2
- ❖ Destination Rules ← تتحكم في سلوك الترافيك بعد التوجيه
- ❖ Subsets ← بتستخدم labels عشان تعمل grouping (غالباً حسب version)
- ❖ Retries ← لو request فشل ← يعيد المحاولة
- ❖ Timeouts ← لو الخدمة إتاخرت في الرد ← يقطع الطلب
- ❖ Circuit Breaking ← لو Service واقعة ← يوقف إرسال requests لها (زي الفيوز في دايرة الكهرباء كذا)
- ❖ A/B Testing ← تجرب Version جديد على جزء من المستخدمين
- ❖ Fault Injection ← تعمل Errors متعمدة لاختبار النظام

Security

- ❖ Istio بيوفر:
- Certificates
- Authentication
- Authorization
- Istio بيامن التواصل بين الـ services

Observability 📊

❖ باستخدام:

• Prometheus

• Grafana

○ تقدر تشوف:

▪ Metrics

▪ Performance

▪ Errors

🔍 Distributed Tracing

❖ تتبع الطلب من:

• أول ما يدخل

• لحد ما يمر بكل الخدمات

🎯 مثال واقعي

❖ بدون Istio :

• كل Service مسؤولة عن :

○ Retry

○ Logging

○ Security

❖ مع Istio :

• كله Managed من برا

○ يوزع الترافيك

○ يعمل Retry

○ يراقب الأداء

○ يطبق Security

🔥 والأهم ان كل دا من غير مايزود حاجة في الكود الأساسي

🧠 الخلاصة 🔥

❖ Microservices محتاجة إدارة للتواصل بينها

❖ Service Mesh بيحل المشكلة دي

❖ Istio هو Tool بيطبق Service Mesh على Kubernetes

❖ بيديك:

• Traffic Control

• Security

• Monitoring

• Testing

Pre-Requests

مقدمة

- ❑ قبل ما تبدأ تتعلم Istio و Service Mesh ، لازم تكون واقف على أرض ثابتة في Kubernetes
- ❑ يعني مش هينفع تفهم Service Mesh كويس لو مش فاهم Kubernetes أساساً !!
- ❖ لأن Istio مبني فوق Kubernetes ، وبيعتمد على مفاهيمه بشكل مباشر.

أولاً: إيه المطلوب تكون عارفه؟

❖ الكورس مش بيطلب Expert level ، لكن محتاج منك الأساسيات دي:

Setting up Kubernetes Cluster with MiniKube

```
>
$ minikube start
minikube v1.16.0 on Darwin 10.15.5
minikube 1.19.0 is available! Download it:
https://github.com/kubernetes/minikube/releases/tag/v1.19.0
To disable this notice, run: 'minikube config set WantUpdateNotification false'

Automatically selected the docker driver
Starting control plane node minikube in cluster minikube
Pulling base image ...
Creating docker container (CPUs=2, Memory=4000MB) ...
This container is having trouble accessing https://k8s.gcr.io
To pull new external images, you may need to configure a proxy:
https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
Preparing Kubernetes v1.20.0 on Docker 20.10.0 ...
  Generating certificates and keys ...
  Booting up control plane ...
  Configuring RBAC rules ...
Verifying Kubernetes components...
Enabled addons: storage-provisioner, default-storageclass
Done! kubectrl is now configured to use "minikube" cluster and "default" namespace by default
```

```
>
$ minikube addons enable ingress

Exiting due to MK_USAGE: Due to networking limitations of driver docker on darwin,
ingress addon is not supported.
Alternatively to use this addon you can use a vm-based driver:

'minikube start --vm=true'

To track the update on this work in progress feature please check:
https://github.com/kubernetes/minikube/issues/7332
```

1. Kubernetes Cluster

- تكون فاهم يعني إيه Cluster
- وإزاي تشغل واحد (حتى لو بشكل بسيط)

```
>
$ minikube start --vm=true
minikube v1.16.0 on Darwin 10.15.7
Automatically selected the hyperkit driver
Starting control plane node minikube in cluster minikube
Creating hyperkit VM (CPUs=2, Memory=4000MB, Disk=20000MB) ...
Preparing Kubernetes v1.20.0 on Docker 20.10.0 ...
  Generating certificates and keys ...
  Booting up control plane ...
  Configuring RBAC rules ...
Verifying Kubernetes components...
Enabled addons: storage-provisioner, default-storageclass

Want kubectrl v1.20.0? Try 'minikube kubectrl -- get pods -A'
Done! kubectrl is now configured to use "minikube" cluster and "default" namespace by default

$ minikube addons enable ingress
Verifying ingress addon...
The 'ingress' addon is enabled
```

2. Pods

- أصغر وحدة تشغيل
- فيها Container أو أكثر
- دي اللي عليها كل الشغل

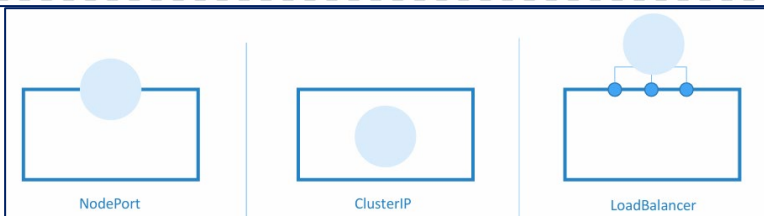
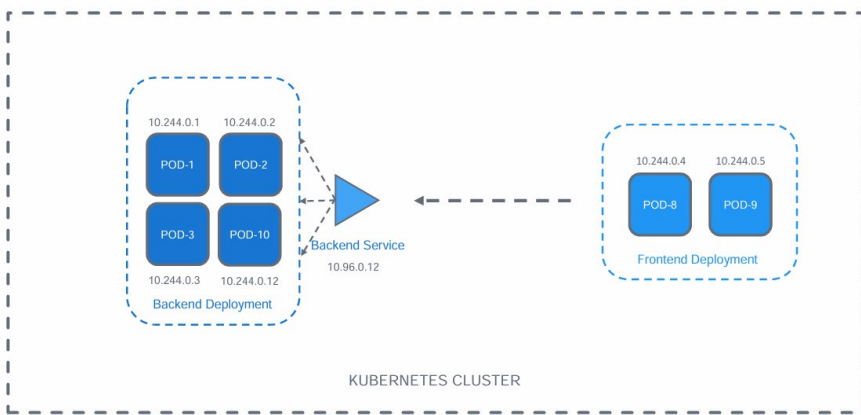
3. ReplicaSets / Deployments

- تشغل أكثر من نسخة من نفس ال Pod
- وتضمن إنها تفضل شغالة
- مثال:
- لو عندك App :
- فيه Deployment بيشتغل 3 Pods
- فلو واحد وقع ← بيتعمله Replace عطلول

4. Services

- بتخلي Pods تكلم بعض
- وبتيدي IP ثابت بدل ال IPs المتغيرة
- دي نقطة مهمة جداً لأن:
- Istio بيتحكم في الترافيك بين ال Services

Kubernetes Services



ثانيًا: الحاجات التي يركز عليها قبل Istio

❖ Kubernetes Services

- عشان تفهم الترافيك بيمشي إزاي بين الخدمات

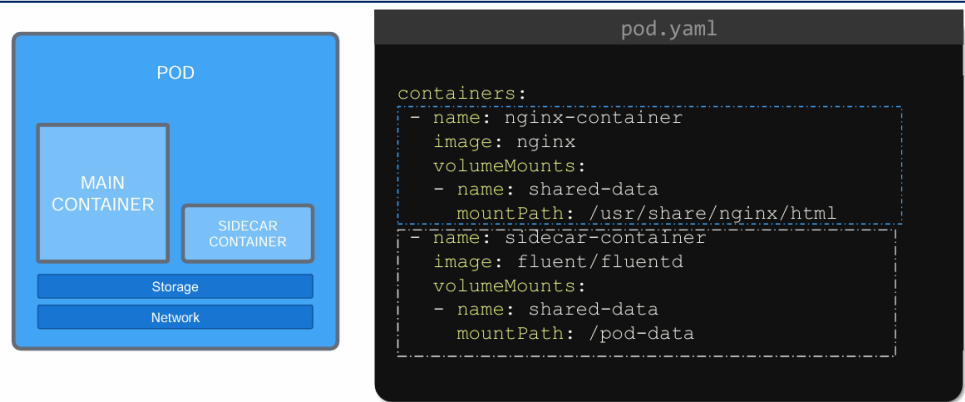
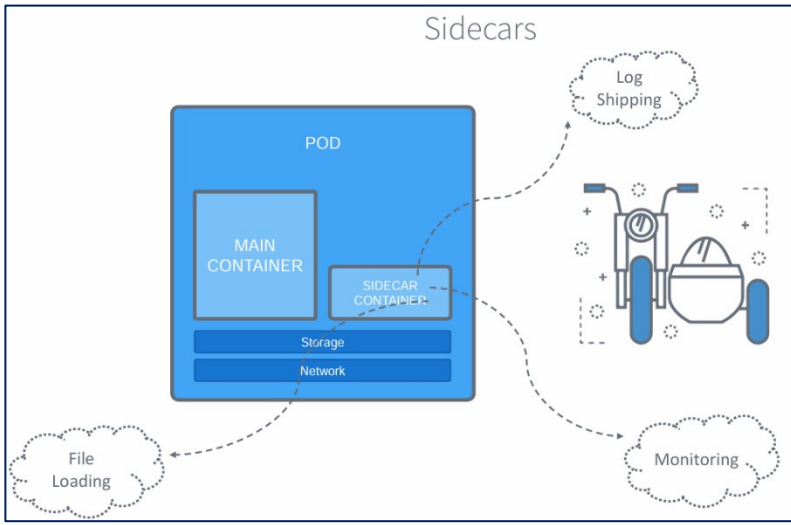
❖ Sidecar (مهمة جدًا)

- ده Concept أساسي في Istio
- ببساطة:

○ Container إضافي جنب التطبيق

✓ بيهتم بالـ Logging/Monitoring/Networking

بدل مالتطبيق الأساسي هو اللي يعمل كدا



❖ Envoy Proxy

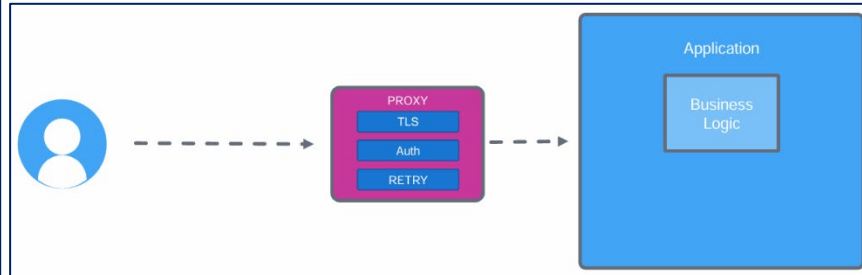
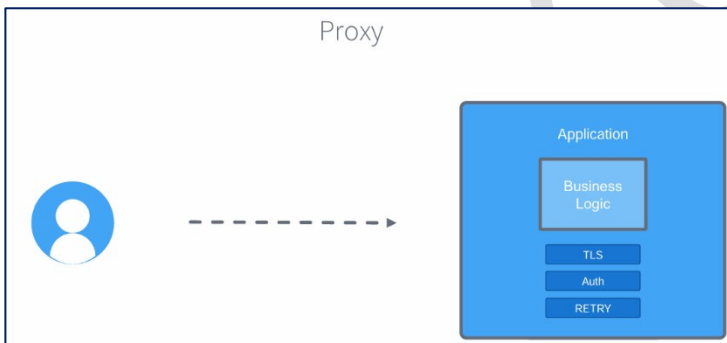
- ده الـ Engine اللي بيعتمد عليه

- هو اللي:

○ يراقب الترافيك

○ يتحكم فيه

○ يطبق السياسات



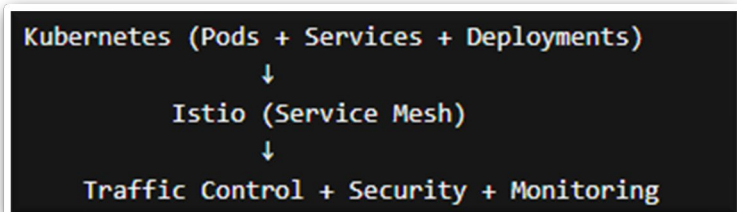
مثال يوضح ليه المتطلبات دي مهمة

❖ تخيل إنك مش فاهم Kubernetes Services وبعدين دخلت Istio وسمعت إننا هنعمل Routing بين Services

👉 فمش هتفهم الترافيك أصلاً ماشي إزاي

❖ أو مثلاً مش فاهم Pods :

👉 يعني مش هتفهم Sidecar اتحط فين ولا بيعمل إيه



Monolithic vs. Microservice

مقدمة

الجزء ده مش بيتكلم عن Istio مباشرة، لكن بيحكي قصة تطور تصميم الأنظمة من 20 سنة لحد دلوقتي علشان يوصلك لنقطة مهمة جدًا:

ليه انتقلنا من Monolith ← Microservices ← واحتجنا Service Mesh

الشرح التفصيلي

المرحلة الأولى: المشكلة القديمة (قبل 2001)

زمن:

- تطوير البرامج كان بطيء جدًا والمشاريع ممكن تاخذ سنين طويلة جدًا
- المشكلة:
- السوق بيتغير قبل ما البرنامج يخلص وبالتالي فلوس بتضيع

المشكلة الجديدة

لما يكون عندك Application كبير (Monolith) وليكن اسمه **Book Info App** : (متناسش الاسم عشان هايكمل معانا بقية الكورس)

التطبيق كله:

- كود واحد
- Deploy واحد
- كل حاجة مربوطة ببعض

المشاكل:

- لو جزء وقع ← كله يقع
- صعب تعمل Scale لجزء معين
- أي تغيير ← Deploy للتطبيق كله

الحل Microservices

قسمنا التطبيق لـ MicroServices صغيرة: Product → Reviews → Ratings → Details

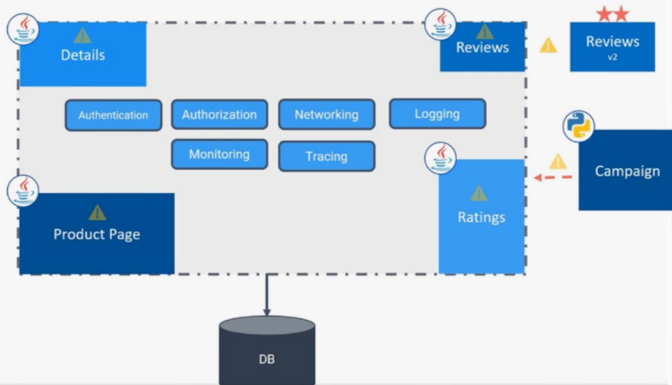
كل Service ✓

- مستقلة
- يتعملها Deploy لوحدها
- يتعملها Scale لوحدها

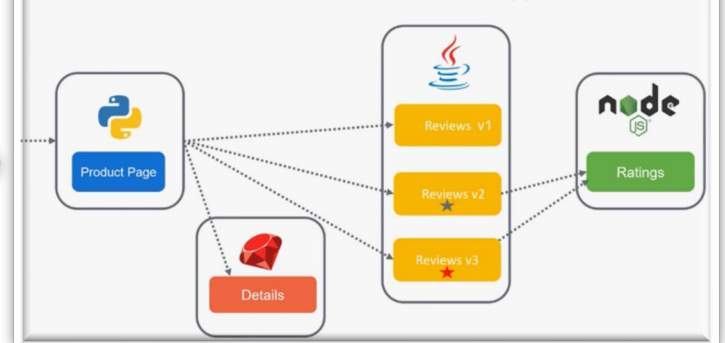
المميزات 3

- سرعة في التطوير
- مرونة
- كل Team يشتغل بلغته

A Monolithic Book Info App



A Microservices Book Info App



4 المشكلة الجديدة (أهم نقطة)

❖ كل Service محتاجة:

- Networking
- Security
- Logging
- Monitoring

✖ فبالتالي كل Service بدأت تكتب ده بنفسها

👉 النتيجة:

- تكرار
- تعقيد
- صعوبة إدارة

5 المشكلة الأكبر

❖ بقى عندك أسئلة زي:

- كل Service هاتوصل للتانية إزاي؟
- الترافيك يتوزع إزاي؟
- Timeout؟ Retry؟ (هايتشرحوا قدام بالتفصيل)
- Security بين الـ services؟

Pros of Microservices

- Scalability
- Faster, smaller releases
- Technology and language agnostic Development lifecycle
- System resiliency and isolation
- Independent and easy to understand services

Cons of Microservices

- Complex Service Networking
- Security
- Observability
- Overload for Traditional Operation Models

💡 الحل (اللي جاي بعده)

❖ Service Mesh (Istio) هو اللي يشيل:

- Networking
 - Security
 - Observability
- ✖ من الـ Services نفسها

📋 ملخص

- Monolith = بسيط بس غير مرن
- Microservices = مرن بس معقد
- التعقيد سببه تكرار نفس الـ logic في كل Service
- ✖ الحل ← Service Mesh يدير ده كله مركزياً

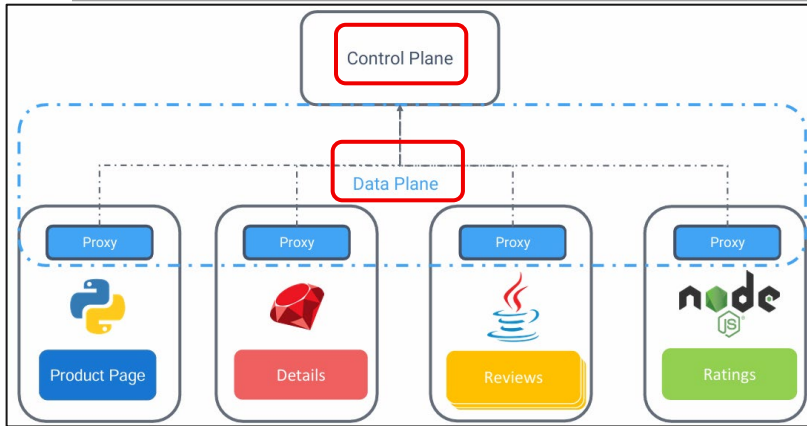
Service Mesh

مقدمة

- ❑ بعد ما شفتنا إن **Microservices** حلت مشاكل كبيرة لكن في نفس الوقت عملت تعقيد في التواصل بين الخدمات
- ❑ فعشان كذا كان لازم يظهر :

❖ حل ينقل الـ **Networking + Security + Observability** برا الكود نفسه

ومن هنا ظهر مفهوم **Service Mesh**



1 الفكرة الأساسية

▽ بدل ما كل **Service** يكون فيها :

- **Retry logic**
- **Security**
- **Logging**
- **Load Balancing**
- ❖ بنشيل كل ده منها

✓ ونحطه في **Proxy** (اسمه **Sidecar**) جنب كل **Service**

2 شكل النظام بقى عامل إزاي؟

```
[Service A] <-> [Proxy A]
[Service B] <-> [Proxy B]
```

❖ كل **Service** معاها **Proxy** ←

💡 المهم:

• الـ **Services** مش بتكلم بعض مباشرة .. ولكن ↓

✓ الـ **Proxies** هي اللي بتكلم بعضها البعض عن طريق الـ **Data Plane** + بيتكلموا كمان مع

Control Plane (اسمه **Server-side Component**)

3 Data Plane vs Control Plane

◆ **Data Plane** (الترافيك)

- يحتوي علي كل الـ **Proxies**
- مسئول عن الترافيك الحقيقي (يعني الاتصال بين الخدمات)

◆ **Control Plane** (الإدارة)

❖ العقل المدبر

• بيقول للـ **Proxies** :

- الترافيك يروح فين (بيتحكم في الترافيك اللي داخل واللي خارج من الـ **Services** بناعتني عن طريق الـ **Proxies**)
- يعمل **Retry** ولا لا + يستخدم أنهي **Rules**

4 النتيجة المهمة

❖ كذا كل الـ **Networking Logic** بقى برا الكود (وهذا هو معنى الـ **Service Mesh**)

✓ يعني:

- الكود ← **Business Logic**
- الشبكة ← **Service Mesh**

❖ بدل ما تعمل ده [Networking - Security – Logging – Monitoring] في كل Service :

✓ الـ Service Mesh يخلي الـ Proxy يعمل ده كله لوحده .. وكمان بيديك :

Security 🛡️

- تشفير تلقائي بين الخدمات (mTLS)

Traffic Control 🔄

- تعمل :

Retry

Timeout

Routing (A/B Testing)

Observability 📊

- تشوف :

مين بيكلم مين

فين المشكلة

Latency فين

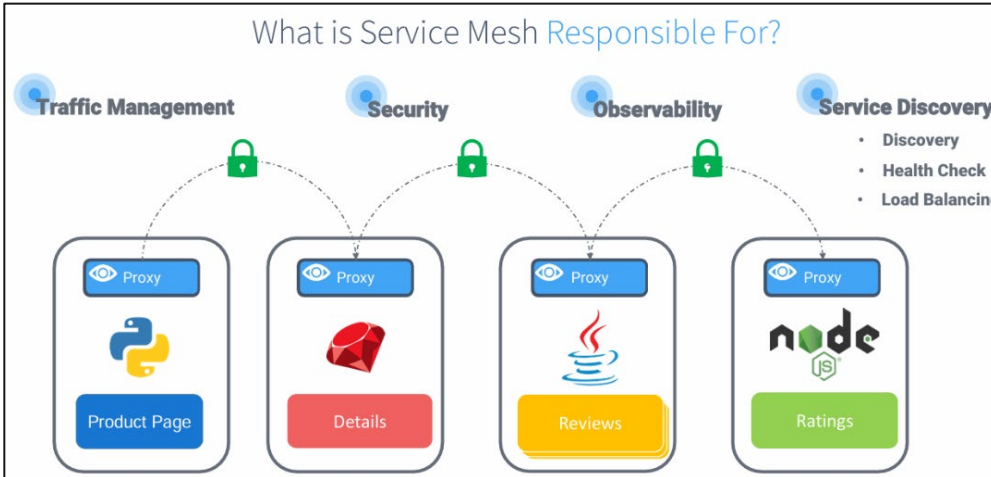
Service Discovery 🔍

- الخدمات تعرف بعض تلقائيًا
- مين شغال ومين واقع

Load Balancing ⚖️

- الترافيك يروح للـ Pods السليمة بس

What is Service Mesh Responsible For?



ملخص سريع 📝

• Service Mesh = Layer زيادة فوق الـ Microservices

• بيستخدم Sidecar Proxy لكل Service

• Data Plane = الترافيك

• Control Plane = الإدارة

💡 Service Mesh يفصل الشبكة عن الكود ويخلي التواصل بين الخدمات مُدار مركزياً

❑ بعد ما فهمنا:

- يعني إيه Service Mesh
- وإنه بيعتمد على:

- Sidecar Proxy
- Data Plane
- Control Plane

❑ دلوقتي زمانك بتسأل إيه الأداة اللي بتطبق الكلام ده فعليًا؟ Istio

1 يعني إيه Istio ؟

❖ Istio هو: Service Mesh جاهز .. وبستخدمه بدل ما تبني واحد بنفسك

✓ بيديك:

- Security
 - Traffic Management
 - Monitoring
- 🚦 وكل ده بدون ما تغير كود التطبيق

2 بيشتغل على إيه؟

- Kubernetes (أساسي)
- وكمان ممكن مع أنظمة ثانية
- 🚦 يعني مناسب للـ Cloud Native Systems

3 مكونات Istio الأساسية

♦ أولاً Data Plane :

- ▽ ده اللي فيه الترافيك الحقيقي
- عبارة عن Envoy Proxies
- كل Pod فيه (Sidecar) Proxy

Envoy ده:

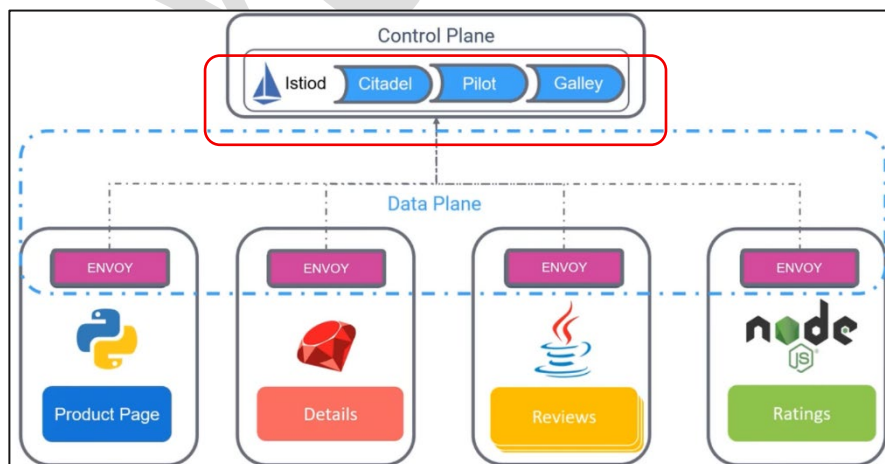
- Proxy قوي جدًا
- Open-source
- High performance
- 🚦 Envoy Proxy + App = Pod

♦ ثانيًا Control Plane : (العقل المدبر)

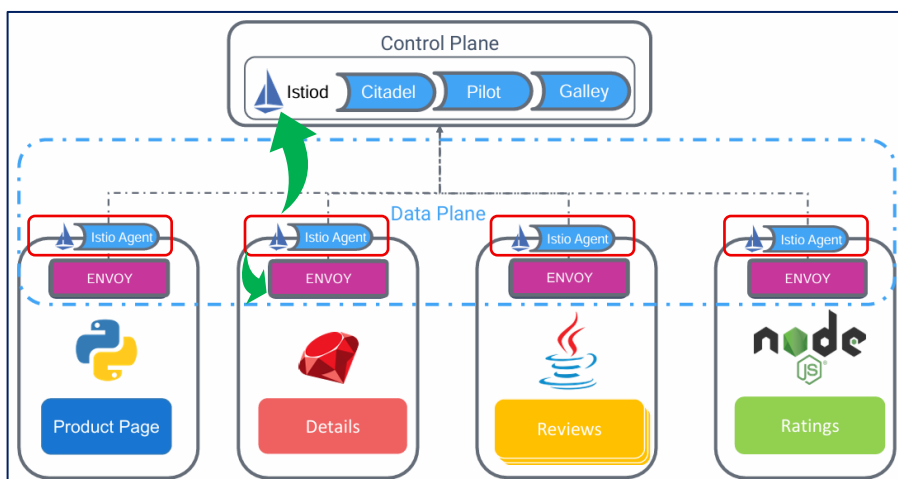
❖ في الأول كان 3 components

- Citadel → Security (Certificates)
- Pilot → Routing & Service Discovery
- Galley → Config Validation

🚦 لكن بعد كده اتجمعوا في حاجة واحدة اسمها: Istiod



4 Istiod بيعمل إيه؟



- ❖ هو المسؤول عن:
- توزيع الـ Config للـ Proxies
- إدارة الترافيك
- إصدار الشهادات (mTLS)
- Service Discovery
- يعني ببساطة:

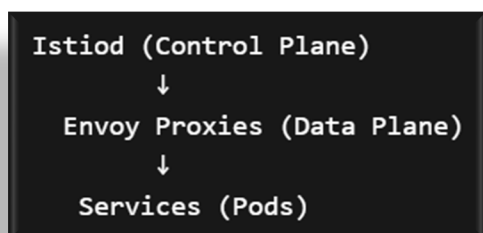
هو اللي بيقول لكل Proxy يعمل إيه

5 Istio Agent (جوه كل Pod)

- ❖ فيه Component صغير كده: **Istio Agent**
- ✓ وظيفته:

- يوصل الـ Config والـ Secrets من Istiod ← للـ Envoy

💡 الفكرة المهمة جدًا



- ❖ بدل ما كل Service تعمل :

- Security
- Routing
- Monitoring

✗ Istio بيشتيل ده كله ✓ ويحطه في:

- Envoy (تنفيذ)
- Istiod (إدارة)

والـ Istio Agent دا الوسيط اللي بينهم

🔗 ملخص سريع

- Service Mesh Implementation = Istio
- Proxy (Data Plane) = Envoy
- Control Plane = Istiod

💡 احفظها كده:

- ❖ Envoy بينفذ الترافيك
- ❖ Istiod بيديره

Installing Istio – Introduction

مقدمة

Installing istioctl

```
>_
$ curl -L https://istio.io/downloadIstio | sh -
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 102 100 102 0 0 123 0 --:--:-- --:--:-- --:--:-- 123
100 4573 100 4573 0 0 3606 0 0:00:01 0:00:01 --:--:-- 3606

Downloading istio-1.9.4 from
https://github.com/istio/istio/releases/download/1.9.4/istio-1.9.4-osx.tar.gz ...
Istio 1.9.4 Download Complete!

Istio has been successfully downloaded into the istio-1.9.4 folder on your system.

Next Steps:
See https://istio.io/latest/docs/setup/install/ to add Istio to your Kubernetes
cluster.

To configure the istioctl client tool for your workstation,
add the /Users/sevikarakose/Downloads/istio-1.9.4/bin directory to your environment
path variable with:
export PATH="$PATH:/Users/sevikarakose/Downloads/istio-1.9.4/bin"

Begin the Istio pre-installation check by running:
istioctl x precheck

Need more information? Visit https://istio.io/latest/docs/setup/install/
```

دي أول خطوة بتعملها عشان ننزل
الـ binaries بتاعة Istio عندك
على الجهاز

```
>_
$ cd istio-1.10.0
$ ls
LICENSE      README.md    bin          manifest.yaml manifests
samples     tools
$ export PATH=$PWD/bin:$PATH
$ istioctl verify-install
0 Istio control planes detected, checking --revision "default" only
0 Istio injectors detected
Error: could not load IstioOperator from cluster: the server could not find the
requested resource. Use --filename
```

محتاجين نخلي الجهاز "يشوف" أمر
istioctl من أي مكان:

بعد ما فهمنا: ☐

• يعني إيه Service Mesh

• واشتغلنا على Istio كـ Concept

• دلوقتي بننقله لمرحلة التنفيذ: ☐

• إزاي ننزل Istio على Kubernetes Cluster ؟

1 طرق تثبيت Istio

• عندك 3 طرق:

1. باستخدام istioctl (الأبسط)

• CLI Tool رسمي من Istio

• سريع وسهل

• مناسب للتجارب والـ Learning

2. باستخدام Istio Operator

• Kubernetes-native

• مناسب للـ Production

• بيديك تحكم أكثر

3. باستخدام Helm

💡 في الكورس ← هنستخدم: istioctl

2 تثبيت Istio فعليًا على الكلاستر :

• الأمر الأساسي: `istioctl install --set profile=demo -y`

• ثم ← `istioctl verify-insatall`

3 يعني إيه Profile ؟

• Profile = إعدادات جاهزة

• أمثلة:

• demo ← للتجارب (فيه كل حاجة)

• default ← متوازن

• minimal ← خفيف

• production ← optimized

💡 هنا استخدمنا demo عشان نتعلم بسهولة

التثبيت على الكلاستر

istio-ingressgateway

Istiod

istio-egressgateway



istio-system

4 حصل إيه جوه الكلستر؟

❖ أول ما نتفخ الأمر 🙌

• هاتعمل Namespace جديد: **istio-system**

• هاینزل Component مهم جدًا: **Istiod**

○ وده زي ما قولنا:

✓ عبارة عن Control Plane فيه (Pilot + Security + Config)

• كمان اتعمل:

○ Ingress Gateway

✓ دخول الترافيك من بره للكلستر

○ Egress Gateway

✓ خروج الترافيك من الكلستر لبرا

• واتعمل:

○ Services

○ Deployments

○ CRDs (Custom Resources)

```
>_
$ istioctl verify-install
1 Istio control planes detected, checking --revision "default" only
✓ Deployment: istio-ingressgateway.istio-system checked successfully
✓ PodDisruptionBudget: istio-ingressgateway.istio-system checked successfully
✓ Role: istio-ingressgateway-sds.istio-system checked successfully
✓ RoleBinding: istio-ingressgateway-sds.istio-system checked successfully
✓ Service: istio-ingressgateway.istio-system checked successfully
✓ ServiceAccount: istio-ingressgateway-service-account.istio-system checked successfully
✓ Deployment: istio-egressgateway.istio-system checked successfully
✓ PodDisruptionBudget: istio-egressgateway.istio-system checked successfully
✓ Role: istio-egressgateway-sds.istio-system checked successfully
✓ RoleBinding: istio-egressgateway-sds.istio-system checked successfully
✓ Service: istio-egressgateway.istio-system checked successfully
✓ ServiceAccount: istio-egressgateway-service-account.istio-system checked successfully
✓ ClusterRole: istiod-istio-system.istio-system checked successfully

✓ EnvoyFilter: tcp-stats-filter-1.9.istio-system checked successfully
Checked 12 custom resource definitions
Checked 3 Istio Deployments
✓ Istio is installed and verified successfully
```

5 بعني إيه CRDs هنا؟

❖ Istio بيضيف Resources جديدة زي:

• VirtualService

• DestinationRule

• Gateway

🔧 دي اللي هنستخدمها عشان نقدر نتحكم في الترافيك

6 التأكد إن كل حاجة شغالة

❖ **istioctl verify-install**

✓ لو تمام ← هيقولك إن كل components شغالة

```
Kubernetes Cluster
├── istio-system
│   ├── istiod (Control Plane)
│   ├── ingress gateway
│   └── egress gateway
└── (بعد كده Pods + Envoy باقي التطبيقات)
```

💡 الصورة الكاملة بعد التثبيت

📄 ملخص سريع

❖ عندك 3 طرق تثبيت أسهلهم (istioctl)

• استخدمنا demo = profile

• اتعمل:

○ istio-system namespace

○ istiod (Control Plane)

○ Gateways

○ CRDs

Demo - Installing Istioctl

مقدمة

❑ قبل ما نثبت Istio ، لازم يكون عندنا:

❖ Kubernetes Cluster شغال

❑ وفي الديمو هنا استخدمنا : Minikube (Local Kubernetes)

الشرح خطوة بخطوة

1 تشغيل Minikube

❖ أبسط أمر: **minikube start**

💡 لو عندك Docker شغال:

• Minikube هيستخدمه كـ Driver

○ وهيشتغل كـ Container

2 مشكلة ممكن تقابلك ⚠ (مهم)

❖ لو على macOS + Docker :

✗ ممكن تشوف Error زي: **ingress add-on is not supported**

السبب ← Docker على Mac عنده Limitations في Networking

3 الحل

❖ بديل Docker Driver ← استخدم VM Driver

• تسمح الكلستر : **minikube delet**

○ وتشفله بـ VM : **minikube start --driver=virtualbox**

➡ كده ingress هيشتغل طبيعي

4 تفعيل Ingress

❖ **minikube addons enable ingress**

✓ مهم عشان بعد كده نستخدم Gateway

5 تحميل Istio

❖ الأمر بيعمل Download لأحدث Version : **curl -L https://istio.io/downloadIstio | sh -**

💡 خذ بالك:

• هيّنزل في الـ current directory

• هيعمل فولدر باسم version (مثلاً istio-1.x.x)

```
istiotraining@local ~$ minikube start
minikube v1.16.0 on Darwin 10.15.7
+ Automatically selected the docker driver. Other choices: hyperkit, virtualbox
Starting control plane node minikube in cluster minikube
Creating docker container (CPUs=2, Memory=1987MB) ...
Preparing Kubernetes v1.20.0 on Docker 20.10.0 ...
  Generating certificates and keys ...
  Booting up control plane ...
  Configuring RBAC rules ...
  Verifying Kubernetes components...
+ Enabled addons: storage-provisioner, default-storageclass
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
istiotraining@local ~$ minikube addons enable ingress
Exiting due to MK_USAGE: Due to networking limitations of driver docker on darwin, ingress addon is not supported.
Alternatively to use this addon you can use a vm-based driver:
```

```
To track the update on this work in progress feature please check:
https://github.com/kubernetes/minikube/issues/7332

istiotraining@local ~$ minikube delete
Deleting "minikube" in docker ...
Deleting container "minikube" ...
Removing /Users/istiotraining/.minikube/machines/minikube ...
Removed all traces of the "minikube" cluster.
istiotraining@local ~$ minikube start --vm=true
minikube v1.16.0 on Darwin 10.15.7
+ Automatically selected the hyperkit driver
Starting control plane node minikube in cluster minikube
Creating hyperkit VM (CPUs=2, Memory=4000MB, Disk=20000MB) ...
Preparing Kubernetes v1.20.0 on Docker 20.10.0 ...
  Generating certificates and keys ...
  Booting up control plane ...
```

```
istiotraining@local ~$ minikube addons enable ingress
Verifying ingress addon...
The 'ingress' addon is enabled
istiotraining@local ~$
```

```
istiotraining@local ~$ curl -L https://istio.io/downloadIstio | sh -
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 182 100 182 0 0 44 0 0:00:02 0:00:02 --:--:-- 44
100 4573 100 4573 0 0 1697 0 0:00:02 0:00:02 --:--:-- 1697

Downloading istio-1.10.3 from https://github.com/istio/istio/releases/download/1.10.3/istio-1.10.3-osx.tar.gz ...
Istio 1.10.3 Download Complete!

Istio has been successfully downloaded into the istio-1.10.3 folder on your system.

Next Steps:
See https://istio.io/latest/docs/setup/install/ to add Istio to your Kubernetes cluster.

To configure the istioctl client tool for your workstation,
add the /Users/istiotraining/istio-1.10.3/bin directory to your environment path variable with:
export PATH=$PATH:/Users/istiotraining/istio-1.10.3/bin

Begin the Istio pre-installation check by running:
istioctl x precheck

Need more information? Visit https://istio.io/latest/docs/setup/install/
istiotraining@local ~$
```

6 الدخول على فولدر Istio

❖ **cd istio-1.x.x**

❖ هتلاقي جواه:

- **/bin** ← فيه **istioctl**
- **/samples** ← تطبيقات جاهزة
- **/tools** ← أدوات إضافية

7 إضافة istioctl للـ PATH

```
export PATH=$PWD/bin:$PATH
```

✓ عشان تقدر تستخدم **istioctl** من أي مكان

8 التأكد إن istioctl شغال

```
istiotraining@local istio-1.10.3 $ istioctl version
no running Istio pods in "istio-system"
1.10.3
```

❖ **istioctl version** ← المفروض يظهر لك **version** ✓

9 التأكد إن Istio مش مثبت

❖ **istioctl verify-install**

• هتلاقي لسا مفيش **Istio Control Plane**

🚩 وده طبيعي لأننا لسه هنثبته

```
istiotraining@local istio-1.10.3 $ istioctl verify-install
0 Istio control planes detected, checking --revision "default" only
error while fetching revision : the server could not find the requested resource
0 Istio injectors detected
Error: could not load IstioOperator from cluster: the server could not find the requested resource. Use --filename
```

10 الخطوة الجاية (المهمة)

❖ دلوقتي بقت جاهز تثبت Istio عن طريق **istioctl install --set profile=demo -y**

```
Local Machine
↓
Minikube (K8s Cluster)
↓
Install Istio
↓
istiod + gateways + CRDs
```

💡 الصورة الكاملة

📄 ملخص سريع

- شغل **Minikube**
- لو فيه مشكلة ← استخدم **VM driver**
- فغل **ingress**
- نزل **Istio**
- ضيف **istioctl** للـ **PATH**
- اتأكد إنه مش مثبت
- بعد كده تبدأ تثبيت **Istio**

🚩 أخيرا :

- ▽ **Minikube** ← البيئة
- ▽ **istioctl** ← أداة التحكم
- ▽ **Istio** ← اللي هنركبه فوق الكلستر

Installing Istio on your Cluster

مقدمة

بعد ما جهزنا: ☐

Minikube •

istioctl •

دلوقتي بننفذ أهم خطوة : تثبيت Istio فعليًا على الكلستر ☐

الشرح خطوة بخطوة

1 تنفيذ التثبيت

❖ الأمر الأساسي: `istioctl install --set profile=demo -y`

اللي بيحصل هنا:

• Istio بيتثبت على الكلستر

○ باستخدام demo profile (مناسب للتجارب)

```
istiotraining@local istio-1.10.3 $ istioctl install --set profile=demo -y
✓ Istio core installed
✓ Istiod installed
✓ Ingress gateways installed
✓ Egress gateways installed
✓ Installation complete
Thank you for installing Istio 1.10. Please take a few minutes to tell us
MPByq7akrYA
```

2 بعد التثبيت ... حصل إيه؟

❖ Istio ضاف حاجات كتير جوه الكلستر

• Components أساسية:

Control Plane ← istiod ○

istio-ingressgateway ○

istio-egressgateway ○

Kubernetes Resources:

❖ Istio بيعمل Integration قوي

مع Kubernetes ، فهتلاقي:

◆ ClusterRoles / RoleBindings

• عشان يدي صلاحيات للـ Components

◆ CRDs (Custom Resources)

• زي:

VirtualService ○

Gateway ○

DestinationRule ○

```
istiotraining@local istio-1.10.3 $ istioctl verify-install
1 Istio control planes detected, checking --revision "default" only
✓ ClusterRole: istiod-istio-system.istio-system checked successfully
✓ ClusterRole: istio-reader-istio-system.istio-system checked successfully
✓ ClusterRoleBinding: istio-reader-istio-system.istio-system checked successfully
✓ ClusterRoleBinding: istiod-istio-system.istio-system checked successfully
✓ Role: istiod-istio-system.istio-system checked successfully
✓ RoleBinding: istiod-istio-system.istio-system checked successfully
✓ ServiceAccount: istio-reader-service-account.istio-system checked successfully
✓ ServiceAccount: istiod-service-account.istio-system checked successfully
✓ ValidatingWebhookConfiguration: istiod-istio-system.istio-system checked successfully
✓ CustomResourceDefinition: destinationrules.networking.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: envoyfilters.networking.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: gateways.networking.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: serviceentries.networking.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: sidecars.networking.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: virtualservices.networking.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: workloadentries.networking.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: workloadgroups.networking.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: authorizationpolicies.security.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: peerauthentications.security.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: requestauthentications.security.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: telemetries.telemetry.istio.io.istio-system checked successfully
✓ CustomResourceDefinition: istiooperators.install.istio.io.istio-system checked successfully
✓ ConfigMap: istio.istio-system checked successfully
✓ Deployment: istiod.istio-system checked successfully
✓ ConfigMap: istio-sidecar-injector.istio-system checked successfully
✓ MutatingWebhookConfiguration: istio-sidecar-injector.istio-system checked successfully
✓ PodDisruptionBudget: istiod.istio-system checked successfully
✓ Service: istiod.istio-system checked successfully
✓ EnvoyFilter: metadata-exchange-1.10.istio-system checked successfully
```

3 التأكد إن كل حاجة شغالة

❖ `istioctl verify-install`

✓ لو كل حاجة تمام:

• هيقولك إن التثبيت ناجح

• ومفيش Errors

❖ دلوقتي الكلستر بقى فيه :

💡 الفكرة المهمة

❖ Istio دلوقتي مثبت ... بس لسه مش شغال على التطبيقات !

ليه؟ 🤔

لأن لازم نفعل:

(الشرح الجاي) **Sidecar Injection (Envoy Proxy)**

🚀 ملخص سريع

- استخدمنا **istioctl install**
- نزل:

Control Plane ○

Gateways ○

CRDs ○

- تأكدنا بـ **verify-install**

🚀 وبكدا الكلستر بقى جاهز



التثبيت = تجهيز البنية

لكن التنفيع الحقيقي = Sidecar Injection

Kubernetes Cluster

```

├── istio-system
│   ├── istiod
│   ├── ingress gateway
│   ├── egress gateway
│   └── CRDs + configs
└── Apps (جاهزة تدخل Istio)
```


Deploying Our First Application on Istio

المقدمة

- ❑ في الجزء ده بنشوف عملياً إزاي نخلي Istio يشتغل فعلاً مع التطبيقات اللي بنعملها على Kubernetes
- ❑ رغم إن Istio متسطب، ممكن تلاقي إنه مش بيأثر على الـ Pods .. وده بسبب حاجة مهمة جداً اسمها: **Sidecar Injection**

الشرح بالتفصيل

مشروح بالتفصيل
في صفحة رقم 6

❖ أول حاجة عملناها إننا نشرنا تطبيق جاهز اسمه **Bookinfo**

- وده **microservices app** فيه كذا **Microservice** زي **productpage** و **reviews** و **ratings** و **Details**

```
> _  
$ kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml  
deployment.apps/details-v1 created  
deployment.apps/ratings-v1 created  
deployment.apps/reviews-v1 created  
deployment.apps/reviews-v2 created  
deployment.apps/reviews-v3 created  
deployment.apps/productpage-v1 created  
...
```

❖ بعد ما عملنا **deploy** ←

❖ رحنا نبص على الـ Pods باستخدام: **kubectl get pods**

- المفروض — بما إن Istio متسطب — إن كل Pod يكون جواه:

○ **Container** الأساسي (التطبيق)

○ **Container** ثاني (**Envoy Proxy** ← اللي هو الـ **sidecar**)

✚ لكن اللي حصل:

هتلاقي إن كل Pod فيه **Container** واحد بس

وده معناه:

❌ Istio مش بيشتغل على الـ Pods دي

Single Container

```
> _  
$ kubectl get pods -A  
NAMESPACE NAME READY STATUS RESTARTS AGE  
default details-v1-66b6955995-zgrd8 1/1 Running 0 6m57s  
default productpage-v1-5d9b4c9849-dckrt 1/1 Running 0 6m57s  
default ratings-v1-fd78f799f-s94cc 1/1 Running 0 6m58s  
default reviews-v1-6549ddccc5-bxv4n 1/1 Running 0 6m58s  
default reviews-v2-76c4865449-2qmhj 1/1 Running 0 6m58s  
default reviews-v3-6b554c875-vbf8v 1/1 Running 0 6m58s
```

طبيب ليه Istio ما يشتغلش؟

❖ لما استخدمنا الأمر: **istioctl analyze**

- ظهر إن المشكلة: ← **Sidecar Injection** مش مفعل على الـ **Namespace**

يعني إيه **Sidecar Injection** ؟

❖ بما أن Istio بيشتغل عن طريق إنه:

- يضيف **Proxy (Envoy)** جنب كل **Container**

○ والبروكسي ده هو اللي يتحكم في:

✓ الترافيك

✓ السيكيوريتي

✓ اللوجز

✓ المونيتورينج

❖ فالكلام دا مش بيحصل تلقائي في كل الـ **Namespaces** .. ولازم تقول له: فعل الـ Istio هنا في الـ ns دي

❖ بنمسح الـ Pods القديمة اتعملت قبل ما نفعل الـ Injection .. وبالتالي مش هيتضافلها sidecar

```
> _
$ kubectl delete -f samples/bookinfo/platform/kube/bookinfo.yaml
service "details" deleted
serviceaccount "bookinfo-details" deleted
service "ratings" deleted
serviceaccount "bookinfo-ratings" deleted
serviceaccount "bookinfo-reviews" deleted
service "productpage" deleted
serviceaccount "bookinfo-productpage" deleted
```

❖ بنفعل الـ Injection على الـ Namespace (مثلاً الـ default)

```
> _
$ kubectl label namespace default istio-injection=enabled
namespace/default labeled
```

• بعد كذا:

○ أي Pod جديد يتعمل في الـ namespace ده
Istio هايضيفله Sidecar Proxy أوتوماتيك

❖ نعمل redeploy تاني

```
> _
$ kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
deployment.apps/details-v1 created
deployment.apps/ratings-v1 created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
deployment.apps/productpage-v1 created
...
```

للتأكد :

لما عملنا `istioctl analyze` ومجبناش أي error
فكدا الدنيا تمام

لما عملت تاني : `kubectl get pods`

• لقينا كل Pod بقى فيه:

○ Container الأساسي

○ Envoy Proxy

يعني: Istio بدأ يشتغل فعلاً 🎉

مثال بسيط يثبت الفكرة 🍌

❖ تخيل عندك Pod فيه : [App Container]

▽ قبل Istio :

• مفيش تحكم في الترافيك

▽ بعد تفعيل Injection : بقا عندك [Envoy Proxy] + [App Container]

• الـ Envoy بقى:

✓ يتحكم في الترافيك - يعمل retries - يطبق security - يسجل metrics

🍌 الملخص

• Istio مش بيشتغل على طول على كل الـ Pods ولازم تفعل الـ Sidecar Injection على الـ Namespace الأول.

• لو مش متفعل: ← مش هيتم إضافة Envoy Proxy ❌

○ الحل: `kubectl label namespace default istio-injection=enabled`

• لازم تعيد إنشاء الـ Pods بعد التفعيل

• بعد كذا:

✓ كل Pod هيبقى فيه sidecar proxy

✓ Istio هايشتغل بالكامل

Demo Deploying Our First Application on Istio

المقدمة

- ❑ في الجزء ده بنعمل خطوة مهمة جدًا:
- ❖ تشغيل الـ Service Mesh فعليًا على التطبيق (Bookinfo)
- ❑ مش بس نعمل Deploy ، لكن نتأكد إن:
 - Istio شغال
 - Sidecar Proxy (Envoy) موجود جوه كل Pod
 - وبالتالي نقدر نتحكم في الترافيك والسيكويريتي بعد كدا

الشرح المتتابع

1 نشر التطبيق (Bookinfo)

- ❖ بدأنا إننا نشرنا التطبيق باستخدام YAML جاهز :

kubectl apply -f bookinfo.yaml

- الملف ده بيعمل:

- Services
- ServiceAccounts
- Deployments

- وكمان بيضمن:

- reviews service (v1, v2, v3) ← 3 versions

2 التأكد إن التطبيق شغال

❖ **kubectl get pods**

- ✓ لو كل الـ Pods في حالة :

- Running

- Ready

- ده معناه إن التطبيق شغال

▽ بس مش شرط يكون معناه إن Istio شغال

3 ملاحظة المشكلة

- ❖ لما بصينا على عدد الـ containers :
- لقينا ان كل Pod فيه Container واحد بس
- ❖ المفروض يكون:
 - App Container
 - Envoy Proxy
- إذا : مفيش Service Mesh لسه

```
istiotraining@local istio-1.10.3 $ kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
service/details created
serviceaccount/bookinfo-details created
deployment.apps/details-v1 created
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created
istiotraining@local istio-1.10.3 $
```

```
istiotraining@local istio-1.10.3 $ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-79f774bdb9-5gqjb         1/1     Running   0           22s
productpage-v1-6b746f74dc-k486v     1/1     Running   0           21s
ratings-v1-b6994bb9-ds6gk           1/1     Running   0           22s
reviews-v1-545db77b95-spv16         1/1     Running   0           21s
reviews-v2-7bf8c9648f-h9php         1/1     Running   0           22s
reviews-v3-84779c7bbc-df5lc         1/1     Running   0           22s
istiotraining@local istio-1.10.3 $
```

```

istiotraining@local istio-1.10.3 $ istioctl analyze
Info [IST0102] (Namespace default) The namespace is not enabled for Istio injection. Run 'kubectl label namespace default istio-injection=enabled' to enable it, or 'kubectl label namespace default istio-injection=disabled' to explicitly mark it as not needing injection.
istiotraining@local istio-1.10.3 $ kubectl label namespace default istio-injection=enabled
namespace/default labeled
istiotraining@local istio-1.10.3 $ kubectl delete deployments --all
deployment.apps "details-v1" deleted
deployment.apps "productpage-v1" deleted
deployment.apps "ratings-v1" deleted
deployment.apps "reviews-v1" deleted
deployment.apps "reviews-v2" deleted
deployment.apps "reviews-v3" deleted
istiotraining@local istio-1.10.3 $ kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
service/details unchanged
serviceaccount/bookinfo-details unchanged
deployment.apps/details-v1 created
service/ratings unchanged
serviceaccount/bookinfo-ratings unchanged
deployment.apps/ratings-v1 created
service/reviews unchanged
serviceaccount/bookinfo-reviews unchanged
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage unchanged
serviceaccount/bookinfo-productpage unchanged
deployment.apps/productpage-v1 created
istiotraining@local istio-1.10.3 $

```

4 تشخيص المشكلة

❖ استخدمنا: **istioctl analyze**

• وطلع: **default namespace is not enabled for Istio injection**
يعني:

✗ Namespace (default) مش مفعل عليه Istio

5 الحل — تفعيل Sidecar Injection

❖ بنضيف label على الـ namespace : **kubectl label namespace default istio-injection=enabled**
كدا قلنا لـ Istio :
أي Pod هنا ← ضيفه Envoy Proxy

6 إعادة إنشاء الـ Pods

❖ الـ Pods القديمة : مش هيتضاف لها sidecar
وبالتالي لازم نعيد إنشائها: ▽

- **kubectl delete -f bookinfo.yaml**
- **kubectl apply -f bookinfo.yaml**

```

istiotraining@local istio-1.10.3 $ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-79f774bdb9-9qvrh        2/2     Running   0           14s
productpage-v1-6b746f74dc-8q972    2/2     Running   0           14s
ratings-v1-b6994bb9-ds6gk          1/1     Terminating   0           97s
ratings-v1-b6994bb9-ww2bk          2/2     Running   0           14s
reviews-v1-545db77b95-msd68        2/2     Running   0           14s
reviews-v2-7bf8c9648f-jnnr4        2/2     Running   0           14s
reviews-v3-84779c7bbc-6dkvx        2/2     Running   0           14s

```

7 التأكد إن Istio اشتغل

❖ **kubectl get pods**

✓ هنلاقي كل Pod بقى فيه : 2 Containers

كدا ← Envoy Sidecar اتضاف فعلاً

8 التحقق النهائي

❖ **istioctl analyze**

✓ النتيجة:

- مفيش Errors
- مفيش مشاكل

الرسالة : **"Everything is OK"**

```

istiotraining@local istio-1.10.3 $ istioctl analyze
✓ No validation issues found when analyzing namespace: default.
istiotraining@local istio-1.10.3 $

```

- نشرنا تطبيق Bookinfo
 - التطبيق اشتغل لكن بدون Istio
- السبب:
 - ✗ Sidecar Injection مش مفعل
- الحل : **kubectl label namespace default istio-injection=enabled**
 - لازم نعيد إنشاء الـ Pods
- بعد كذا:
 - Envoy Proxy اتضاف لكل Pod
 - Service Mesh بقي شغال
- وبالتالي بقينا نقدر نبدأ نستخدم Features زي :
 - Routing
 - Security (mTLS)
 - Monitoring

Visualizing Service Mesh with Kiali

المقدمة

❑ بعد ما عملنا Service Mesh باستخدام Istio ، بقى عندنا مشكلة جديدة:

❖ إزاي نشوف فعليًا اللي بيحصل بين الـ microservices ؟

• لأن:

- فيه Services كتير
- Requests رايحة جاية بينهم
- Traffic rules و retries و timeouts شغالة

➡ هنا ببيجي دور أداة مهمة جدًا:

Kiali ← **Dashboard** بتخليك تشوف وتفهم الـ Service Mesh



1) ليه محتاجين Kiali ؟

❖ في الـ microservices :

- كل Service بتكلم Service تانية
- فيه routing rules
- فيه load balancing
- فيه failures ممكن تحصل

! من غير visualization : هتبقى "ماشى أعمى"

2) Kiali هو إيه ؟

❖ ببساطة: Web UI (واجهة رسومية) بتعرض لك كل حاجة بتحصل في Istio

❖ تقدر من خلالها:

- تشوف كل الـ services
- تشوف الترافيك بينهم
- تفهم المشاكل بسهولة

3) Kiali بيعمل إيه عمليًا ؟

1. Visualization (أهم حاجة) ❖

❖ بيوريك شكل الـ system كـ graph : productpage ← reviews ← ratings

✓ كل Service

✓ وكل Connection بينهم

✓ والترافيك ماشى إزاي

2. فهم الـ Traffic Rules ومراقبته (Observability) ❖

❖ تقدر تشوف:

• Request rate (عدد الريكويستات)

• Latency (التأخير)

• Errors

❖ يعني:

✓ تعرف مين بطيء؟

✓ مين بيكسر؟

✓ مين عليه ضغط؟

3. Validation : ◇

- ❖ يتأكد إن الـ config يتاع Istio صح
- ❖ ينبهك لو فيه مشاكل

4. Wizards (ميزة قوية) : ◇

- ❖ بدل ما تكتب YAML بإيدك:
- Kiali يساعدك تعمل بشكل أسهل :
- Routing rules
- Traffic splitting

4. Ⓛيه لازم نعمل Traffic ؟

- ❖ Kiali بيعتمد على الـ requests الحقيقية
- ❖ لو مفيش traffic :
- ✗ مش هتشوف حاجة
- ❖ عشان كدا:
- لازم تولد test traffic (مثلاً curl أو فتح التطبيق)

مثال بسيط

- | | |
|---|---|
| <ul style="list-style-type: none">❖ مع Kiali : ✓❖ تشوف Graph زي كدا: User ← productpage ← reviews ← ratings ✓❖ وتحت كل link :• latency• error rate• traffic %🚦 وبالتالي كل حاجة بقت واضحة 🚦 | <ul style="list-style-type: none">❖ بدون Kiali : ✗• عندك services :○ productpage○ reviews○ ratings• ومش عارف :○ مين بيكلم مين○ فين المشكلة |
|---|---|

الملخص

- Kiali = Dashboard لعرض الـ Service Mesh
- بيشتغل مع Istio
- بيوريك:
- ✓ العلاقات بين services
- ✓ الترافيك
- ✓ الأداء (latency / errors)
- ببساعدك:
- ✓ تفهم النظام
- ✓ تكتشف المشاكل
- ✓ تبني configs بسهولة
- لازم يكون فيه traffic عشان تشوف البيانات

Demo Installing Kiali

المقدمة

❑ بعد ما فهمنا إن Kiali بيخاينا نشوف الـ Service Mesh بشكل visual ، دلوقتي بنخس في خطوة عملية:

❖ إزاي نثبت Kiali ونشغله ونبدأ نشوف الـ microservices ؟

❑ وفي نفس الوقت هنفهم إن Kiali مش لوحده... ده جزء من Stack كامل للـ Observability

```
>
$ kubectl apply -f samples/addons
$ kubectl rollout status deployment/kiali -n istio-system

monitoringdashboard.monitoring.kiali.io/springboot-jvm created
monitoringdashboard.monitoring.kiali.io/springboot-tomcat created
monitoringdashboard.monitoring.kiali.io/thorntail created
monitoringdashboard.monitoring.kiali.io/tomcat created
monitoringdashboard.monitoring.kiali.io/vertx-client created
monitoringdashboard.monitoring.kiali.io/vertx-eventbus created
monitoringdashboard.monitoring.kiali.io/vertx-jvm created
monitoringdashboard.monitoring.kiali.io/vertx-pool created
monitoringdashboard.monitoring.kiali.io/vertx-server created
serviceaccount/prometheus created
configmap/prometheus created
clusterrole.rbac.authorization.k8s.io/prometheus created
clusterrolebinding.rbac.authorization.k8s.io/prometheus created
service/prometheus created
deployment.apps/prometheus created
```

1 تثبيت Kiali (ومعاه أدوات تانية)

❖ لما بنثبت Kiali من الـ samples : `kubectl apply -f samples/addons`

- مش بس Kiali اللي بيتثبت
- دا بينزل معاه أدوات تانية مهمة:

- Prometheus ← يجمع Metrics
- Grafana ← يعرض Metrics بشكل Charts
- Jaeger ← يعمل Tracing للـ Requests
- Kiali ← يعمل Visualization للـ Mesh

👉 دول مع بعض بيكونوا Observability Stack

2 التأكد إن Kiali اشتغل

❖ `kubectl get deployments -n istio-system`

- لازم تلاقي Kiali :
- Running ○
- Ready ○

```
istiotraining@local istio-1.10.3 $ kubectl -n istio-system get svc kiali
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kiali     ClusterIP   10.102.20.34  <none>         20001/TCP,9090/TCP 93s
```

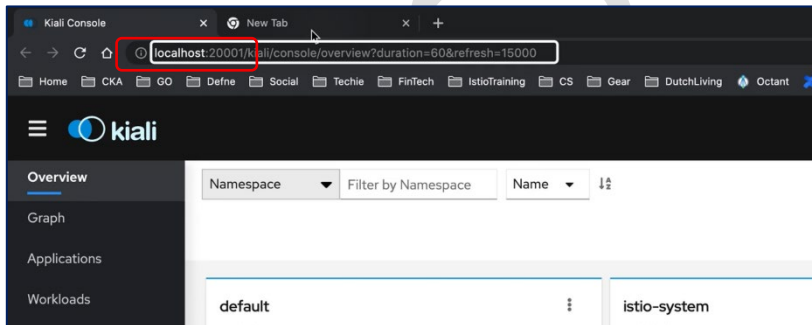
3 تشغيل Dashboard

❖ `istioctl dashboard kiali`

- ده:

- يفتح Browser
- ويوديكي على Kiali UI

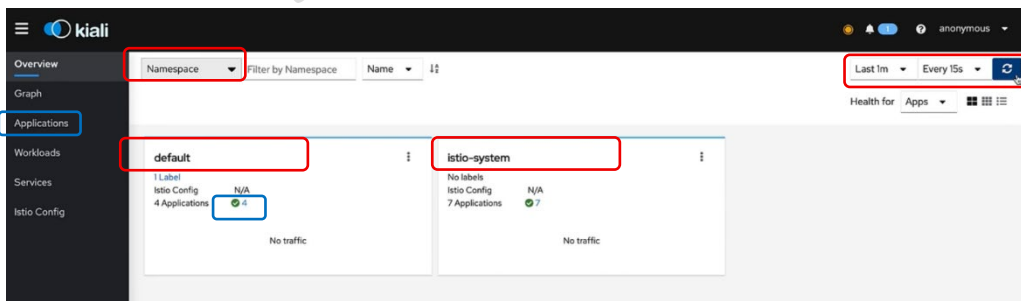
• عادة بيشتغل على: `localhost:20001`

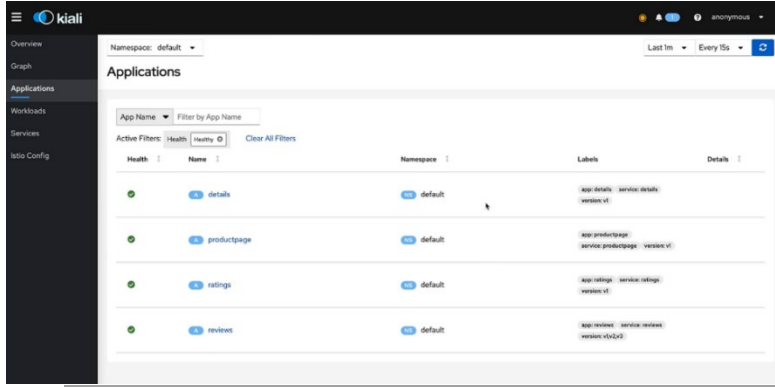


4 استكشاف Kiali UI

Overview

- بيعرض كل الـ Namespaces
- وعدد التطبيقات في كل واحد



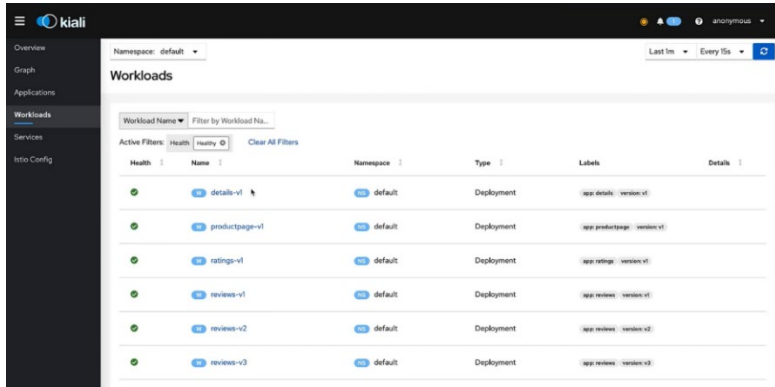


Applications ♦

هتلاقي:

- productpage
- reviews
- ratings
- details

👉 دول microservices بتوع Bookinfo

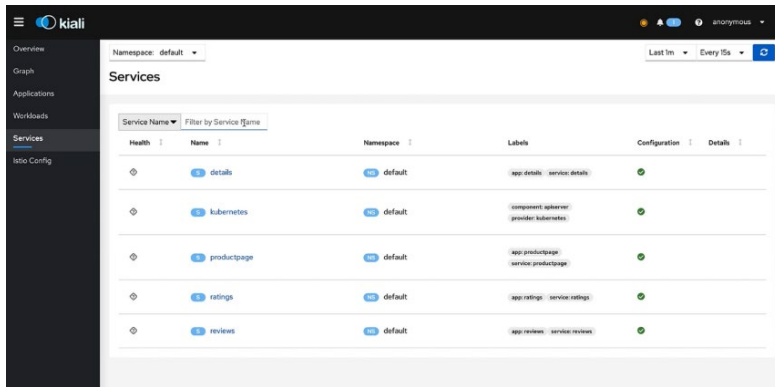


Workloads ♦

- بيعرض ال Deployments / Pods

Services ♦

- بيعرض Kubernetes Services (ممکن يتأخر شوية لحد ما يظهر)



Istio Config ♦

❖ فاضي في الأول !

- ليه؟

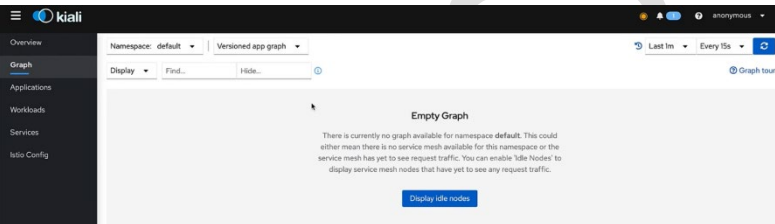
👉 لأننا لسه ما عملناش :

VirtualService ○

DestinationRule ○

Graph (أهم جزء) ♦

❖ في الأول : فاضي علشان مفيش Traffic لسه



5 ⬢ فيه لازم Traffic أصلاً ؟

❖ Kiali بيعتمد على ← ال metrics اللي جاية

من Prometheus

❖ ولو مفيش requests :

❌ مفيش data

مفيش graph

6 ⬢ توليد Traffic

❖ مثلاً لو فتحت التطبيق في ال browser أو عملت Curl

- ساعتها هاتبدأ تشوف ال Graph

👉 مثال : `while true; do curl -s http://$INGRESS_HOST:$INGRESS_PORT/productpage > /dev/null; done`

- دا بيعمل Request بشكل مستمر ومن غير مايطبع output

🚩 النتيجة : Kiali هياظهر Graph واضح .. وبالتالي تقدر تلاحظ ال behavior المختلفة وال errors بسهولة.

❖ هتشوف حاجة زي: User → productpage → reviews → ratings

❖ ومع كل link :

- request rate
- latency
- errors

👉 كأنك شايف السيستم شغال قدامك 🤖 live

🧠 الملخص

- ثبتنا Kiali + أدوات (Prometheus / Grafana / Jaeger) Observability
- شغلنا Dashboard على port 20001
- استكشفنا :
 - Applications
 - Workloads
 - Services
 - Graph
- Graph كان فاضي لأن مفيش traffic
- بعد ما عملنا traffic :
- ✓ ظهر شكل الـ Service Mesh
- ✓ بقينا نشوف الترافيك والـ metrics

Demo Create Traffic Into Your Mesh

1 الفكرة الأساسية

- ☐ تشغيل التطبيق من برا الـ cluster + توليد traffic + مراقبته على Kiali
- ☐ وكمان نشوف إيه اللي يحصل لما service تقع

2 الخطوات الأساسية

إنشاء Gateway

❖ تشغيل: `kubectl apply -f bookinfo-gateway.yaml`

- يعمل الآتي :
 - يفتح access من برا الـ cluster
 - يوصل الترافيك للـ services

```
istiotraining@local ~ $ cd istio-1.10.3/
istiotraining@local istio-1.10.3 $ kubectl apply -f samples/bookinfo/networking/bookinfo-gateway.yaml
gateway.networking.istio.io/bookinfo-gateway created
virtualservice.networking.istio.io/bookinfo created
```

التأكد من الـ config

❖ `istioctl analyze`

- الهدف : ← يتأكد إن مفيش مشاكل في الـ mesh

نحجب IP + Port

❖ `minikube ip` + `kubectl -n istio-system get svc istio-ingressgateway`

بنستخدمهم عشان نوصل للتطبيق اللي عالـ Local cluster

```
istiotraining@local istio-1.10.3 $ minikube ip
192.168.64.20
istiotraining@local istio-1.10.3 $ export INGRESS_HOST=$(minikube ip)
istiotraining@local istio-1.10.3 $ echo $INGRESS_HOST
192.168.64.20
istiotraining@local istio-1.10.3 $ export INGRESS_PORT=$(kubectl -n istio-system get service istio-ingressgateway -o jsonpath='{.spec.ports[?(@.name=="http2")].nodePort}')
istiotraining@local istio-1.10.3 $ echo $INGRESS_PORT
31285
istiotraining@local istio-1.10.3 $ curl "http://$INGRESS_HOST:$INGRESS_PORT/productpage"
istiotraining@local istio-1.10.3 $ echo "http://$INGRESS_HOST:$INGRESS_PORT/productpage"
http://192.168.64.26:31950/productpage
istiotraining@local istio-1.10.3 $
```

تجيب الـ IP بتاع الكلاستر

تخزين الـ IP في variable اسمه INGRESS_HOST بدل ماقعد اكتبه كل شوية

تجيب الـ NodePort الخاص بالـ HTTP بتاع الـ istio-Ingress Gateway

تبع request للتطبيق علي Port + IP وكمان route:/productpage (احدي الـ endpoint الخاصة بالـ API)

نفس الفكرة بس أسرع شوية

تجربة التطبيق

❖ `curl http://IP:PORT/productpage`

- النتيجة:

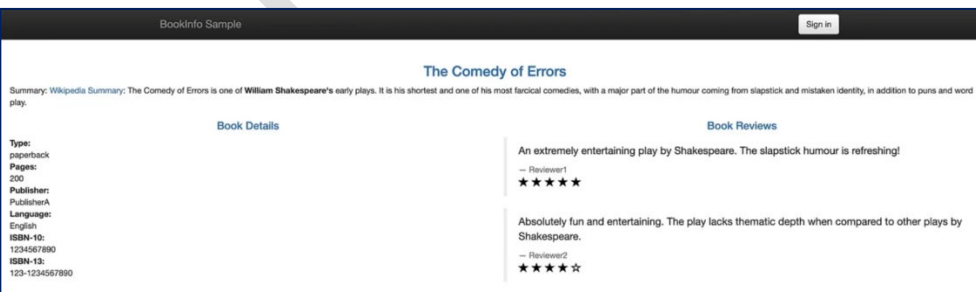
✓ يرجع HTML

✓ التطبيق شغال

- أو من browser :

✓ تشوف الصفحة

✓ النجوم بتتغير (3 versions من reviews)



ne



•

•

3

2.

●

نو

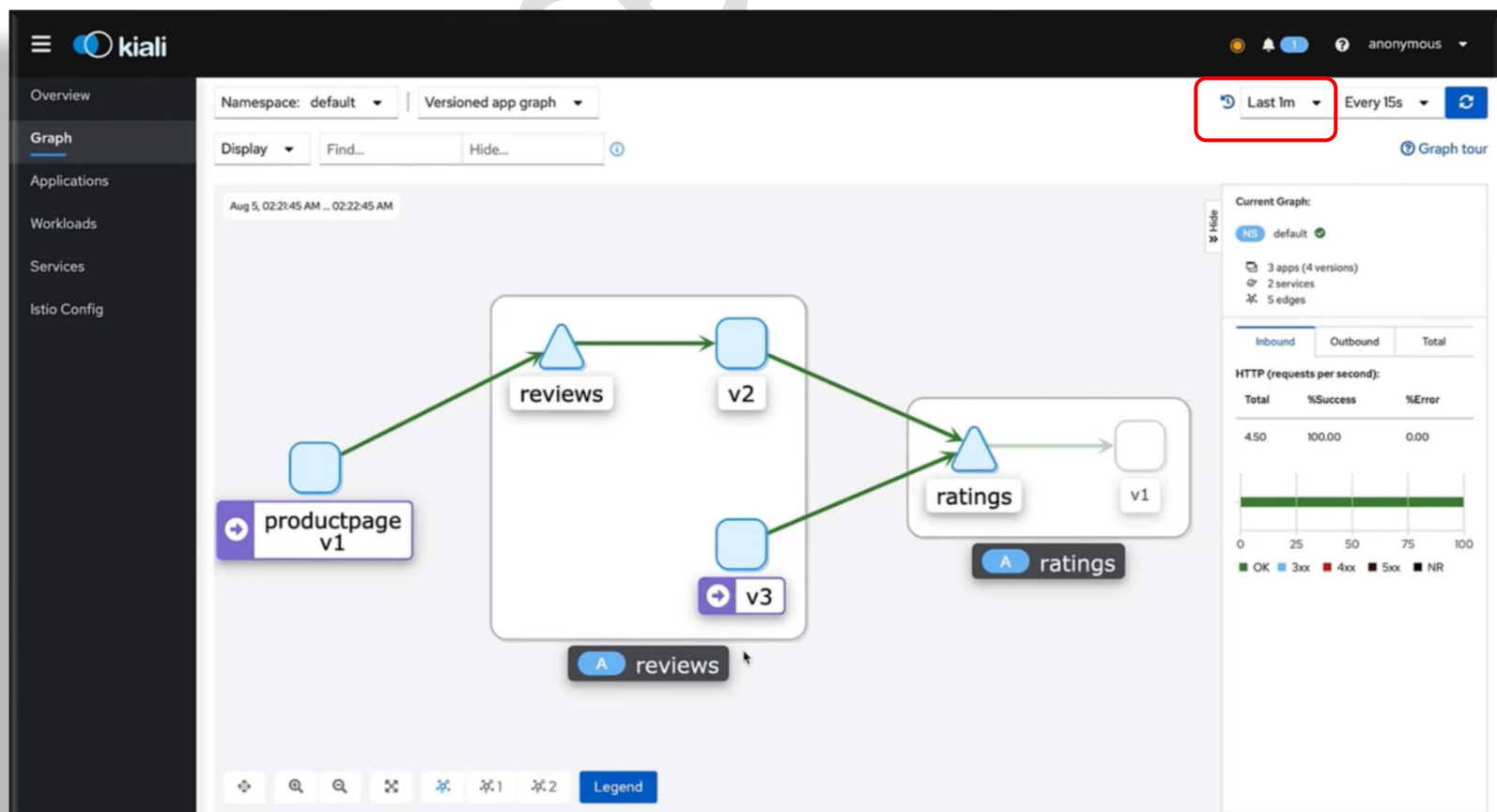
نو

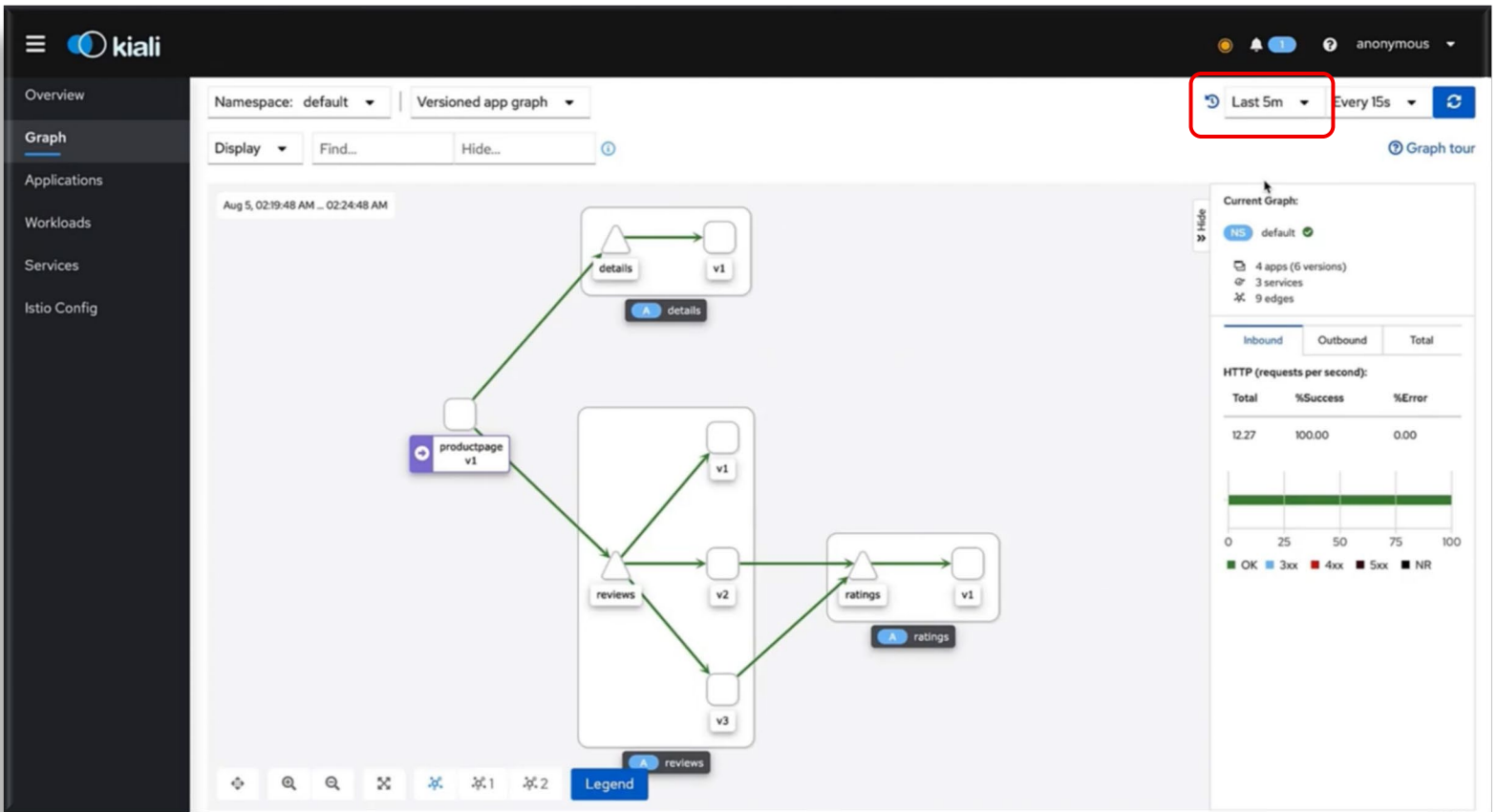
•

•

●

4



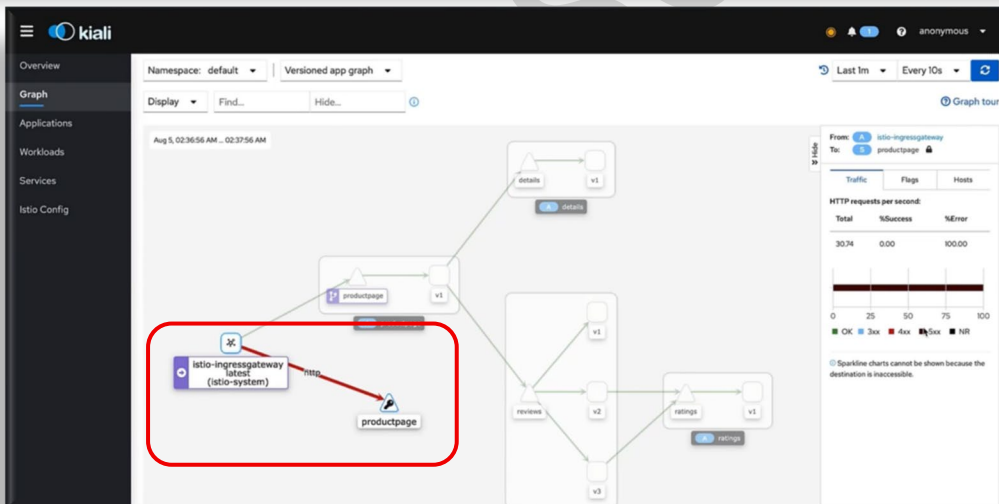


5 ♦ نبوظ الدنيا (اختبار)

kubectl delete deployment productpage

```
istiotraining@local istio-1.10.3 $ kubectl delete deployments/productpage-v1
deployment.apps "productpage-v1" deleted
istiotraining@local istio-1.10.3 $
```

6 ♦ النتيجة بعد الـ failure



في Kiali :

- productpage اختفى
- traffic كله fail
- 500 = errors
- باقي services :
- مفيش traffic
- health unknown (?)

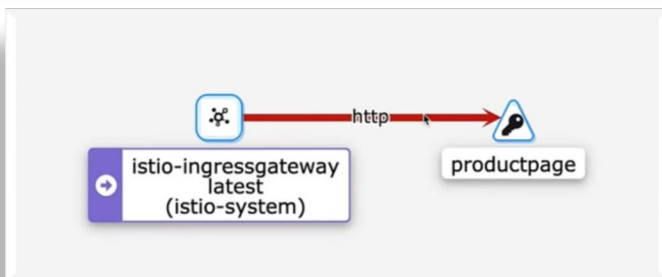
الخلاصة

• Gateway = دخول الترافيك

• curl loop = users وهميين

• Kiali = مراقبة

• delete deployment = اختبار failure

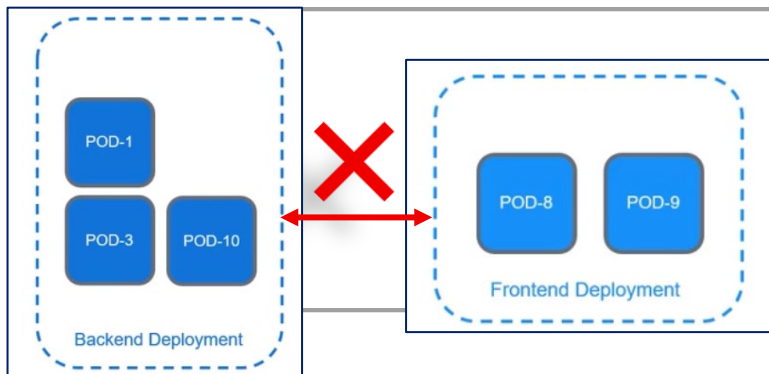


Kubernetes Services

1 ♦ الفكرة الأساسية

❑ الـ Pods بتغير الـ IPs بتاعها (ephemeral)
وبالتالي انت محتاج طريقة ثابتة تخلي services تكلم بعض
الحل Kubernetes Service

2 ♦ المشكلة



❑ كل Pod ليه IP .. ولكن :
الـ Pods لما بتتمسح وتتعمل تاني ← الـ IP بيتغير
يعني :

frontend مش هابقي عارف يوصل للـ backend

3 ♦ الحل (Service)

❑ Service = abstraction فوق الـ Pods

- يدي IP ثابت .. وبالتالي يوصل للـ Pods اللي وراه وهكذا
بيعتمد على Labels عشان يختار الـ Pods

4 ♦ مثال عملي



❑ عندك : backend + frontend
• فبدل ما frontend يكلم Pod مباشرة علي
الـ IP 10.1.2.3 (اللي هو ممكن يتغير)
• فبيكلمه علي ← backend-service

Service بقى هو اللي يودي الترافيك للـ Pods الصح

6 ♦ النتيجة

❑ stable endpoint = Service

- وبالتالي حتي لو الـ IPs بتاع الـ Pods إتغير ← فالـ communication يفضل شغال

الخلاصة

- Pod = dynamic (IP بيتغير)
- Service = (IP / DNS) ثابتين
- Service هو اللي بيعمل routing للـ Pods

- ❑ في Kubernetes ، التطبيق مش لازم يكون عبارة عن Container واحد بس بيعمل كل حاجة.
- ❑ بدل كده، ممكن يتقسم لأكثر من Container داخل نفس الـ Pod علشان نفصل المهام عن بعض ونخلي النظام أسهل في الإدارة والتطوير.
- ❑ من أهم الأنماط (Patterns) المستخدمة هنا هو Sidecar Pattern، وده شائع جدًا في أنظمة الـ Cloud Native و DevOps

الشرح التفصيلي

1. يعني إيه Sidecar في Kubernetes ؟

- ❑ الـ Sidecar Container هو Container إضافي بيشتغل جنب الـ Main Container داخل نفس الـ Pod
 - الـ Main Container : بيحتوي على منطق التطبيق الأساسي (Business Logic)
 - الـ Sidecar Container : بيقوم بمهام مساعدة للتطبيق .
- ❑ الاتنين بيشتغلوا مع بعض داخل نفس الـ Pod ، وبالتالي:
 - يشاركون نفس الـ Network
 - ممكن يشاركون نفس الـ Storage
 - وبيعيشوا نفس دورة الحياة (يشتغلوا ويقفوا مع بعض)

2. ليه اسمه Sidecar ؟

- ❑ الاسم جاي من الموتوسيكلات اللي بيبقى فيها كرسي صغير جنب الموتوسيكل (ويبقى اسمه Sidecar)
 - الموتوسيكل = التطبيق الأساسي (Main Container)
 - الـ Sidecar = الجزء المساعد
- 🚩 يعني هو مش الأساسي، لكنه بيدعمه ويكمل وظيفته.

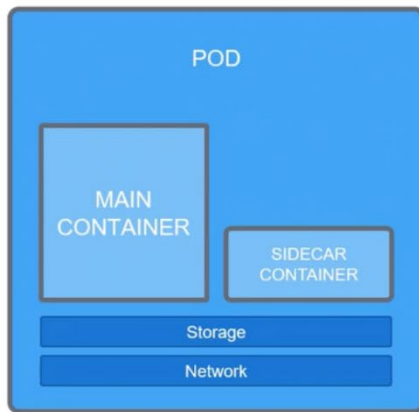
3. الـ Sidecar بيعمل إيه؟

- ❑ الـ Sidecar بيقوم بمهام مساعدة زي:
 - 📁 جمع ورفع اللوجات (Logs) زي Fluentd
 - 📊 المراقبة (Monitoring & Metrics)
 - 📡 البروكسي والـ Traffic Routing زي Envoy
 - 📁 التعامل مع الملفات أو تحميلها
 - 🛡️ مهام أمنية زي الشهادات (Certificates)
- 🚩 الفكرة إنه يشيل المهام دي من التطبيق الأساسي علشان يبقى أبسط.

4. مثال عملي في Kubernetes

- ❑ لو عندنا Pod فيه:
 - Container 1: Nginx
 - ده التطبيق الأساسي اللي بيستقبل الطلبات
 - Container 2: Fluentd
 - ده الـ Sidecar
 - بيجمع اللوجات من Nginx وبيعته لسيرفر مركزي

🚩 كده بدل ما نضيف كود logging داخل التطبيق نفسه، خليه في Container منفصل.



```
pod.yaml

containers:
- name: nginx-container
  image: nginx
  volumeMounts:
  - name: shared-data
    mountPath: /usr/share/nginx/html
- name: sidecar-container
  image: fluent/fluentd
  volumeMounts:
  - name: shared-data
    mountPath: /pod-data
```

مميزات استخدام Sidecar

- فصل المسؤوليات (Separation of Concerns)
- إعادة استخدام نفس الـ Sidecar في تطبيقات مختلفة
- سهولة الصيانة والتعديل
- مرونة أعلى في التصميم
- تحسين قابلية التوسع (Scalability)

ملخص

- الـ Sidecar Container هو Container مساعد بيشتغل جنب التطبيق الأساسي داخل نفس الـ Pod
- يشارك نفس الشبكة والـ storage مع التطبيق .
- بيقوم بمهام زي اللوجات، المراقبة، أو البروكسي .
- مثال : Nginx التطبيق + Fluentd (Sidecar للـ logs)
- الهدف منه هو تحسين التنظيم، وتقليل تعقيد التطبيق الأساسي.

- ❑ في عالم Service Mesh و Microservices، نحتاج أداة تتحكم في حركة المرور (Traffic) بين الخدمات بدون ما نكتب كل الـ Logic ده داخل التطبيق نفسه.
- ❑ هنا بيظهر دور Envoy Proxy، وهو واحد من أشهر وأقوى الـ Proxies المستخدمة في الأنظمة الحديثة.
- ❑ الفكرة الأساسية: بدل ما التطبيق يتعامل مع كل حاجة (أمان، إعادة المحاولة، تتبع الطلبات)، بنفصل ده في خدمة خارجية اسمها Proxy — وأشهر مثال عليها هو Envoy

الشرح التفصيلي

1. يعني إيه Proxy ؟

- ❖ الـ Proxy هو وسيط بين المستخدم (Client) والتطبيق (Server)
- يعني بدل ما المستخدم يتواصل مباشرة مع التطبيق .. هيتواصل مع الـ Proxy (المستخدم ← Proxy ← التطبيق)
- ❖ الـ Proxy هنا بيكون مسؤول عن تمرير الطلبات وإضافة وظائف إضافية.

2. المشكلة قبل استخدام الـ Proxy

❖ التطبيق نفسه كان بيبقى مسؤول عن حاجات كتير غير الـ Business Logic ، زي:

- تأمين الاتصال باستخدام TLS
- التحقق من الهوية (Authentication)
- إعادة المحاولة (Retry requests)
- تتبع الطلبات (Monitoring / Logging)
- كل ده كان بيخلي الكود معقد وصعب الصيانة .

الحل: فصل المسؤوليات باستخدام Proxy

❖ بدل ما نحط كل ده داخل التطبيق:

- نخلي التطبيق يركز فقط على Business Logic
- وننقل المهام المساعدة إلى Proxy خارجي
- وبكده التطبيق يبقى أبسط وأخف وأسهل في التطوير .

ما هو Envoy ؟

❖ Envoy هو Proxy مفتوح المصدر (Open Source) مصمم خصيصًا للأنظمة الحديثة مثل:

- Microservices
- Distributed Systems
- Service Mesh

3. Envoy داخل (Sidecar Pattern) Kubernetes

❖ Envoy غالبًا بيشتغل كـ Sidecar Container جنب التطبيق داخل نفس الـ Pod

❖ يعني:

• التطبيق الأساسي ← Business Logic

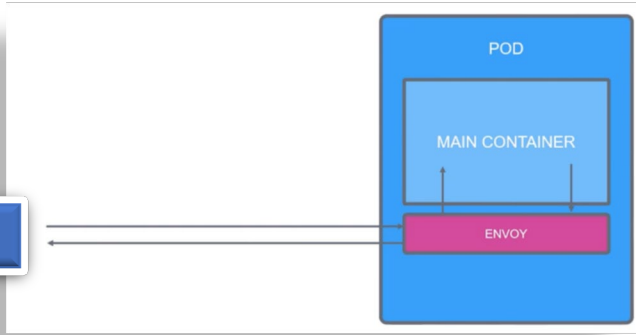
• Envoy ← إدارة الترافيك

❖ وبالتالي أي Request :

• يدخل الأول على Envoy

• Envoy يقرر يمرره أو يعالجه أو يعيد المحاولة أو يراقبه

✓ ثم يوصله للتطبيق



4. ليه Envoy مهم في Service Mesh ؟

❖ لأنه بيقدم قدرات قوية جدًا زي:

• Load Balancing

• TLS encryption

• Retry & Failover

• Observability (Logging & Metrics)

• Traffic control بين الخدمات

• وعشان كده أغلب حلول الـ Service Mesh زي Istio بتعتمد عليه.

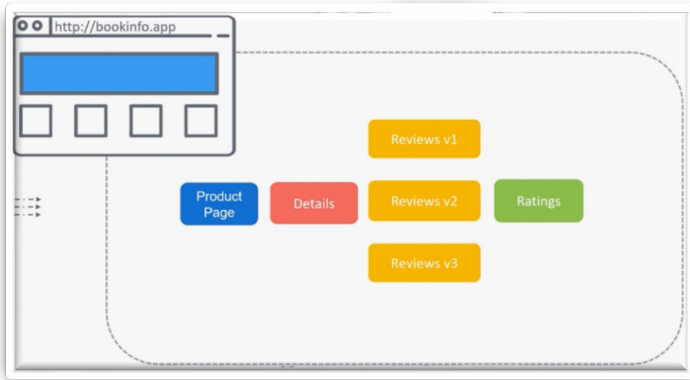
ملخص

- الـ Proxy هو وسيط بين المستخدم والتطبيق .
- الهدف منه فصل المهام المساعدة عن التطبيق الأساسي .
- Envoy هو Proxy قوي مفتوح المصدر مخصص للـ Microservices
- بيشتغل غالبًا كـ Sidecar داخل Kubernetes
- بيستخدم في Service Mesh لإدارة الترافيك، الأمان، والمراقبة.

Gateways

مقدمة

- ❑ في عالم Kubernetes مع Istio Service Mesh ، بعد ما ننشر التطبيقات والخدمات داخل الكلاستر، يظهر سؤال مهم جدًا: ❖ إزاي نخلي المستخدمين من خارج الكلاستر يوصلوا للتطبيقات دي؟
- ❑ هنا بييجي دور مفهوم مهم اسمه Gateway في Istio ، والتي يعتبر نقطة دخول (Entry Point) لـ Service Mesh



الشرح التفصيلي

1. المشكلة الأساسية

❖ عندنا تطبيقات شغالة داخل Kubernetes (Microservices) ، زي:

- Product Page Service
- Reviews Service
- Details Service

والسيناريو :

▽ إننا عندنا مستخدم خارجي عاوز يوصل لـ Product Page Service اللي جوا الكلاستر

▽ ومثلا محتاجين إنه لما يخش بالـ URL ده <http://bookinfo.app> فالمفروض يشوف صفحة الـ Product Service

✓ فبالتالي إحنا محتاجين طريقة نوجه الترافيك من بره الكلاستر لجوا الخدمات دي.

2. الحل في Kubernetes : Ingress

❖ في Kubernetes التقليدي بنستخدم:

- Ingress
- و Ingress Controller (زي NGINX)
- ❖ وده بيعمل:

• يستقبل الترافيك من بره الكلاستر

• يطبق Rules

• يوجه الطلبات لـ Services المناسبة

مثال:

bookinfo.app → Product Service



bookinfo-ingress.yaml

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: ingress
spec:
  rules:
  - host: bookinfo.app
    http:
      paths:
      - path: /
        backend:
          serviceName: productpage
          servicePort: 8000
```

3. الحل في Istio : Gateway

❖ Istio بيقدم بديل أقوى من Ingress اسمه : Istio Gateway

❖ الـ Gateway هو:

- نقطة دخول وخروج للترافيك من وإلى Service Mesh

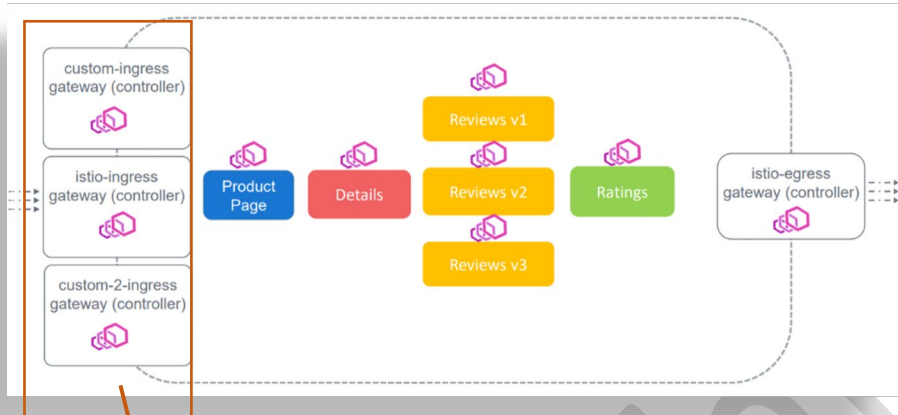
4. الفرق بين Ingress و Istio Gateway

Gateway (Istio)	Ingress (Kubernetes)
Routing + Security + Monitoring	بسيط Routing Rules
مبني على Envoy Proxy	Controller زي NGINX
متكامل مع Service Mesh	محدود في المزايا

5. Istio Ingress & Egress Gateway

❖ عند تثبيت Istio في الكلاستر، سيتم إنشاء:

- Istio Ingress Gateway ← يدخل الترافيك (Inbound)
- Istio Egress Gateway ← يخرج الترافيك (Outbound)



6. دور Envoy في الـ Gateway

❖ الـ Gateways في Istio مبنية على Envoy Proxy

❖ لكن الفرق:

- Sidecar Envoy ← جنب كل Pod
- Gateway Envoy ← عند حافة (Edge) الـ Service Mesh

يعني:

- مش داخل التطبيق
- لكنه نقطة دخول وخروج رئيسية

7. إزاي الترافيك يتم إدارته ؟

❖ لما المستخدم يدخل: <http://bookinfo.app> .. إيه اللي بيحصل ؟!؟

❖ الخطوات:

1. الترافيك يدخل على Istio Ingress Gateway
2. الـ Gateway يقرأ الـ rules
3. يحدد الوجهة المناسبة (Service)
4. يمرر الطلب داخل الـ Mesh

8. تعريف Gateway في Istio

❖ بننشئ Object اسمه Gateway :

• بيحدد :

- البروتوكول (HTTP)
- البورت (80)
- الدومين (bookinfo.app)

• ويحدد أي Ingress Controller نستخدمه
مثال:

- اختيار default Istio Ingress Gateway
- باستخدام selector (labels)

This is a Gateway Object to catch all traffic coming through Istio Ingress Gateway Controller and route all traffic at hostname bookinfo.app to our product page service

```
bookinfo-gateway.yaml
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: bookinfo-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "bookinfo.app"

$ kubectl apply -f bookinfo-gateway.yaml
gateway.networking.istio.io/bookinfo-gateway created
```

لاحظ ان في الصورة دي كان موجود
3 ingress controller واحنا عاوزين نختار
الـ default فعشان كذا استخدمنا الـ Selector




```
>_
$ kubectl get gateway
NAME      AGE
bookinfo-gateway 9d

$ kubectl describe gateway bookinfo-gateway
Name:      bookinfo-gateway
Namespace: default
Labels:    <none>
Annotations: API Version: networking.istio.io/v1beta1
Kind:      Gateway
...
Spec:
Selector:
Istio: ingressgateway
Servers:
Hosts:
-
Port:
Name:      http
Number:    80
Protocol:  HTTP
Events:    <none>
```

9. عرض وإدارة الـ Gateways

- ❖ أوامر Kubernetes :
 - عرض كل الـ Gateways : **kubectl get gateway**
 - تفاصيل Gateway معين : **kubectl describe gateway <name>**

10. مهم جدًا

- ❖ الـ Gateway لا يحدد أين يذهب الترافيك فعليًا
▽ هو فقط: ← يستقبل الترافيك ويطبق entry rules
- ❖ لكن التوجيه الحقيقي داخل الخدمات يتم عن طريق : Virtual Service (مشرح تاليا)

ملخص

- الـ Gateway في Istio هو نقطة دخول الترافيك إلى Service Mesh
- بديل أقوى من Kubernetes Ingress
- مبني على Envoy Proxy
- فيه Ingress Gateway (دخول) و Egress Gateway (خروج)
- يحدد كيف يدخل الترافيك لكن لا يحدد الوجهة النهائية .
- التوجيه الحقيقي يتم عبر Virtual Services

❑ بعد ما عرفنا إن Istio Gateway هو باب دخول الترافيك إلى الـ Service Mesh ، ببيجي سؤال مهم جدًا:

- ❖ بعد ما الترافيك يدخل، يروح فين بالضبط؟
- ❖ وإزاي نتحكم في توزيعه بين الخدمات أو حتى بين نسخ مختلفة من نفس الخدمة؟
- ➦ هنا بيظهر دور أهم Component في Istio اسمه Virtual Service

الشرح التفصيلي

1. المشكلة الأساسية

❖ عندنا تطبيق شغال داخل Istio ، وأي مستخدم يدخل علي الدومين: <http://bookinfo.app> ده ← هايوصل للـ Gateway

❖ لكن المشكلة: بعد كده نوديه فين؟

- product page ؟
- reviews service ؟
- API ؟
- static files ؟

➦ لازم نحدد Rules واضحة للتوجيه .

2. الحل Virtual Service :

❖ الـ Virtual Service هو كيان في Istio مسؤول عن:

- تحديد Routing Rules للترافيك داخل الـ Service Mesh

❖ يعني ببساطة:

- يقول للترافيك يروح فين بعد الـ Gateway
- بيحدد طريقة توزيع الطلبات بين الخدمات

3. وظائف Virtual Service

❖ الـ Virtual Service بيقدر يعمل حاجات قوية جدًا:

- Routing حسب URL (Path-based routing)
- Routing حسب Hostname
- توزيع الترافيك بين نسخ مختلفة (Versions)
- دعم Canary & A/B Testing
- دعم Regex و Prefix matching

4. مثال عملي (bookinfo.app)

❖ عندنا: <http://bookinfo.app/productPage>

❖ الهدف:

- أي طلب على /productPage ← يروح لـ Product Service
- أي طلب على /static ← نفس الخدمة
- /api أو /login ← نفس الوجهة حسب القاعدة

➦ كل ده بيتحدد داخل Virtual Service

5. Virtual Service بيشتغل إزاي؟

❖ التدفق بيكون كده:

1. المستخدم يدخل على bookinfo.app
2. الطلب يوصل لـ Istio Gateway
3. Gateway يمرره لـ Virtual Service
4. Virtual Service يطبق Rules
5. يوجه الطلب للخدمة المناسبة داخل الـ Mesh

6. ربطه بالـ Gateway

❖ داخل تعريف الـ Virtual Service بنقول:

- يشتغل على Host معين : bookinfo.app
- مرتبط بـ Gateway معين : bookinfo-gateway

❖ يعني:

- مش أي Gateway
- لازم نحدد أنه Gateway بالظبط

7. Routing Rules (Match)

❖ فيه طريقتين أساسيتين للمطابقة:

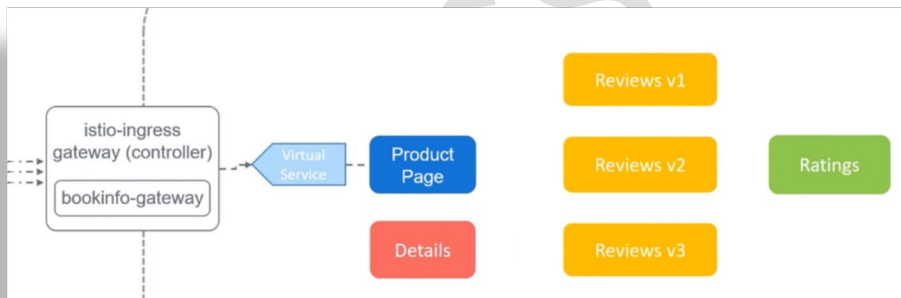
Exact Match ◆

○ يعني المسار لازم يكون مطابق 100% ← /productPage

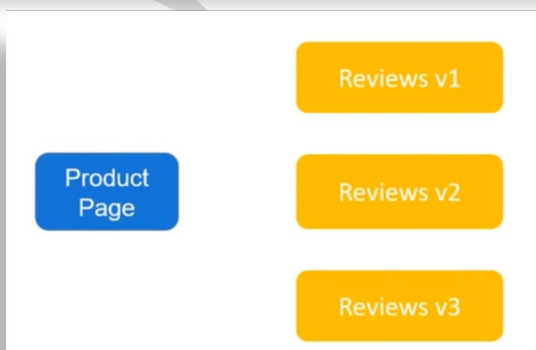
Prefix Match ◆

○ أي حاجة تبدأ بـ prefix معين /static/* OR /api/v1/*

إمشي معايا السيناريو ده عشان تفهم



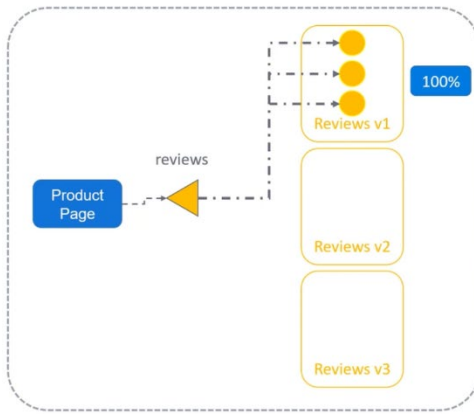
دلوقت الترافيك كله ببيجي من الـ bookinfo-gateway وبيخس عالـ Virtual Service



ودلوقت احنا عاوزين ان مثلا الـ Product page يكلم الـ Reviews V1 (يعني بالبلدي كدا الـ review service دي كان فيها فقط Review v1 فقط وبالتالي الترافيك كله كان بيطلع عليها .. طاب انا دلوقت ضيفت فيهم كذا versions فازاي نقدر ندخل النسخ الجديدة بشكل تدريجي، ونبيعت لها نسبة صغيرة بس من الترافيك عشان نختبرها قبل ما نوجه كل الترافيك ليها ؟!

```
review-v1-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: reviews-v1
spec:
  replicas: 3
  <...>
  template:
    metadata:
      labels:
        app: reviews
        version: v1
```

```
review-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: reviews
spec:
  ports:
    - port: 9080
      name: http
  selector:
    app: reviews
```

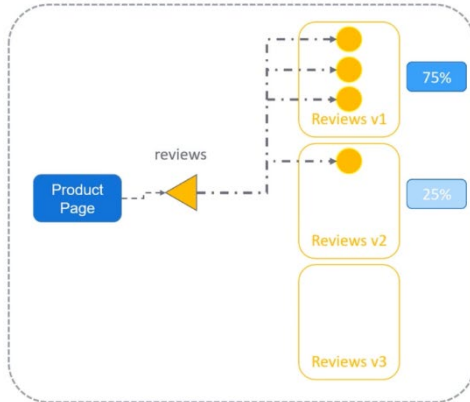


الأول هاتشوف الموضوع كان بيتم ازاي من غير Istio :

- 1- Will deploy the review microservice as a Deployment (v1) ClusterIP Service وافترض مثلا ان عندهم 3 ReplicaSets ويستخدموا Labels: reviews + version:v1 وكمان الـ 3 Pods هاتوزع 100% من الترافيك عاله الـ Product page وبالتالي فالـ Reviews V1 في

```
review-v2-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: reviews-v2
spec:
  replicas: 1
  <...>
  template:
    metadata:
      labels:
        app: reviews
        version: v1
```

```
review-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: reviews
spec:
  ports:
    - port: 9080
      name: http
  selector:
    app: reviews
```

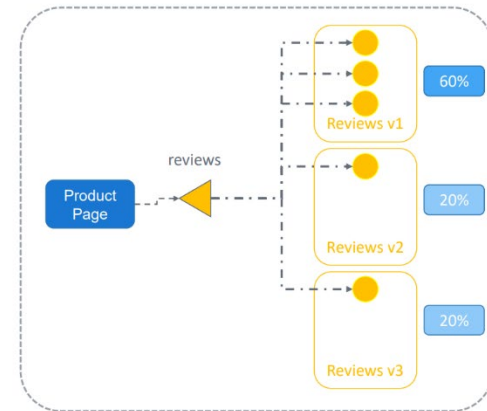


الأول هاتشوف الموضوع كان بيتم ازاي من غير Istio :

- 2- When we deploy a new version of the review service (version2) هاتعملها برود As a Deployment .. بس هاتبدأ بإتينا نحط جواها 1Pod وهاتستخدم نفس الـ Labels فهتلاقى ان الـ Service حصل إن الـ EndPoints بقا عندها 4 (اللي هما 4 Pods) وبالتالي هاتوزع الترافيك عليهم بالتساوي

```
review-v3-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: reviews-v3
spec:
  replicas: 1
  <...>
  template:
    metadata:
      labels:
        app: reviews
        version: v1
```

```
review-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: reviews
spec:
  ports:
    - port: 9080
      name: http
  selector:
    app: reviews
```



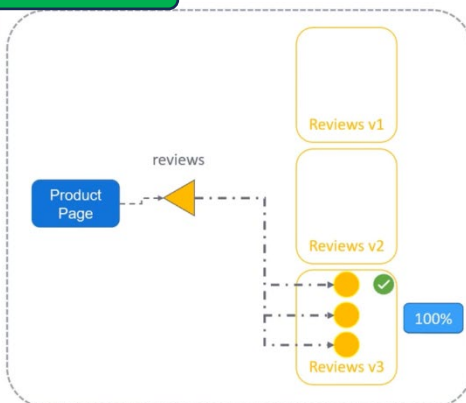
الأول هاتشوف الموضوع كان بيتم ازاي من غير Istio :

- 3- When we deploy a new version of the review service (version3) هاتعملها برود As a Deployment .. بس هاتبدأ بجواها 1Pod وهاتستخدم نفس الـ Labels فهتلاقى ان الـ Service حصل إن الـ EndPoints بقا عندها 5 (اللي هما 5 Pods) وبالتالي هاتوزع الترافيك عليهم بالتساوي

U can consider this as an A/B Test

```
$ kubectl scale deployment reviews-v3 --replicas=3
deployment.apps/reviews-v3 scaled
$ kubectl scale deployment reviews-v2 --replicas=0
deployment.apps/reviews-v2 scaled
$ kubectl scale deployment reviews-v1 --replicas=0
deployment.apps/reviews-v1 scaled
```

```
review-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: reviews
spec:
  ports:
    - port: 9080
      name: http
  selector:
    app: reviews
```



المشكلة اللي ظهرت ::

خلينا نقول مثلا إننا اكتشفنا إن النسخة v3 هي الأفضل، لأن المستخدمين حبوها أكثر. فبالتالي قررنا نوجه كل الترافيك ليها.

فعلشان نعمل كذا :

- 1- We first scale up replicas of v3 to 3 pods وبالتالي بقا عندنا ترافيك متساوي بيروح لـ V1 & V3
- 2- Then we scale down replicas on v1 & v2 وبالتالي بقا الترافيك كله رايح لـ V3 ... حوار وتعقيد مش كذا !!!

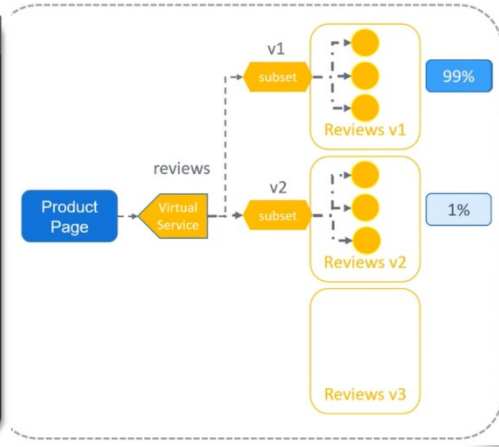
يعنى فى K8S لازم تزود أو تقلل عدد الـ Pods عشان تتحكم فى الترافيك

حل مع Istio (الأقوى)

خُليًا نقول مثلاً إنني عاوز أوزع 99% من الترافيك علي V1 و 1% بس علي V2 .. وبغض النظر عن عدد الـ Pods اللي في كل version

- 1- Create a virtual service (instead of a service) and will call it reviews
- 2- Then define 2 Destination Rules for traffic Distribution → subset (v1 & v2)
- 3- Set a weight for each subset
 - ✓ 99% for v1
 - ✓ 1% for v2

وبالتالي حتي لو فيها أكثر من Pod ← فهاخد برده 1% من الترافيك



```
review-service.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
        weight: 99
    - destination:
        host: reviews
        subset: v2
        weight: 1
```

8. مفهوم Subsets

❖ الـ Subset هو طريقة لتجميع Pods بناءً على Labels :

- v1 subset
- v2 subset
- v3 subset

وبعدين Istio يستخدمهم في توزيع الترافيك. 🚀

ميزة ضخمة في Istio

❖ حتى لو: v2 ← عنده 1 Pod أو 10 Pods

- النسبة تفضل ثابتة (1%)

❖ يعني:

- التحكم في الترافيك مستقل عن عدد الـ Pods
- مرونة أعلى في التجارب (Testing & Canary Releases)

ملخص

- Virtual Service هو قلب Routing في Istio
- يحدد الترافيك يروح فين بعد الـ Gateway
- يدعم Routing حسب URL, Host, Prefix, Regex
- بيخليك تتحكم في توزيع الترافيك بين نسخ الخدمات .
- ممكن تعمل Canary Deployments بدقة (زي 99% / 1%) .
- بي فصل بين عدد الـ Pods وتوزيع الترافيك .
- مع Destination Rules + Subsets ، تقدر تدير نسخ التطبيقات بسهولة .

Destination Rules

مقدمة

- ❑ بعد ما فهمنا Virtual Service في Istio وإنه مسؤول عن توجيه الترافيك (Routing) بين الخدمات، بيبجي سؤال مهم جدًا:
 - ❖ إزاي نحدد النسخ المختلفة من نفس الخدمة (V1, V2, V3) ؟
 - ❖ وإزاي نتحكم في طريقة توزيع الترافيك بينهم؟
- 🚩 هنا بيظهر دور عنصر مهم جدًا في Istio اسمه Destination Rule

الشرح التفصيلي

يعني إيه Destination Rule ؟

- ❖ الـ Destination Rule هو إعداد في Istio **بيطبق بعد ما الترافيك يوصل للخدمة.**
- ❖ يعني:
 - Virtual Service ← بيحدد "الترافيك يروح فين"
 - Destination Rule ← بيحدد "يتم التعامل مع الترافيك إزاي بعد ما يوصل"

Destination Rules apply routing policies after the traffic is routed to a specific service

مفهوم Subsets (المهم جدًا)

- ❖ الـ Subset هو طريقة لتقسيم نفس الخدمة إلى **versions مختلفة**. ثم يستخدمهم في توجيه الترافيك بالمقدار اللي انا عاوزه
 - مثلاً خدمة Reviews :
 - v1 ← النسخة الأولى
 - v2 ← النسخة الثانية
- 🚩 في Kubernetes ، كل نسخة ليها Pods مختلفة لكن نفس الخدمة.
- 🚩 في Istio ، بنعرفهم كـ Subsets داخل Destination Rule

إزاي بنعرف Subsets ؟

- ❖ بننشئ Destination Rule ونحدد:

- اسم الخدمة (Host) ← reviews
- Subsets :

- v1 (مرتبط بـ label version: v1)
- v2 (مرتبط بـ label version: v2)

🚩 يعني Istio بيستخدم labels على الـ Pods عشان يميز بين النسخ.

review-destination.yaml

```
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews-destination
spec:
  host: reviews
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
```

review-v1-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: reviews-v1
spec:
  replicas: 3
  <...>
  template:
    metadata:
      labels:
        app: reviews
        version: v1
```


❖ لو عندك Virtual Service يقول:

• $v2 \leftarrow 1\%$ & $v1 \leftarrow 99\%$

❖ يبقى الـ Destination Rule هو اللي هيقول إن :

• $v1$ = هي كل الـ Pods اللي عليها label: v1

• $v2$ = هي كل الـ Pods اللي عليها label: v2

وبالتالي بدون Destination Rule ، Istio مش هيعرف يعني إيه $v1$ أو $v2$ أصلاً.

Load Balancing داخل Destination Rule

❑ Istio (عن طريق Envoy) بيستخدم Load Balancing افتراضي:

• Round Robin (توزيع بالتساوي)

• Random

• Least Connection / Pass-through (الأقل ضغطاً)

❑ بس أنت ممكن تخصص Load Balancing لكل Subset عن طريق إضافة Traffic Policy

❖ ميزة قوية جداً إنك تقدر تعمل :

• $v1 \leftarrow$ Round Robin

• $v2 \leftarrow$ Random

❖ يعني كل نسخة ليها طريقة توزيع مختلفة حسب الحاجة.

Traffic Policy ▾

❖ داخل Destination Rule تقدر تضيف:

• Load Balancer policy

• TLS settings

• mTLS (Mutual TLS)

❖ مثلاً:

• Simple TLS ← اتصال مشفر عادي

• mTLS ← تبادل شهادات بين الخدمات (أمان أعلى)

Simple: PASSTHROUGH >>

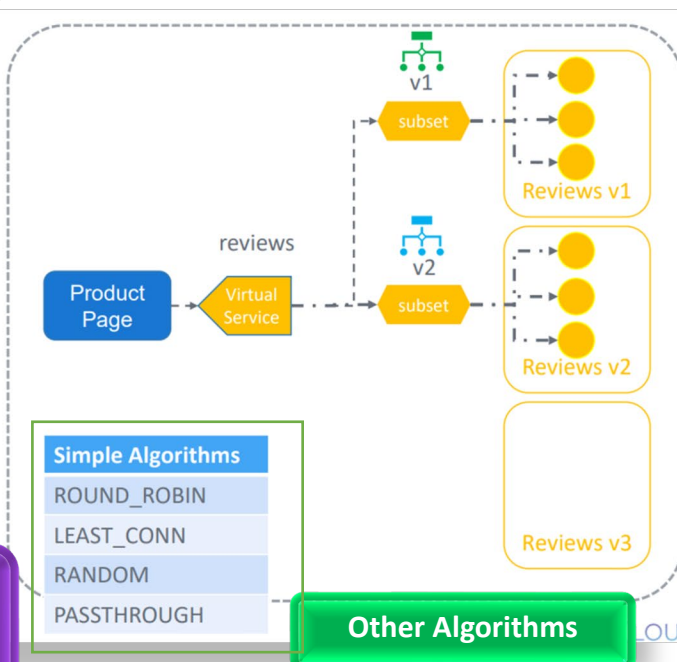
Will route traffic to the host that has fewer active request

review-destination.yaml

```
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews-destination
spec:
  host: reviews
  trafficPolicy:
    loadBalancer:
      simple: PASSTHROUGH
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
      trafficPolicy:
        loadBalancer:
          simple: RANDOM
```

What if u want to Overwrite a diff LB Policy for specific subsets ?

يعني كمثال احنا عاوزين نعط Subset V2 Random LB Policy .. وبالتالي هاتعرف الـ Policy علي مستوي Subset v2

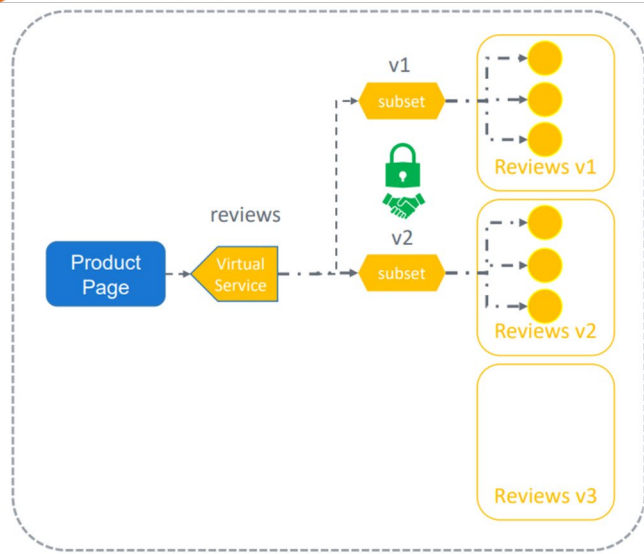


To configure a client to use TLS, specify simple TLS or to configure mutual TLS

- 1- Set the mode to mutual
- 2- Provide the path to the certificate

مثال آخر :-

```
review-destination.yaml
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
  name: reviews-destination
spec:
  host: reviews.default.svc.cluster.local
  trafficPolicy:
    tls:
      mode: MUTUAL
      clientCertificate: /myclientcert.pem
      privateKey: /client_private_key.pem
      caCertificates: /rootcacerts.pem
```



نقطة مهمة جدًا (Host name)

- ❖ في Destination Rule كنا كاتبين: (host: reviews)
- ❖ لكن في مشكلة:

• لو استخدمت الاسم المختصر (short name)

○ Istio ممكن يفترض namespace غلط

الحل الأفضل

- ❖ استخدم الاسم الكامل (FQDN) : reviews.default.svc.cluster.local
- ❖ ليه؟

▪ يمنع أخطاء الـ namespace

▪ يضمن توجيه صحيح للخدمة

▪ أفضل practice في production

An Example With Fully Qualified Domain Names

```
virtual-service.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: productpage
  namespace: default
spec:
  hosts:
  - productpage.prod.svc.cluster.local
  http:
  - timeout: 5s
    route:
    - destination:
        host: productpage.prod.svc.cluster.local
```

ملخص

- Destination Rule يحدد كيفية التعامل مع الترافيك بعد وصوله للخدمة .
- بيعرّف Subsets (زي v1 و v2) باستخدام labels
- Virtual Service يوجّه الترافيك، و Destination Rule يفسر النسخ .
- يتيح التحكم في :

○ Load Balancing (Round Robin / Random / Pass-through)

○ إعدادات TLS / mTLS

- مهم جدًا استخدام FQDN بدل الاسم المختصر لتجنب أخطاء namespace

Demo – Gateways

مقدمة

❑ في الجزء ده من التدريب بنشوف تجربة عملية (Demo) على Istio Gateway و Virtual Service
❖ هدفها إننا نفهم إزاي نخلي التطبيقات داخل Kubernetes متاحة للمستخدمين من خارج الكلاستر
باستخدام hostname حقيقي زي : <http://bookinfo.app>

الشرح التفصيلي

مراجعة ال Gateway الموجود

❖ أول خطوة في الديمو:

• فيه Gateway موجود بالفعل تم تطبيقه من

ملفات ال samples

○ هذا ال Gateway كان فيه wildcard host '*'

❖ يعني بيقبل أي host تقريباً

لكن المطلوب في الديمو:

نخليه يعتمد على hostname محدد بدل wildcard

تجهيز التطبيق مرة أخرى

❖ قبل ما نكمل:

• تم إعادة إنشاء product page deployment

○ باستخدام ملف **bookinfo.yaml**

وذا للتأكد إن كل الخدمات شغالة بشكل صحيح

```
istiotraining@local istio-1.10.3 $ kubectl edit gateway
Please edit the object below. Lines beginning with a '#' will be ignored,
and an empty file will abort the edit. If an error occurs while saving this file will be
reopened with the relevant failures.
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1alpha3","kind":"Gateway","metadata":{"annotations":{},"name":"bookinfo-gateway","namespace":"default"},"spec":{"selector":{"istio":"ingressgateway"},"servers":[{"hosts":["*"],"port":{"name":"http","number":80,"protocol":"HTTP"}}]}}
  creationTimestamp: "2021-10-09T22:19:31Z"
  generation: 7
  name: bookinfo-gateway
  namespace: default
  resourceVersion: "18550"
  uid: 4686ab5f-31f8-463c-940f-72577c65364a
spec:
  selector:
    istio: ingressgateway
  servers:
  - hosts:
    - "*"
    port:
      name: http
      number: 80
      protocol: HTTP
```

```
istiotraining@local istio-1.10.3 $ kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
service/details unchanged
serviceaccount/bookinfo-details unchanged
deployment.apps/details-v1 unchanged
service/ratings unchanged
serviceaccount/bookinfo-ratings unchanged
deployment.apps/ratings-v1 unchanged
service/reviews unchanged
serviceaccount/bookinfo-reviews unchanged
deployment.apps/reviews-v1 unchanged
deployment.apps/reviews-v2 unchanged
deployment.apps/reviews-v3 unchanged
service/productpage unchanged
serviceaccount/bookinfo-productpage unchanged
deployment.apps/productpage-v1 created
istiotraining@local istio-1.10.3 $
```

```
istiotraining@local istio-1.10.3 $ kubectl get gateway
NAME                AGE
bookinfo-gateway    102s
istiotraining@local istio-1.10.3 $ kubectl delete gateway bookinfo-gateway
gateway.networking.istio.io "bookinfo-gateway" deleted
```

```
istiotraining@local istio-1.10.3 $ kubectl apply -f - <<EOF
> apiVersion: networking.istio.io/v1alpha3
> kind: Gateway
> metadata:
>   name: bookinfo-gateway
> spec:
>   selector:
>     istio: ingressgateway
>   servers:
>   - port:
>       number: 80
>       name: http
>       protocol: HTTP
>     hosts:
>     - "bookinfo.app"
> EOF
gateway.networking.istio.io/bookinfo-gateway created
istiotraining@local istio-1.10.3 $
```

إنشاء Gateway جديد بـ hostname محدد

❖ تم إنشاء Gateway جديد فيه:

• البروتوكول HTTP :

• البورت: 80

• شرط مهم: ← فقط الطلبات اللي جاية على : bookinfo.app

➡ يعني أي حاجة غير ال hostname ده ← مش هتدخل

دور Virtual Service هنا

- ❖ حتى لو عندك Gateway شغال، فلوحده مش كفاية ..
- ❖ ولازم Virtual Service علشان يحدد :
 - الطلب يروح لأي Service
 - routing rules
 - path matching

أهم نقطة: تطابق الـ Host

- ❖ لازم يحصل تطابق بين 3 حاجات:

- Gateway host
- Virtual Service host
- request host (من المستخدم)

لو واحد منهم مختلف ❌ ← الترافيك مش هيمشي

تجربة الوصول باستخدام curl

- ❖ تم اختبار الوصول عن طريق:

- إرسال request مع HTTP header :

← Host: bookinfo.app ودا باستخدام curl -H

وده بيخلي الطلب يعدي من الـ Gateway بنجاح ✓

نجاح التجربة

- ❖ بعد الإعداد:

- المستخدم يقدر يوصل للتطبيق عبر : <http://bookinfo.app/productpage>

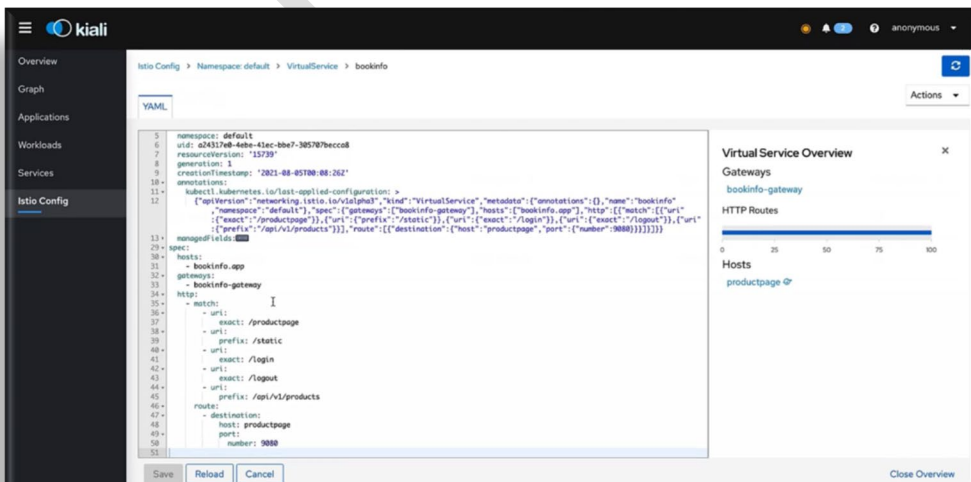
- ❖ وده بيأكد إن:

- Gateway شغال
- Virtual Service شغال
- routing صحيح

استخدام Kiali للمراقبة

- ❖ تم استخدام Kiali (Dashboard بتاع Istio) علشان:

- نعرض الـ Gateway
- نعرض الـ Virtual Service
- نتابع الترافيك بصرياً
- وده بيسهل فهم تدفق الطلبات داخل الـ mesh

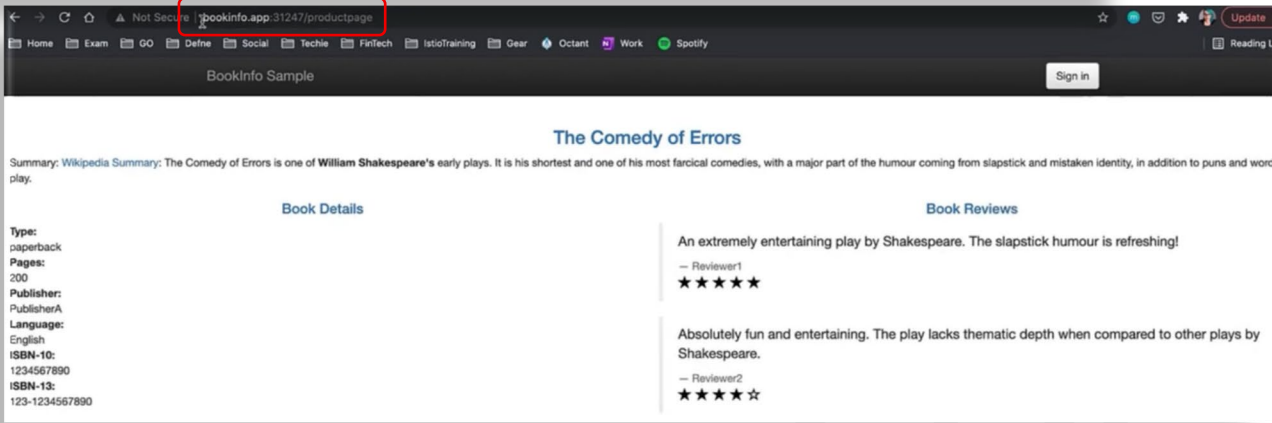


❖ علشان المتصفح يعرف bookinfo.app :

• تم تعديل ملف hosts وربط الدومين بـ IP الخاص بالكلاستر

🌐 ده يسمح إنك تكتب في المتصفح: <http://bookinfo.app/productpage> بدل IP طويل

```
istiotraining@local istio-1.10.3 $ echo -e "$(minikube ip)\tbookinfo.app" | sudo tee -a /etc/hosts
Password:
192.168.64.43 bookinfo.app
```



تجربة المتصفح

في النهاية:

- تم فتح المتصفح
- وتم الوصول للتطبيق بنجاح باستخدام hostname

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1alpha3","kind":"Gateway","metadata":{"annotations":{"name":"bookinfo-gateway"},"n
      ault"},"spec":{"selector":{"istio":"ingressgateway"},"servers":[{"hosts":["*"],"port":{"name":"http","number":80,"protocol":"
      creationTimestamp: "2021-10-09T22:19:31Z"
      generation: 10
      name: bookinfo-gateway
      namespace: default
      resourceVersion: "20541"
      uid: 4686ab5f-31f8-463c-940f-72577c65364a
spec:
  selector:
    istio: ingressgateway
  servers:
  - hosts:
    - bookinfo.app
    port:
      name: http
      number: 80
      protocol: HTTP
```

الرجوع لـ wildcard (Reset)

❖ بعد التجربة:

- تم الرجوع إلى إعدادات wildcard في الـ Gateway ()
- عشان التدريب يكمل بشكل مرن
- وأحياناً يتم استخدام IP بدل hostname

ملخص

- تم عمل Demo عملي على Istio Gateway + Virtual Service
- تم تحويل Gateway من wildcard إلى hostname محدد (bookinfo.app)
- لازم تطابق بين :

Gateway host ○

Virtual Service host ○

request host ○

- تم اختبار الوصول باستخدام curl والمتصفح
- Kiali استخدم لمراقبة الترافيك بصرياً
- تم استخدام hosts file لربط الدومين بالـ IP
- في النهاية تم الرجوع لإعدادات wildcard للتدريب 🌐

Demo – Virtual Services

مقدمة

- ❑ الديمو ده بيركز على واحدة من أقوى مميزات Istio :
 - ❖ التحكم الذكي في الترافيك (Traffic Control)
- ❑ مش بس نوزع الترافيك بنسب (زي 75% / 25%)، لكن كمان:
 - نوجه مستخدمين معينين لنسخ مختلفة من التطبيق
 - نعمل A/B Testing و Canary Deployment بسهولة

الشرح التفصيلي

الخطوة الأولى: تعريف النسخ (Versions)

- ❖ قبل ما نوزع الترافيك، لازم نكون معرفين :

- v1
- v2
- v3

- ❖ وده بيتم باستخدام: Destination Rule

- اللي بيعمل:

- Subsets لكل version اعتمادًا على labels في الـ Pods

```
istiotraining@local istio-1.10.3 $ kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
destinationrule.networking.istio.io/productpage created
destinationrule.networking.istio.io/reviews created
destinationrule.networking.istio.io/ratings created
destinationrule.networking.istio.io/details created
istiotraining@local istio-1.10.3 $
```

```
istiotraining@local istio-1.10.3 $ vi virtual-service1.yaml
```

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
        weight: 75
    - destination:
        host: reviews
        subset: v2
        weight: 25
```

Just checking everything is good

```
istiotraining@local istio-1.10.3 $ istioctl analyze
✓ No validation issues found when analyzing namespace: default.
istiotraining@local istio-1.10.3 $
```

إنشاء Virtual Service للتوزيع

- ❖ تم إنشاء Virtual Service باسم: reviews وفيه:

v1 ← Route 1 ◦

v2 ← Route 2 ◦

- مع توزيع:

75% → v1 ◦

25% → v2 ◦

🚩 مهم: لازم مجموع الـ weights = 100%

To generate Traffic

مراقبة الترافيك باستخدام Kiali

- ❖ في Kiali :

- تقدر تشوف :

Destination Rules ◦

Virtual Service ◦

توزيع الترافيك بشكل بصري (Graph) ◦

- ❖ ولو بصيت على التطبيق:

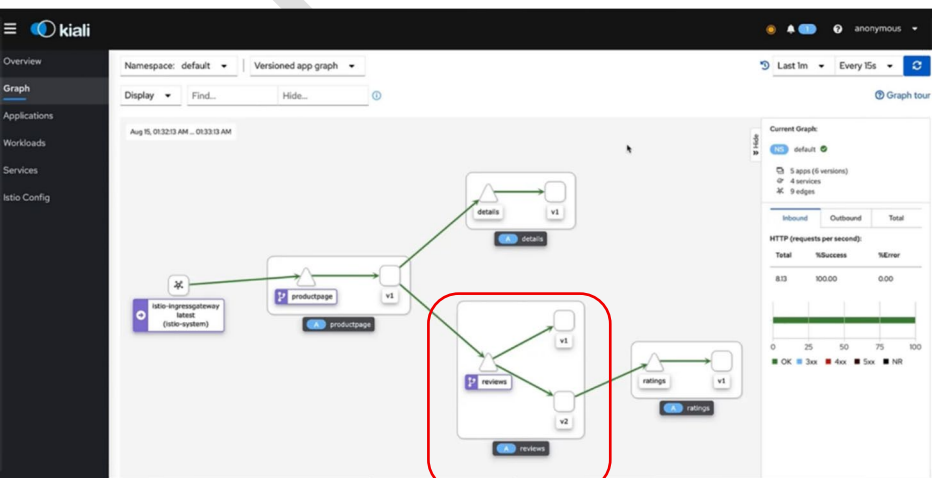
- هتلاقي :

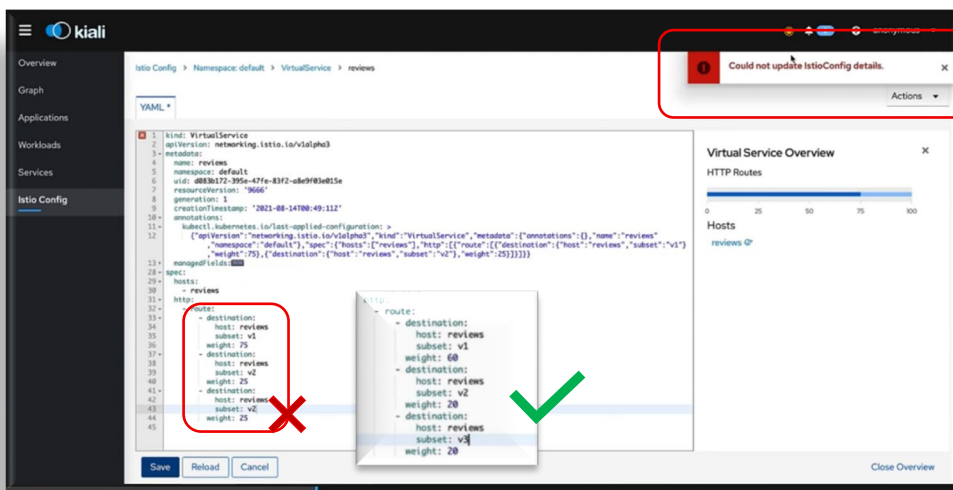
v1 (no stars) ◦

v2 (black stars) ◦

- v3 مش ظاهر لأنه مش داخل في التوزيع

```
istiotraining@local istio-1.10.3 $ while sleep 0.01;do curl -sS 'http://"$INGRESS_HOST":'$INGRESS_PORT'/productpage\' &> /dev/null ; done
```





تعديل التوزيع (Dynamic Change)

- ❖ تم تعديل التوزيع من خلال Kiali UI :
- ❌ حصل خطأ لما المجموع كان 125%
- ✅ تم تصحيحه لأن لازم = 100%

إضافة Version جديدة (v3)

- ❖ تم إدخال:
- v3 (red stars) ← ب نسبة صغيرة جدًا
- النتيجة:
- بدأ يظهر أحيانًا في المتصفح
- وظهر في Graph

تجربة توزيع دقيق جدًا

- ❖ تم تطبيق:

- v1 ← 99%
- v2 ← 1%
- Delete v3

(مسحه عشان مش محتاجه .. هو بس عاوز يثبتك الفكرة في المثال ده)

- النتيجة:

- v1 شبه ظاهر طول الوقت
- v2 نادر جدًا يظهر
- وده يوضح قوة Istio مقارنة بـ Kubernetes العادي

مثال آخر : الجزء الأقوى (Advanced Routing)

Routing حسب المستخدم (Header-Based Routing)

- ❖ بدل ما نوزع عشوائي : ← نوجه حسب المستخدم نفسه
- ❖ مثال:

مستخدم: "KodeKloud"

- هايشوف v2 (black stars)

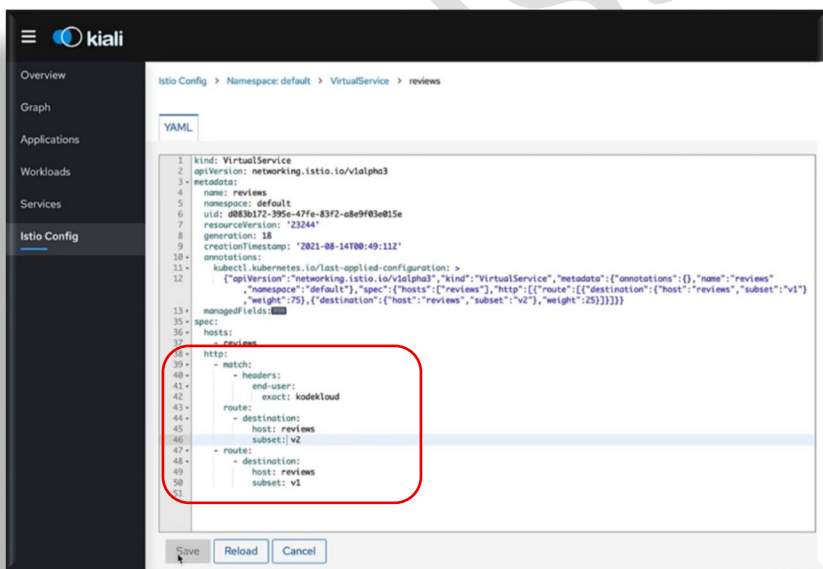
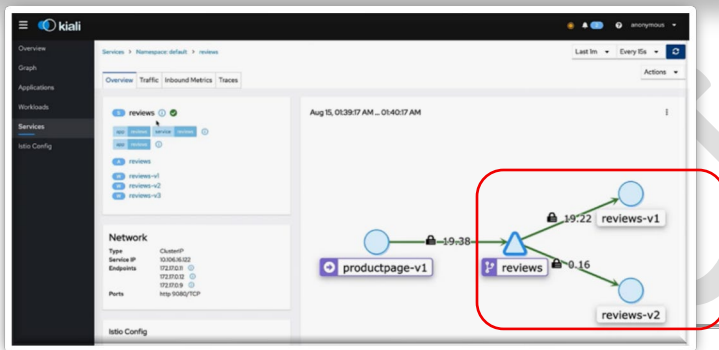
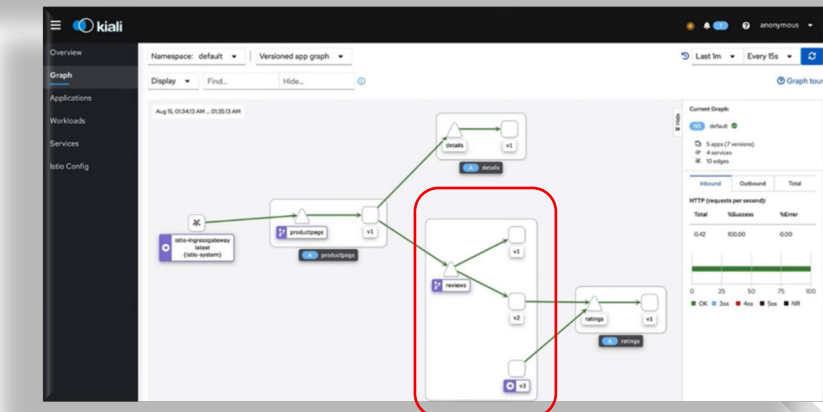
أي مستخدم ثاني:

- هايشوف v1 (no stars)

إزاي؟ ▽

- باستخدام ←

KodeKloud = end-user : request header



تجربة فعلية

- المستخدم يعمل Sign In
- التطبيق يضيف Header تلقائي
- Istio يقرأ ال header
- يوجهه لنسخة معينة
- النتيجة:

- نفس المستخدم دائماً يشوف نفس النسخة
- مش random زي weight-based

إضافة User Group جديدة (Test Users)

- تم إضافة Rule جديدة : test user:
- يروح لـ v3 (red stars)

النسخة	المستخدم
v1	عادي
v2	KodeKloud
v3	test user

النتيجة:

ملاحظة مهمة

- كل Rule في match :
- لما يتحقق الشرط ← بياخد 100% من الترافيك
- يعني مش بيستخدم weights هنا

ملخص

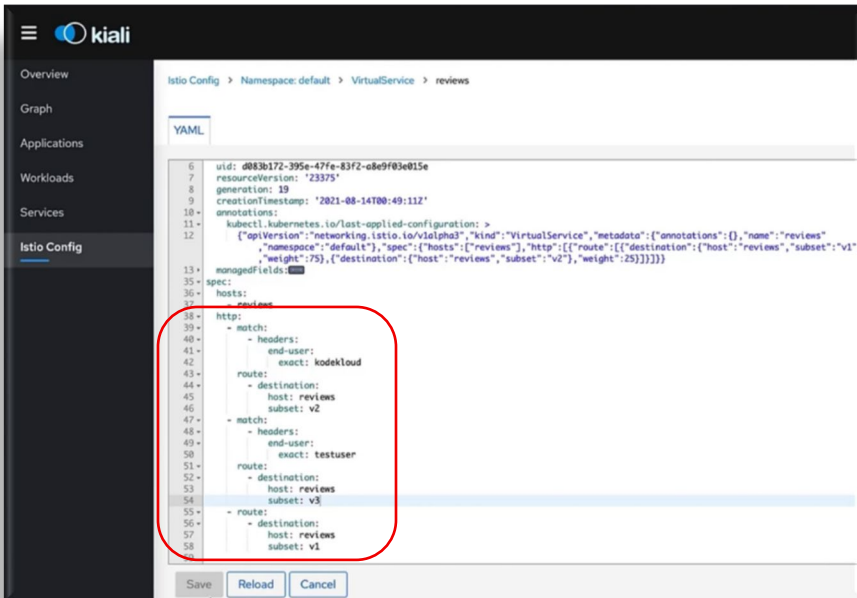
- استخدمنا Destination Rule لتعريف النسخ (v1, v2, v3)
- استخدمنا Virtual Service لتوزيع الترافيك
- قدرنا نعمل :
 - توزيع نسبي (75% / 25%)
 - توزيع دقيق (99% / 1%)
- استخدمنا Kiali لمراقبة الترافيك
- عملنا Routing حسب المستخدم (Header-based)
- قدرنا نخصص تجربة مختلفة لكل user group

الخلاصة الكبيرة

Istio بيديك 3 مستويات تحكم:

- Gateway ← دخول الترافيك
- Virtual Service ← التوجيه (Routing logic)
- Destination Rule ← تعريف النسخ + السياسات

ومع بعض بيدوك تحكم كامل في الترافيك بشكل مستحيل تعمله بـ Kubernetes لوحده



Demo – Destination Rules

مقدمة

- ❑ في الديمو ده بنشوف مستوى أعمق من التحكم في Istio
- ❑ يعني مش بس نوزع الترافيك بين (v1, v2, v3) versions، لكن كمان:

- ❖ نعمل Grouping مخصص للـ Pods باستخدام Labels
- ❖ ونبني Subsets جديدة بمرونة حسب السيناريو اللي عايزينه
- 🚀 وده بيدينا قوة رهيبية في إدارة الترافيك والتجارب (Testing)

الشرح التفصيلي

الحالة الحالية

- ❖ كان عندنا بالفعل:

• Destination Rule فيه :

- subset v1
- subset v2
- subset v3

- ❖ وكل subset مرتبط بـ :

Label : version = v1 / v2 / v3 + Virtual Service بيستخدمهم في التوزيع.

```
1 kind: DestinationRule
2 apiVersion: networking.istio.io/v1alpha3
3 metadata:
4   name: reviews
5   namespace: default
6   uid: 7a7ad15c-4168-4948-a331-14ca49c64fc4
7   resourceVersion: '26753'
8   generation: 7
9   creationTimestamp: '2021-08-15T22:17:54Z'
10  annotations:
11    kubectl.kubernetes.io/last-applied-configuration: >
12      [{"apiVersion": "networking.istio.io/v1alpha3", "kind": "DestinationRule", "metadata": {"annotations": {}, "name": "reviews", "namespace": "default"}, "spec": {"host": "reviews", "subsets": [{"labels": {"version": "v1"}, "name": "v1"}, {"labels": {"version": "v2"}, "name": "v2"}, {"labels": {"version": "v3"}, "name": "v3"}]}}]
13  managedFields:
14    - operation: Create
15      time: 2021-08-15T22:17:54Z
16      user: kubelet
17      fields:
18        f:spec:
19          subsets:
20            - name: v1
21              labels:
22                version: v1
23            - name: v2
24              labels:
25                version: v2
26            - name: v3
27              labels:
28                version: v3
```

```
13+ generation: 9
14+ creationTimestamp: '2021-08-15T22:18:03Z'
15+ annotations:
16+   kubectl.kubernetes.io/last-applied-configuration: >
17+     [{"apiVersion": "networking.istio.io/v1alpha3", "kind": "VirtualService", "metadata": {"annotations": {}, "name": "reviews", "namespace": "default"}, "spec": {"hosts": ["reviews"], "http": [{"route": [{"destination": {"host": "reviews", "subset": "v1"}, "weight": 100}], "route": [{"destination": {"host": "reviews", "subset": "v2"}, "weight": 100}], "route": [{"destination": {"host": "reviews", "subset": "v3"}, "weight": 100}]}]}]
18+ managedFields:
19+   - operation: Create
20+     time: 2021-08-15T22:18:03Z
21+     user: kubelet
22+     fields:
23+       f:spec:
24+         route:
25+           - destination:
26+               host: reviews
27+               subset: v1
28+             weight: 100
29+           - destination:
30+               host: reviews
31+               subset: v2
32+             weight: 100
33+           - destination:
34+               host: reviews
35+               subset: v3
36+             weight: 100
```

الفكرة الجديدة

- ❖ طيب لو عايزين نعمل grouping مختلف؟

- مثلاً:

- نجمع v2 و v3 مع بعض ونستبعد v1

- ❖ ليه؟

- نختبر features جديدة
- نعمل Beta testing
- نعرض نسخة معينة لمجموعة معينة

```
istiotraining@local istio-1.10.3 $ vi reviews.yaml
```

الحل: استخدام Labels جديدة

❖ تم إضافة Label جديد : test: beta

• لكن :

• اتضاف لـ v2 و v3 فقط ✓

• لم يتم إضافته لـ v1 ✗

❖ بعدها مسحنا الـ review.yaml وعملناه تاني عشان يسمع

```
istiotraining@local istio-1.10.3 $ kubectl delete -f reviews.yaml
service "reviews" deleted
serviceaccount "bookinfo-reviews" deleted
deployment.apps "reviews-v1" deleted
deployment.apps "reviews-v2" deleted
deployment.apps "reviews-v3" deleted
istiotraining@local istio-1.10.3 $ kubectl apply -f reviews.yaml
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
istiotraining@local istio-1.10.3 $
```

مش هاتضيفه هنا عشان
هو في السيناريو عاوز كذا
بس هاتضيفه في v2 & v3

Workload Name	Name	Namespace	Type	Labels
details-v1	details-v1	default	Deployment	app: details, version: v1
productpage-v1	productpage-v1	default	Deployment	app: productpage, version: v1
ratings-v1	ratings-v1	default	Deployment	app: ratings, version: v1
reviews-v1	reviews-v1	default	Deployment	app: reviews, version: v1
reviews-v2	reviews-v2	default	Deployment	app: reviews, test: beta, version: v2
reviews-v3	reviews-v3	default	Deployment	app: reviews, test: beta, version: v3

إنشاء Subset جديد

❖ في Destination Rule :

• احتفظنا بـ v1

• أضفنا subset جديد : test-beta اللي بيضم:

• v2 + v3 مع بعض

(وعشان متوهش .. فده طبيعي حصل نتيجة إننا في الـ review.yaml حطينا الـ Label في v2 & v3 فبالتالي أوتوماتيك لما أضيف الـ label في الـ Destination Rule فهايجمعلي الإثنين مع بعض)

تعديل الـ Virtual Service (توزيع الترافيك)

❖ مثلاً:

• v1 ← 90%

• test-beta ← 10%

• معنى كده:

• 90% من المستخدمين يشوفوا v1

• 10% يشوفوا إما v2 أو v3

النتيجة في التطبيق

في المتصفح:

• v1 ظاهر أغلب الوقت

• v2 و v3 بيظهروا أحياناً (rare)

```
#####
# Reviews service
#####
apiVersion: v1
kind: Service
metadata:
  name: reviews
  labels:
    app: reviews
    service: reviews
spec:
  ports:
    - port: 9080
      name: http
      selector:
        app: reviews
  type: ClusterIP
#####
apiVersion: v1
kind: ServiceAccount
metadata:
  name: bookinfo-reviews
  labels:
    account: reviews
#####
apiVersion: apps/v1
kind: Deployment
metadata:
  name: reviews-v1
  labels:
    app: reviews
    version: v1
spec:
  replicas: 1
  selector:
    matchLabels:
      app: reviews
```

```
13 managedFields:
14 -
15   apiVersion: v1
16   kind: Service
17   name: reviews
18   namespace: default
19   operation: Update
20   subresource: status
21   timestamp: 2020-07-20T14:00:00Z
22   user: istiotraining
23   uid: 11111111-1111-1111-1111-111111111111
24   version: v1
25 spec:
26   ports:
27     - port: 9080
28       name: http
29       selector:
30         app: reviews
31       targetPort: 9080
32   type: ClusterIP
33 status:
34   loadBalancerIP: 10.10.10.10
35   loadBalancerStatus:
36     ingresses:
37     - ip: 10.10.10.10
38       ports:
39         - port: 9080
40           targetPort: 9080
```

```
35 spec:
36   hosts:
37     - reviews
38   http:
39     - route:
40       - destination:
41         host: reviews
42         subset: v1
43         weight: 90
44       - destination:
45         host: reviews
46         subset: test
47         weight: 10
48
```

مراقبة الترافيك (Kiali)

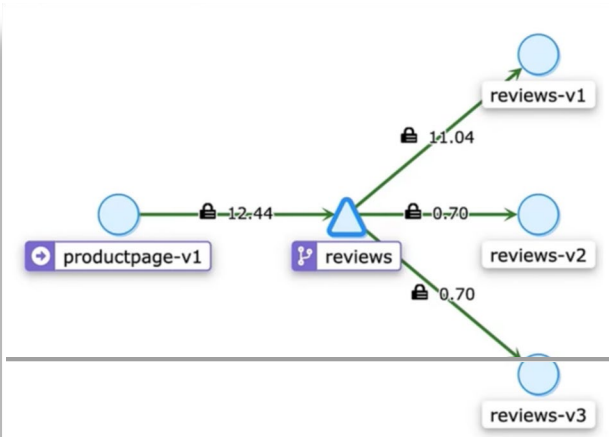
❖ في Kiali :

• نقدر نشوف :

○ التوزيع

○ graph

○ إن v1 واخذ تقريبًا 9 أضعاف الترافيك



تغيير التوزيع

❖ تم تعديل الـ weights :

• بحيث إن الترافيك بدأ يروح أكثر لـ test-beta

🚩 النتيجة:

• v2 و v3 بقوا ظاهرين أكثر

```
35 spec:
36   host: reviews
37   subsets:
38   - labels:
39     version: v1
40     name: v1
41   - labels:
42     test: beta
43     name: test
44     trafficPolicy:
45       loadBalancer:
46       simple: RANDOM
47
```

تخصيص Load Balancing

❖ تم إضافة Traffic Policy في الـ Destination Rule ← loadBalancer: random

لـ test-beta ← subset

🚩 النتيجة:

• الترافيك بين v2 و v3 بقى عشوائي

```
39 spec:
40   host: reviews
41   trafficPolicy:
42     loadBalancer:
43     simple: ROUND_ROBIN
44
```

تجربة Round Robin

❖ تم إضافة round-robin في الـ destination Rule

❖ ثم حذف الـ subsets في الـ virtual service

🚩 النتيجة:

• الترافيك يتوزع بالتساوي بين كل النسخ

• كل refresh يجيب version مختلف

```
35 spec:
36   hosts:
37   - reviews
38   http:
39   - route:
40     - destination:
41       host: reviews
42
```

الفكرة المهمة جدًا

❖ Istio بيديك 3 مستويات تحكم:

1. Label (Kubernetes)

• لتجميع الـ Pods

2. Destination Rule

• لتحويل الـ labels إلى Subsets

3. Virtual Service

• لتوزيع الترافيك بين Subsets

- أضفنا Label جديد لعمل grouping مخصص (test-beta)
- أنشأنا Subset جديد يجمع v2 + v3
- استخدمنا Virtual Service لتوزيع الترافيك :
 - 90% → v1
 - 10% → test-beta
- استخدمنا Kiali لمراقبة الترافيك
- جربنا :
 - Random Load Balancing
 - Round Robin
- فهمنّا إن Istio يسمح بمرونة عالية جدًا في التحكم في الترافيك

الخلاصة الكبيرة 🔥

- ❖ بدل ما تكون مقيد بـ : version-based routing فقط ❌
- ❖ بقيت تقدر تعمل :

- grouping مخصص
- A/B testing متقدم
- Canary deployment
- Traffic shaping دقيق جدًا

Fault Injection

الفكرة الأساسية

❑ بدل ما تستنى المشكلة تحصل في production .. إنت بتخلقها بنفسك وتراقب behavior

🌟 يعني إيه Fault Injection في Istio ؟

❖ ببساطة:

أنت بتعمل مشاكل متعمدة (Fake errors) في السيستم علشان تختبر:

- هل الـ app بيستحمل؟
- هل الـ retries شغالة؟
- هل الـ timeouts مظبوطة؟
- هل الـ circuit breaker بيشتغل؟

🔧 فين بنطبةها؟

❖ بتتعمل في Virtual Service لأن هو المسؤول عن:

- التحكم في الترافيك وبالتالي نقدر نعدل عليه وندخل errors

🔧 أنواع Fault Injection

1. Delay (Latency Injection)

❖ بتبطأ الريكويست

مثال:

🔧 المعنى:

- 10% من الريكويستات هتتأخر 5 ثواني

2. Abort (Error Injection)

❖ بترجع error مباشرة

مثال:

🔧 المعنى:

- 5% من الريكويستات هترجع HTTP 500

fault-injection.yaml

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: my-service
spec:
  hosts:
  - my-service
  http:
  - fault:
      delay:
        percentage:
          value: 0.1
        fixedDelay: 5s
      abort:
        percentage:
          value: 0.1
        httpStatus: 400
  route:
  - destination:
      host: my-service
      subset: v1
```

📺 السيناريو في الفيديو

❖ عملوا delay على خدمة ratings

❖ يعني:

- بعض الريكويستات بقت بطيئة
- وده بيختبر هل :

- productpage هيستنى؟
- ولا هيعمل timeout ؟
- ولا fallback ؟

❖ Test Timeouts

- بحيث لو service اتأخرت ← هل الـ client يستحمل؟

❖ Test Retries

- هل السيستم هيحاول تاني؟

❖ Test User Experience

- هل UI بيهنج ؟ ولا graceful degradation ؟

❖ Chaos Engineering

- زي tools :

○ Chaos Monkey (Netflix style)

⚠️ ملاحظات مهمة

❖ **Percentage (النسبة)** ← يعني مش كل الترافيك بيتأثر .. ولكن جزء بس (زي real-world failure)

❖ **Fault Injection** مش للـ Production بشكل دائم

- تستخدمها testing / staging
- أو بشكل controlled جداً في production

❖ **Fault Injection** الـ بتتطبق على Routes معينة

- يعني تقدر تقول:

○ user معين

○ path معين

○ version معين

🎯 مثال (Interview Style)

❖ عايز تختبر resilience بتاع microservice

- فتقول:

○ هأضيف 5s delay لـ 10% من requests

○ ثم أراقب :

▪ latency

▪ retries

▪ error rate

💡 الخلاصة

النوع	بيعمل إيه
Delay	بيطأ request
Abort	يوقع request بـ error

🧠 أهم فكرة

Istio مش بس routing .. ده كمان testing tool قوي جداً

Demo – Fault Injection

الهدف من الديمو

نعمل Delay متعمد على خدمة details

علشان نشوف:

- هل الـ productpage هيستحمل؟
- هل هيظهر error؟
- هل هيكون فيه timeout؟

الخطوات اللي حصلت

1 عمل Destination Rule

علشان نعرف subset :

ده بيربط details service بالـ pods

2 عمل Virtual Service + Fault Injection

التفسير :

- 70% من requests ← هتتأخر 7 ثواني
- 30% طبيعي

اللي حصل فعلياً

في Kiali

- Red arrows ← requests بطينة / فيها مشاكل
- Green arrows ← requests شغالة طبيعي
- ده بيوضح إن:
- في traffic متأثر
- وفي traffic لسه healthy

في الـ Browser

- أوقات الصفحة بتأخر جداً
- أوقات بتشتغل طبيعي

ليه؟

علشان فيه:

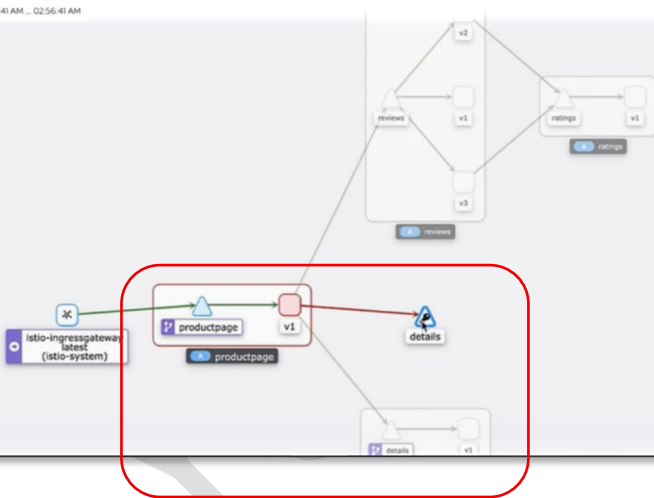
- 70% slow
- 30% fast

```
istiotraining@local istio-1.10.3 $ vi virtual-service-fault.yaml
```

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: details
spec:
  hosts:
  - details
  http:
  - fault:
      delay:
        percentage:
          value: 70.0
        fixedDelay: 7s
    route:
    - destination:
        host: details
        subset: v1
```

```
istiotraining@local istio-1.10.3 $ kubectl apply -f virtual-service-fault.yaml
virtualservice.networking.istio.io/details created
```

Aug 8, 02:55:41 AM ... 02:56:41 AM



- ❖ المشكلة مش إن service وقعت
- ❖ المشكلة إنها بطيئة (Latency issue)
- ❖ وده أخطر أحياناً من إنها يحصلها crash

ليه السيناريو ده مهم؟

❖ Test Timeout

- هل الـ productpage عنده timeout ؟
- ولا هيسبب 7 ثواني؟

❖ Test Resilience

- هل في fallback ؟
- ولا UI يبيوظ؟

❖ Test Retry Logic

- هل بيحاول تاني؟
- ولا request بتفشل وخلاص؟

⚠ ملاحظة مهمة

❖ delay ≠ error

لكن ممكن يسبب:

- timeout
- degraded performance
- bad UX

🔄 مقارنة سريعة

الحالة	التأثير
Delay	بطء
Abort	Error مباشر

🎯 الخلاصة

❖ Fault Injection هنا خلاك تشوف:

- ازاي السيستم بيتصرف مع latency
- هل resilient ولا fragile

Timeouts

المشكلة الأساسية

❑ في الـ microservices :

❖ كل service يعتمد على service ثانية

❖ لو واحدة بطأت ← كله يبطأ (Cascading Failure)

بمعنى

details slow ← productpage waits ← users wait ← الـ system كله يهتج

الحل Timeout :

❖ بدل ما تستنى للأبد ✗

حدد وقت أقصى للانتظار ✓

تعريف Timeout :

❖ Timeout = أقصى وقت تستنى فيه response قبل ما تفشل

✗ حيث أن Fail Fast ← أفضل من Wait Forever

❖ بيتخط في Virtual Service

مثال عملي

المعنى:

• لو الرد أخذ أكثر من 3 ثواني ← request تفشل فوراً

ربطه مع Fault Injection

في الفيديو عملوا:

1 Delay على الـ details :

• delay: 5s (50% traffic)

2 Timeout على الـ productpage

• timeout: 3s

النتيجة

لأن:

• delay = 5s

• timeout = 3s

Insight مهم جداً

❖ Timeout بيتخط على الـ caller مش الـ callee

يعني:

• productpage هو اللي يقرر "أنا هستنى قد إيه" وليس الـ details

```
book-info.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: bookinfo
spec:
  hosts:
  - "bookinfo.app"
  gateways:
  - bookinfo-gateway
  http:
  - match:
    - uri:
        exact: /productpage
      - uri:
        prefix: /static
    <code hidden>
    route:
    - destination:
        host: productpage
        port:
          number: 9080
      timeout: 3s
```

```
details-service.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: details
spec:
  hosts:
  - details
  http:
  - route:
    - destination:
        host: details
        subset: v1
      fault:
        delay:
          fixedDelay: 5s
          percent: 50
```

الحالة	اللي يحصل
request سريع	ينجح
request داخل الـ 50% البطيء	✗ يفشل بعد 3 ثواني

❖ يمنع Cascading Failure

- Service واحدة تبوظ ← كله يقع ❌

❖ يحسن الـ Availability

- بدل ما users تستنى ← ياخدوا error بسرعة

❖ يسمح بـ Fallback

- مثلاً:

show cached data ○

show partial UI ○

⚠️ بدون Timeout

details slow → productpage waits → queue → crash 🌟

⚠️ مع Timeout

details slow → timeout → fail fast → system stable ✅

🧠 علاقة Timeout بباقي concepts

دوره	Concept
يوقف الانتظار	Timeout
يحاول تاني	Retry
يمنع requests مؤقتاً	Circuit Breaker
يختبر السيناريو	Fault Injection

🎯 مثال Interview

❖ ازاي تمنع cascading failure ؟

- تقول:

• أستخدم Timeout (fail fast)

○ Retry محدود

○ Circuit Breaker

💡 الخلاصة

❖ Timeout يحمي السيستم من البطء

❖ Fault Injection بيختبره

🚦 الاتنين مع بعض = production-ready system

Demo – Timeouts

الفكرة الأساسية

عازين نثبت حاجة:

❖ لو Service بطينة ← الـ system ماينفعش يستناها للأبد

الخطوات اللي حصلت

```
istiotraining@local istio-1.10.3 $ vi virtual-service-details.yaml
```

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: details
spec:
  hosts:
  - details
  http:
  - fault:
      delay:
        percent: 100
        fixedDelay: 5s
    route:
    - destination:
        host: details
        subset: v1
```

```
29 spec:
30   hosts:
31     - "*"
32   gateways:
33     - bookinfo-gateway
34   http:
35     - match:
36       - uri:
37         exact: /productpage
38       - uri:
39         prefix: /static
40       - uri:
41         exact: /login
42       - uri:
43         exact: /logout
44       - uri:
45         prefix: /api/v1/products
46     route:
47       - destination:
48         host: productpage
49         port:
50           number: 9080
51       timeout: 3s
52
```

1 عمل (Fault Injection) Delay

كل requests هتأخر 5 ثواني

2 عمل Timeout

❖ timeout: 3s

▪ أقصى انتظار = 3 ثواني

⚡ النتيجة : request fails with timeout ❌
لأن :

• details بيرد بعد 5 ثواني

• productpage مستتي 3 ثواني بس

▽ وبالتالي هياظهر Error لأن delay (5s) > timeout (3s)

لما غيروا النسبة

❖ من 100% ← 70% ← 50%

بقي فيه:

• requests slow ❌

• requests fast ✅

في البراوزر

• أوقات الصفحة تفتح طبيعي

• أوقات تظهر Timeout

⚡ حسب إذا كانت request دخلت في الـ % delayed ولا لا

أهم Insight

❖ Timeout بياظهر نتيجة لـ delay

❖ يعني:

• delay = سبب المشكلة

• timeout = الحماية

❖ Prevent Cascading Failure

• slow service → waiting → queue → crash ❌

❖ Fail Fast

• slow service → timeout → error → system stable

❖ Better UX

• بدل ⌚ loading forever

○ الـ user ياخذ error بسرعة

⚠️ ملاحظة قوية

❖ حتى لو service شغالة (مش down)

فالـ latency لو عالي ← الـ system يعتبر فاشل

🔗 العلاقة بين المفاهيم

دوره	Concept
تعمل delay	Fault Injection
تحدد max انتظار	Timeout
النتيجة system behavior واضح	

💡 الخلاصة

- Delay بيختبر السيستم
- Timeout بيحمي السيستم
- Resilience حقيقي = الاتنين مع بعض

Retries

🔍 يعني إيه Retry ؟

- ❌ لما Service تفشل في request ← فبدل ما تفشل على طول
- ✅ تحاول ثاني تلقائيًا

💡 الفكرة الأساسية

❖ مش كل failures دائمة... ممكن تكون transient (مؤقتة) زي:

- network glitch
- service مشغولة لحظة
- packet lost

🔧 لاحظ ان بيتم تحديد الـ Retry في Virtual Service

🔧 Istio في Default Behavior

❖ تلقائيًا:

- 2 retries
- delay ≈ 25ms

❖ يعني:

request → fail → retry → retry → fail

📺 مثال عملي

🔍 التفسير

- attempts: 3 ← يحاول 3 مرات
- perTryTimeout: 2s ← كل محاولة تستنى 2 ثانية

🔍 اللي حصل :

▽ بدون Retry (fail ← request)

▽ مع Retry (success ← retry ← fail ← request)

🔥 العلاقة مع Timeout

❖ timeout = perTryTimeout لكل محاولة

- مش نفس الـ global timeout

مثال:

🔍 أقصى وقت ممكن:

- 3 attempts × 2s = 6s

❗ لكن global timeout = 5s ← هيقف قبلها

Virtual-service-timeout.yaml

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: my-service
spec:
  hosts:
  - my-service
  http:
  - route:
    - destination:
        host: my-service
        subset: v1
    retries:
      attempts: 3
      perTryTimeout: 2s
```

By Default

ISTIO DEFAULTS
25ms+ intervals after 1st fail
2 retries before returning an error

```
timeout: 5s
retries:
  attempts: 3
  perTryTimeout: 2s
```

ليه Retries مهمة؟

- ❖ تحسين Success Rate
 - requests اللي كانت هتفشل ← تنجح
- ❖ تخفي مشاكل الشبكة المؤقتة
 - user مش هيحس بيها
- ❖ تقليل Errors
 - خصوصاً في distributed systems

⚠️ مخاطر مهمة

- ❖ **Retry Storm** !
 - لو كل services بت retry ← overload
 - ❖ زيادة latency
 - كل retry = وقت زيادة

وبالتالي فهي مش لكل الحالات

- لو error logic (مثلاً 400) ← retry هنا غلط ❌

🔥 Best Practice

- ❖ استخدم Retry مع:
 - Timeout ⌚
 - Circuit Breaker 🚧

🔗 مثال Interview

- ❖ امتى تستخدم Retry ؟
 - مع transient failures
 - مع network issues
 - مع permanent errors

🧠 الخلاصة

دوره	Concept
يحدد max انتظار	Timeout
يحاول ثاني	Retry
يختبر	Fault Injection

Circuit Breaking

الفكرة:

❑ بدل ما Service تستنى Service بايطة ← نوقف إرسال requests ليها بسرعة
🔧 زي فكرة الفيوز الكهربائي

المشكلة اللي بيحلها:

❖ لو Service (زي details) بطيئة:

- requests تتكدس
- باقي السيستم يتأثر (cascading failure)

الحل:

❖ نحدد limits :

- عدد connections
- عدد requests
- queue

بيتنظبط فين؟

❖ في DestinationRule

❖ مثال :

يعني ايه؟

- يعني أقصى حاجة عندك هي 3 connections .. ولو زاد ← Reject Requests فوراً

الصورة الكبيرة (Flow حقيقي)

الهدف	Feature
يحاول تاني لو فشل	Retry
يوقف الانتظار بعد وقت معين	Timeout
يمنع الضغط على Service بايطة	Circuit Breaking

1. request يروح لـ Service

- لو بطيئة :

○ Timeout يحصل

2. Istio :

○ يعمل Retry

3. لو المشكلة مستمرة :

○ Circuit Breaker يقلل الخط

نصيحة

❖ في production :

- استخدم Retry + Timeout + Circuit Breaking مع بعض

- بس بحذر : ⚠

○ retries كثير = load زيادة

○ drop traffic = circuit aggressive

Demo – Circuit Breaking

🧠 الفكرة الأساسية (إيه اللي حصل؟)

❑ إحنا عملنا Circuit Breaker شديد جدًا (Strict) على خدمة productpage

❖ يعني قلنا:

"الخدمة دي مش هتستقبل requests كتير... ولو الضغط زاد ← ارفض فوراً"

⚙️ الإعدادات اللي اتعملت

❖ في DestinationRule :

maxConnections: 1 ▾

- مسموح Connection واحد بس
- أي connection جديد ← مرفوض

http1MaxPendingRequests: 1 ▾

- لو الخدمة مشغولة :
 - request واحد بس يستنى
 - الباقي ← Reject

maxRequestsPerConnection: 1 ▾

- كل connection يشيل request واحد فقط
- بعده ينقل

🧪 الاختبار باستخدام h2load

- -n → shows number of requests
- -c → shows number of concurrent clients

❖ لو ضيفت Client واحد فقط (c1)

📊 النتيجة :

الـ requests كلها إتهندلت

❖ ولو جربت تضيف Client كمان (c2)

📊 النتيجة

هتلاحظ إن فيه 14 request is denied ودا بسبب
إن الـ Service كتنت مركزة مع أول client

```
27 - spec:
28   host: productpage
29   trafficPolicy:
30     connectionPool:
31       tcp:
32         maxConnections: 1
33       http:
34         http1MaxPendingRequests: 1
35         maxRequestsPerConnection: 1
```

```
istiotraining@local istio-1.10.3 $ h2load -n1000 -c1 'http://"$INGRESS_HOST":"$INGRESS_PORT"/productpage'
starting benchmark...
spawning thread #0: 1 total client(s). 1000 total requests
Application protocol: h2c
progress: 10% done
progress: 20% done
progress: 30% done
progress: 40% done
progress: 50% done
progress: 60% done
progress: 70% done
progress: 80% done
progress: 90% done
progress: 100% done

finished in 50.12s, 19.95 req/s, 95.26KB/s
requests: 1000 total, 1000 started, 1000 done, 1000 succeeded, 0 failed, 0 errored, 0 timeout
status codes: 1000 2xx, 0 3xx, 0 4xx, 0 5xx
traffic: 4.66MB (4888438) total, 12.42KB (12719) headers (space savings 91.23%), 4.62MB (4848664) data
min      max      mean     sd      +/- sd
time for request: 16.31ms 125.48ms 50.10ms 20.78ms 57.90%
time for connect: 515us    515us    515us    0us    100.00%
time to 1st byte: 32.32ms 32.32ms 32.32ms 0us    100.00%
req/s      : 19.95    19.95    19.95    0.00    100.00%
istiotraining@local istio-1.10.3 $
```

```
istiotraining@local istio-1.10.3 $ h2load -n1000 -c2 'http://"$INGRESS_HOST":"$INGRESS_PORT"/productpage'
starting benchmark...
spawning thread #0: 2 total client(s). 1000 total requests
Application protocol: h2c
progress: 10% done
progress: 20% done
progress: 30% done
progress: 40% done
progress: 50% done
progress: 60% done
progress: 70% done
progress: 80% done
progress: 90% done
progress: 100% done

finished in 109.98s, 9.10 req/s, 42.87KB/s
requests: 1000 total, 1000 started, 1000 done, 986 succeeded, 14 failed, 0 errored, 0 timeout
status codes: 986 2xx, 0 3xx, 0 4xx, 14 5xx
traffic: 4.60MB (4824406) total, 15.79KB (16166) headers (space savings 88.86%), 4.56MB (4781266) data
min      max      mean     sd      +/- sd
time for request: 453us    57.36s  218.63ms 2.55s   99.80%
time for connect: 489us    636us   562us    104us   100.00%
time to 1st byte: 8.97ms  78.99ms 43.98ms 49.51ms 100.00%
req/s      : 4.55    4.60    4.57    0.03    100.00%
```


❖ بس جربت تصيف Client كمان (c3)

النتيجة

هتلاقي إن عدد الـ denied request بقا أكثر بكثير

ليه ده حصل؟

لأنك حاطط limits صغيرة جداً:

- connection واحد
- request واحد
- queue واحد

وبالتالي أي ضغط زيادة = Drop مباشرة

اللي ظهر في Kiali

- arrows حمراء ← failures
- success rate نزل
- service مش قادرة تستحمل load

أهم Insight

❖ Circuit Breaking ≠ تحسين الأداء

- مش هدفه يخلي الخدمة أسرع
- ولكن هدفه إنه يحمي السيستم من الانهيار الكامل

الفرق بين الضغط الطبيعي والـ Circuit Breaker

الحالة	السلوك
بدون CB	يستنى → يهنج
مع CB	يرفض بسرعة → يفضل مستقر

في production :

- متخليش القيم aggressive زي كذا ✗
- استخدم values واقعية :
- based on load testing
- based on SLA

ربط كل اللي خدته مع بعض

❖ الـ Flow المثالي:

1. Timeout ⌚ ← متستناش كثير
2. Retry 🔄 ← جرب تاني
3. Circuit Breaker 🛑 ← لو الخدمة بايظة افقل عليها

```
istiotraining@local istio-1.10.3 $ h2load -n1000 -c3 'http://"$INGRESS_HOST":"$INGRESS_PORT"/productpage'
starting benchmark...
spawning thread #0: 3 total client(s). 1000 total requests
Application protocol: h2c
progress: 10% done
progress: 20% done
progress: 30% done
progress: 40% done
progress: 50% done
progress: 60% done
progress: 70% done
progress: 80% done
progress: 90% done
progress: 100% done

finished in 7.37s, 135.64 req/s, 107.59KB/s
requests: 1000 total, 1000 started, 1000 done, 147 succeeded, 853 failed, 0 errored, 0 timeout
status codes: 147 2xx, 0 3xx, 0 4xx, 853 5xx
traffic: 793.21KB (812252) total, 10.71KB (10966) headers (space savings 91.13%), 763.47KB (781798) data
time for request: 287us 155.64ms 11.01ms 27.27ms 88.80%
time for connect: 446us 535us 494us 44us 66.67%
time to 1st byte: 1.68ms 34.19ms 12.54ms 18.75ms 66.67%
req/s : 45.17 396.92 186.97 185.51 66.67%
istiotraining@local istio-1.10.3 $
```

النتيجة	concurrent clients
كله شغال ✅	1
بدأ يحصل رفض ⚠️	2
رفض أكثر ❌	3
أغلب requests مرفوضة 🌟	4

A/B Testing

🎯 يعني إيه A/B Testing ؟

- ❑ بدل ما تطلع Feature جديدة لكل المستخدمين مرة واحدة
- ❖ بتجربها على نسبة صغيرة منهم الأول

🎯 الهدف

- تعرف المستخدمين بيحبوا أنهي Version
- تقلل risk (لو فيه bug)
- تعتمد على data مش تخمين

🔧 إزاي Istio بيعمل ده؟

عن طريق:

- VirtualService ← للتحكم في الترافيك
- DestinationRule ← لتعريف الـ versions (subsets)

🔧 الفكرة

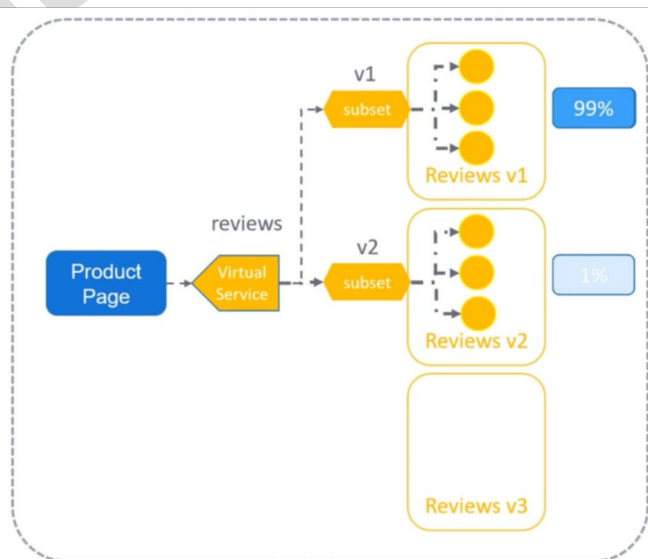
- ❖ عندك Service اسمها reviews :
- ❖ وعندك:
- v1 ← النسخة القديمة
- v2 ← النسخة الجديدة

🔧 بنعمل A/B Testing كدا:

1 نعرف الـ subsets (في DestinationRule) :

2 نوزع الترافيك (في VirtualService)

```
review-service.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
        weight: 99
    - destination:
        host: reviews
        subset: v2
        weight: 1
```



📊 النتيجة

نسبة الترافيك	Version
99%	v1
1%	v2

❖ عدد الـ Pods ملهوش علاقة بالترافيك

• يعني حتى لو:

○ v2 فيها 10 Pods

○ v1 فيها Pod واحد

• الترافيك هيبقى:

○ v1 ← 99%

○ v2 ← 1%

⚠️ ولا حظ إن التحكم بالكامل من Istio مش Kubernetes

🧠 ده بيفيدك في إيه؟

1 Canary Release

• نطلع Feature تدريجي

• 1% → 10% → 50% → 100%

2 Feature Testing

• تشوف الـ users بيحبوا الـ UI الجديد ولا القديم

3 Rollback سريع

• لو v2 فيها مشكلة : ← weight:0 ← لحقتنا المشكلة

دوره	Feature
توزيع الترافيك	VirtualService
تعريف versions	DestinationRule
التعامل مع failures	Retry / Timeout
حماية السيستم	Circuit Breaking
تجربة features	A/B Testing

🔗 ربطه باللي فات

💡 DevOps Insight

في الشركات:

• A/B Testing بيبقى مربوط بـ :

○ Metrics (Prometheus)

○ Dashboards (Kiali / Grafana)

○ Decision based on data

🚀 خلاصة سريعة

❖ Istio بيديك control كامل على traffic وبالتالي تقدر تعمل A/B Testing بدون ما تغيّر الكود 🧠

Security in Istio

1 مشكلة الـ Security في Microservices

□ لما Service تكلم Service ثانية

❖ ممكن حد يعترض الترافيك

❖ أويعدل فيه قبل ما يوصل

➦ ده اسمه (MITM) Man-in-the-Middle Attack

2 الحل الأول Encryption :

❖ لازم الترافيك يبقى مشفر عشان محدش يقدر يقرأه أو يغيره

➦ Istio بيعمل كدا باستخدام:

• mTLS (Mutual TLS)

○ يعني (تشفير + تأكيد هوية) بين الطرفين

3 الحل التاني Access Control :

❖ مش أي Service تكلم أي Service

• يعني بنحدد مين مسموحه يكلم مين

❖ مثال:

• productpage ← مسموح تكلم reviews

• details ← مش مسموح تكلم reviews

➦ ده بيتعمل باستخدام:

▪ Authorization Policies

4 الحل الثالث Auditing (تتبع)

❖ نعرف:

✓ مين عمل request

✓ امتى

✓ عمل ايه

➦ Istio بيوفر:

▪ Audit Logs

الخلاصة 🔥

❖ Istio بيحل 3 مشاكل أساسية في الـ Security :

✓ تشفير الترافيك (mTLS)

✓ التحكم في الوصول (Access Control)

✓ تتبع العمليات (Audit Logs)

Istio Security Architecture

(Big Picture) الصورة الكبيرة

Istio يحقق Security تلقائي بين الـ services بدون ما تغيّر في الكود

❖ إزاي؟

عن طريق:

- Certificates
- Encryption (mTLS)
- Policies

المكونات الأساسية

Istiod (Control Plane) 1

❖ ده عقل Istio

جواه:

- Certificate Authority (CA) مسؤولة عن:

- إصدار certificates
- توقيعها (Signing)
- التحقق منها

إزاي الـ Security بيحصل؟

1 Service (Pod) يشتغل

- Sidecar (Envoy) يتضاف له

2 Envoy يعمل Request

- يطلب:

- Certificate
- Private Key

- من:

Istiod (CA) ← Istio Agent

3 Istiod يرد

- يدي له:

- شهادة (Certificate)
- Key

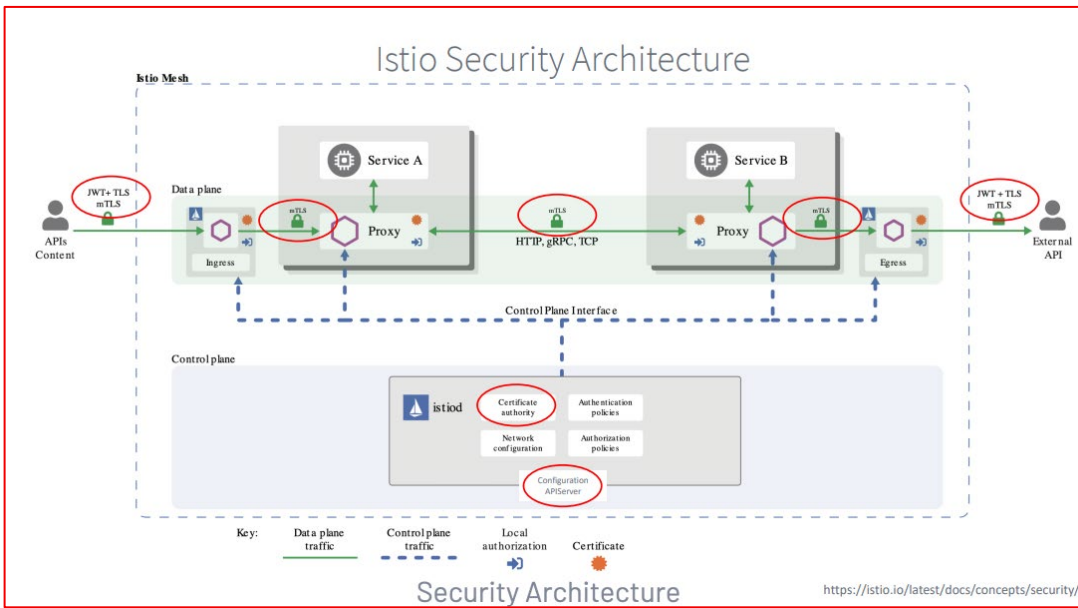
4 يبدأ الاتصال بين Services

- باستخدام:

🔒 mTLS (Mutual TLS)

- يعني:

○ كل Service تثبت هويتها للتانية ← وبالتالي الاتصال كله يبقى encrypted



❖ يعمل:

- Encryption / Decryption
- Authentication
- Authorization
- Policy Enforcement

Configuration API Server

❖ ده بيبيع لل Envoy :

- Policies
- Rules
- Security configs

❖ زي:

- مين يكلم مين
- مين مسموح ومين لا

🔒 مفهوم مهم جدًا Defense in Depth :

❖ الـ Istio ماشي بمبدأ "Security at every point"

❖ يعني:

- مش بس عند الـ Gateway
- لكن : بين كل Service و Service

🎯 مثال عملي

❖ بدل : (User → API → DB)

❖ يبقى : (User → [Envoy] → API → [Envoy] → DB)

- وبالتالي كل hop فيه:

- Auth
- Encryption
- Policy check

DevOps Insight

❖ Istio بيعمل:

• Zero Trust Architecture

يعني مافيش trust لأي service إلا لما تثبت نفسها

ليه ده شيء جامد جدا ؟

❖ بدون Istio :

- لازم تكتب security logic في كل service

❖ مع Istio :

كله centralized + automated (يعني Istio بيحول السيستم لـ Secure-by-default باستخدام mTLS و Envoy بدون ما تغير الكود)

وظيفته	Component
Control plane + CA	Istiod
تنفيذ السياسات	Envoy
تأمين الاتصال	mTLS
توزيع السياسات	Config API

Authentication

يعني إيه Istio Authentication ؟

ببساطة: ☐

نتأكد من هوية الـ Service اللي بتكلم Service ثانية

نوعين Authentication في Istio

1 Service-to-Service (داخل الـ Mesh) :

❖ باستخدام mTLS (Mutual TLS)

2 End-User to Service :

❖ باستخدام JWT (JSON Web Token)

PeerAuthentication ← بيتنظم في:

1 أول نوع : mTLS (الأهم)

❖ الفكرة: إن كل Service ليها:

- Certificate
- Identity

❖ فلما Service تكلم Service :

❖ مثال : (product → reviews)

❖ اللي بيحصل:

1. Envoy في product بيعت request
2. Envoy في reviews يقول: "إثبت إنك الـ product"

3. وبعدها الاتنين يعملوا: mutual verification

النتيجة:

- Encryption ☒
- Authentication ☒

مثال:

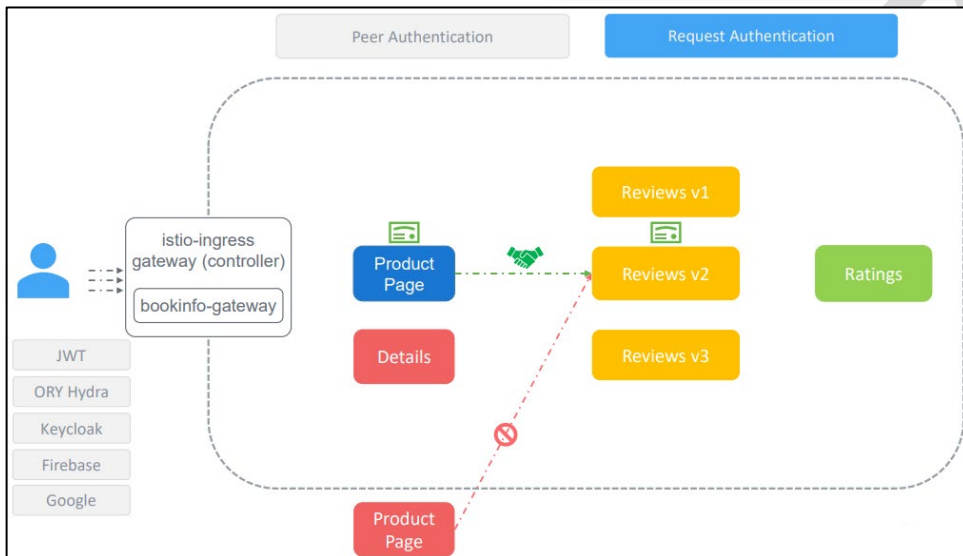
معناه :

- بيتطبق على:
 - هاتطبق علي الـ ns اللي اسمها book-info فقط
 - وتحديدًا علي الـ pods اللي جواها
 - واللي كمان label بتاعها app=reviews

• STRICT

يعني لازم mTLS

(يعني أي request بدون mTLS) ← مرفوض



example-peer-authentication.yaml

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: "example-peer-policy"
  namespace: "book-info"
spec:
  selector:
    matchLabels:
      app: reviews
  mTLS:
    mode: STRICT
```

المعنى	Mode
لازم mTLS	STRICT
يقبل الاثنين (mTLS + plain)	PERMISSIVE
مفیش mTLS	DISABLE

⚠ Modes مهمة:

⚙ بيّنطبط في : ← RequestAuthentication

2️ تاني نوع End-User Authentication :

❖ الفكرة:

❖ نتأكد إن المستخدم نفسه valid

❖ باستخدام:

- JWT
- أو Providers زي :
 - Keycloak
 - Firebase
 - Google

🔑 مثال:

🔍 معناه:

• Istio يتحقق من :

- Token
- Signature
- Issuer

🧠 Scope مهم جدًا

1 Workload-level

❖ يعني هياتطبق علي الـ ns اللي اسمها book-info فقط وتحديداً علي الـ Pods اللي بداخلها وبتمثل [label "app:review"] فقط (زي ماوضحنا من شوية)

2 Namespace-level

❖ لو شيلنا الـ selector ← فكدا الـ Policy هاتطبق علي كل الـ Pods اللي بداخل الـ ns

3 Mesh-wide

❖ ولو غيرت الـ ns إلي الـ Root Namespace (واللي تصادف إن اسمها هنا هو istio-system) هياتطبق علي كل الـ Mesh

```
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: jwt-auth
spec:
  jwtRules:
    - issuer: "https://example.com"
      jwksUri: "https://example.com/.well-known/jwks.json"
```

```
example-peer-authentication.yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: "example-peer-policy"
  namespace: "book-info"
spec:
  selector:
    matchLabels:
      app: reviews
  mTLS:
    mode: STRICT
```

Workload-specific policy

```
example-peer-authentication.yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: "example-peer-policy"
  namespace: "book-info"
spec:
  mTLS:
    mode: STRICT
```

Namespace-wide policy

```
example-peer-authentication.yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: "example-peer-policy"
  namespace: "istio-system"
spec:
  mTLS:
    mode: STRICT
```

Mesh-wide policy

دوره	Feature
تأمين + تعريف الهوية	mTLS
تأكيد المستخدم	JWT
تنفيذ التحقق	Envoy
إصدار certificates	Istiod

DevOps Insight

❖ في الـ production يفضل :

• تبدأ بـ :

○ PERMISSIVE (migration)

• بعد كدا :

○ STRICT (security enforced)

🎯 خلاصة

❖ Authentication في Istio يحصل على مستويين:

• بين الخدمات (mTLS)

• من المستخدم (JWT)

Demo Authentication

السيناريو الذي حصل

عندك: ☐

(فيه Bookinfo) Namespace: default

```
istiotraining@local istio-1.10.3 $ ./samples/bookinfo/platform/kube/cleanup.sh
namespace ? [default]
using NAMESPACE=default
Application cleanup may take up to one minute
service "details" deleted
serviceaccount "bookinfo-details" deleted
deployment.apps "details-v1" deleted
service "ratings" deleted
serviceaccount "bookinfo-ratings" deleted
deployment.apps "ratings-v1" deleted
service "reviews" deleted
serviceaccount "bookinfo-reviews" deleted
deployment.apps "reviews-v1" deleted
deployment.apps "reviews-v2" deleted
deployment.apps "reviews-v3" deleted
service "productpage" deleted
serviceaccount "bookinfo-productpage" deleted
deployment.apps "productpage-v1" deleted
Application cleanup successful
istiotraining@local istio-1.10.3 $
```

هنا بنعمل Clean-up للـ Default
ns علشان نبدأ الشرح ع نضافة

```
istiotraining@local istio-1.10.3 $ kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
service/details created
serviceaccount/bookinfo-details created
deployment.apps/details-v1 created
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created
istiotraining@local istio-1.10.3 $
```

بعدين هاترجع نجيب تاني الـ Bookinfo
App من الـ Samples folder

• Namespace جديد ← bar (فيه أبليكيشن اسمه HTTP Bin)

```
istiotraining@local istio-1.10.3 $ kubectl create ns bar
namespace/bar created
istiotraining@local istio-1.10.3 $
```

```
istiotraining@local istio-1.10.3 $ kubectl apply -f <(istioctl kube-inject -f samples/httpbin/httpbin.yaml) -n bar
serviceaccount/httpbin created
service/httpbin created
deployment.apps/httpbin created
```

Note that HTTP Bin will help us cURL the other services

الكوماتد دا بيعمل Deploy للـ HTTPBin بس بعد ما يضيفه (istio sidecar Envoy)
(ودا اللي بيخلي الترافيك يعدي من خلاله)

الـ istioctl kube-inject بيعمل ايه؟

- بيأخذ yml عادي ويعدله ويضيف فيه Envoy sidecar وبالتالي الـ Pod يبقى جاهز يدخل في الـ Service Mesh. (يعني كاتي بعمل حقن "inject" للـ Envoy جوا الـ Pod) ### خلي بالك ان الطريقة دي Manual ###
- الطريقة الأوتوماتيك بقا للـ Injection ::

Kubectl label namespace bar istio-injection=enabled

وبالتالي أي pod هاتعمله Deploy ← هيتحقق فيه Envoy اوتوماتيك

```
containers:
- name: httpbin
  image: httpbin
```

قبل الـ Injection

```
containers:
- name: httpbin
  image: httpbin
- name: istio-proxy
  image: envoy
```

بعد الـ Injection

● الحالة الأولى (بدون mTLS)

❖ bar → default (productpage)

(يعني من الـ bar namespace عملنا curl للـ default namespace وتحديدًا الـ product page)

✓ عمل curl

✓ رجع 200 OK

➡ يعني:

أي Service تقدر تكلم أي Service بدون تحقق ❌

```
istiotraining@local istio-1.10.3 $ kubectl exec -it "$(kubectl get pod -l app=httpbin -n bar -o jsonpath={.items..metadata.name})" -c istio-proxy -n bar -- curl "http://productpage.default:9080" -s -o /dev/null -w "%{http_code}\n"
200
istiotraining@local istio-1.10.3 $
```

```

istiotraining@local istio-1.10.3 $ kubectl apply -n default -f - <<EOF
> apiVersion: security.istio.io/v1beta1
> kind: PeerAuthentication
> metadata:
>   name: "default"
> spec:
>   mtls:
>     mode: STRICT
> EOF
peerauthentication.security.istio.io/default created
istiotraining@local istio-1.10.3 $

```

```

istiotraining@local istio-1.10.3 $ kubectl exec -it "$(kubectl get pod -l app=httpbin -n bar -o jsonpath={.items..metadata.name})" -c istio
-proxy -n bar -- curl "http://productpage.default:9080" -s -o /dev/null -w "%{http_code}\n"
000
command terminated with exit code 56
istiotraining@local istio-1.10.3 $

```

● الحالة الثانية (بعد تفعيل mTLS STRICT)

❖ طبقناه على الـ namespace default

❖ النتيجة:

الـ curl من الـ bar للـ default فشل

🔍 تفسير المشكلة بالظبط

❖ قبل الـ policy :

• plain HTTP = traffic
• كله شغال

❖ بعد STRICT :

• لازم :
• فقط mTLS ✓

• لكن bar :
• مش مجهز بالكامل (أو injection مش مطلوب)

❓ ليه حصل كذا؟

❖ لأن:

• default namespace :

• بيقبل mTLS فقط

• لكن الـ bar namespace :

• بيبعت plain traffic (مش mTLS)

⚠ القاعدة المهمة جدًا

❖ لو STRICT = destination .. وفي نفس الوقت الـ source مش بيستخدم mTLS
✓ اذا الـ request ← يتمتع ❌

Insight (Interview Question)

❖ ليه بنستخدم PERMISSIVE قبل STRICT ؟

▽ : PERMISSIVE

• يقبل :

• mTLS ✓ ○

• plain traffic ✓ ○

Migration mode 🚦

▽ ولاحظ إن STRICT :

• يقبل :

• فقط mTLS ○

Enforcement mode 🚦

🧠 الصورة الكبيرة

❖ بدون Istio : Service → Service (no verification)

❖ مع Istio + STRICT : Service → Envoy → mTLS → Envoy → Service

🔗 خلاصة

❖ Zero Trust حقيقي = mTLS STRICT

(يعني أي Service مش verified ← ممنوع تدخل ❌)

Authorization

الجزء ده بيشرح Authorization في Istio — وده طبقة مختلفة عن الـ Authentication

ناس كتير بتلخبط بينهم، فخلينا نفصلهم:

Authentication = "إنت مين؟"

• الـ service تثبت هويتها

✓ مثلاً باستخدام mTLS أو JWT

Authorization = "مسموح لك تعمل إيه؟"

• بعد ما اتأكدنا إنت مين، نقرر:

○ ينفع تدخل ولا لا؟

○ ينفع تعمل GET ؟

○ ينفع تعمل POST ؟

○ ينفع توصل للخدمة دي أصلاً؟

الفكرة الأساسية

❖ Istio بيطبق Authorization عن طريق Envoy sidecar

• يعني لما request يوصل: (Request → Envoy Proxy → Authorization Check → Allow / Deny → Service)
○ فالـ service نفسها مش محتاجة تضيف authorization logic في الكود.

➡ وده من أقوى مميزات Istio — وهو إن السيكيوريتي بيكون Centralized ومفصول عن التطبيق

أنواع الـ Actions

1. ALLOW

❖ يسمح للطلبات المطابقة فقط

❖ مثال:

السماح لـ productpage إنه يكلم reviews

2. DENY

❖ يمنع الطلبات المطابقة

❖ مثال:

منع POST requests

3. CUSTOM

❖ بيحوّل القرار لـ external authorization provider

• زي integration مع:

○ OPA

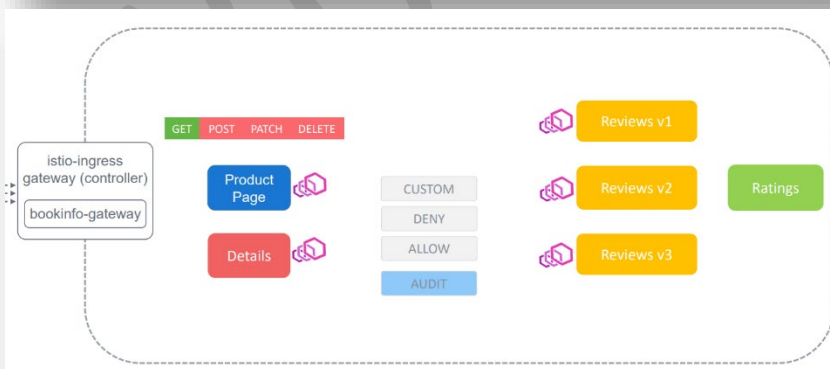
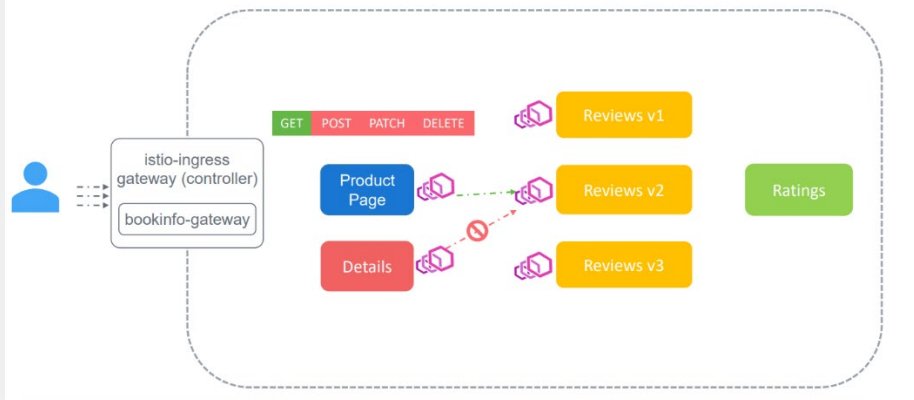
○ external auth service

4. AUDIT

❖ مش deny ولا allow

بس يسجل request للمراجعة /logging

Authorization Actions




```

apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: authdenypolicy
  namespace: bookinfo
spec:
  action: DENY
  rules:
  - from:
    - source:
        namespaces: ["bar"]
    to:
    - operation:
        methods: ["POST"]

```

المثال اللي الفيديو بيشرحه

❖ هو عايز يمنع:

▽ أي POST request جاية من namespace اسمه bar ورايحة لـ bookinfo

❖ يعني:

▽ ممنوع: (POST from bar → reviews)

❖ لكن:

▽ مسموح: (GET from bar → reviews)

معنى كل جزء

❖ action

action: DENY

(يعني أي request يطابق rule هيتمنع)

❖ from

مين اللي جاي؟

❖ source:

namespaces: ["bar"]

(يعني الطلبات الجاية من namespace bar)

❖ to

(العملية المطلوبة)

❖ operation:

methods: ["POST"]

(يعني POST فقط)

السيناريو العملي

❖ عندك services :

- productpage
- reviews
- details

▽ عايز تطبق policy ← تسمح فقط لـ productpage يعمل GET على reviews

▽ وتمنع:

• details → reviews

• أي POST / PUT

🚦 وده هو classic east-west access control

East-West vs North-South

❖ East-West

• يعني داخل الكلاستر ← (service → service)

🚦 وده اللي AuthorizationPolicy غالباً بتتحكم فيه

العلاقة مع mTLS

❖ Authorization غالبًا يعتمد على identity جاية من Authentication ❖

❖ يعني:

1. PeerAuthentication يثبت الهوية (mTLS)

2. AuthorizationPolicy يقرر السماح/المنع

سلسلة القرار: 🚦

Authenticate → Identify → Authorize

سؤال interview مشهور جدًا 🚩

❖ ليه نستخدم Istio Authorization بدل RBAC داخل التطبيق؟ ❖

❖ لأنها:

- centralized
- consistent
- enforced at proxy layer
- لا تحتاج تعديل application code
- easier auditing

الخلاصة:

❖ PeerAuthentication ← يثبت الهوية ❖

❖ AuthorizationPolicy ← يتحكم في الصلاحيات ❖

Demo Authorization

الفكرة الأساسية

هم عملوا حاجة خطيرة شوية (بس صح جداً في production) وهي :

إضافة Authorization Policy للـ default namespace

وبما إن مفيش selector محطوط .. فبالتالي الـ Policy هاتطبق

عالم (namespace-wide) ns +

وكان الـ AuthorizationPolicy كلها بدون rules فتبقى النتيجة ← Deny ALL traffic

يعني:

مفيش أي service تكلم أي service

حتى الـ ingress مش هيخش

كل حاجة هتتف

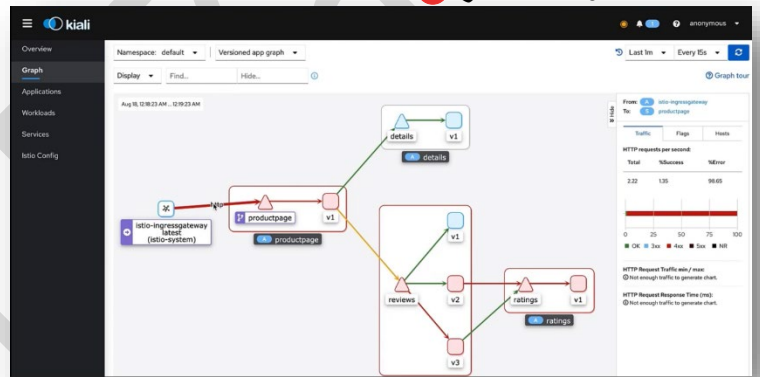
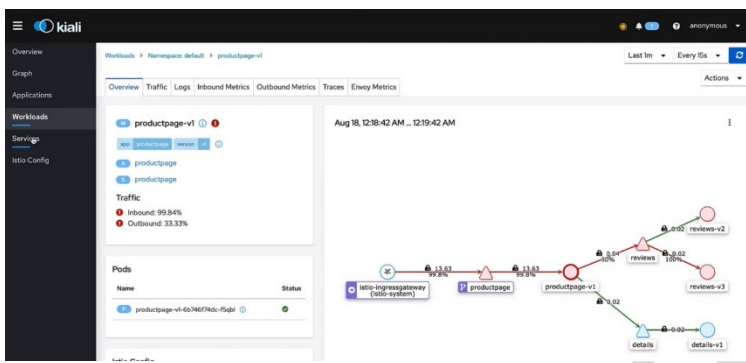
وده اللي شفتة:

RBAC access denied

Kiali كله أحمر

```
istiotraining@local istio-1.10.3 $ kubectl apply -f - <<EOF
> apiVersion: security.istio.io/v1beta1
> kind: AuthorizationPolicy
> metadata:
>   name: allow-nothing
>   namespace: default
> spec:
>   {}
> EOF
authorizationpolicy.security.istio.io/allow-nothing created
istiotraining@local istio-1.10.3 $
```

مفيش allow rules ← وبالتالي
هايبقى كل حاجة blocked



ليه ده بيحصل؟

في Istio :

لو عندك AuthorizationPolicy ومافيش rule تسمح :

✓ يبقى default = DENY

المرحلة 2: نرجع النظام واحدة واحدة

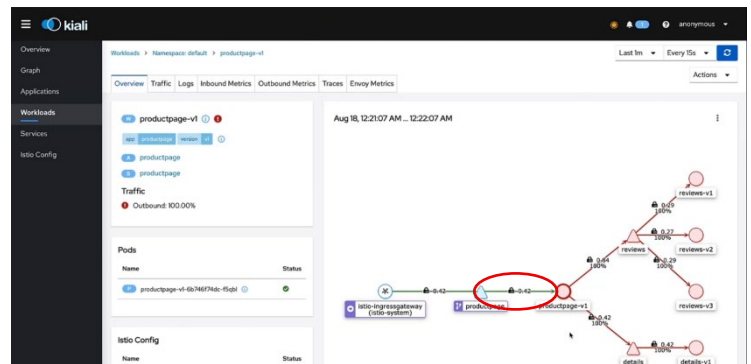
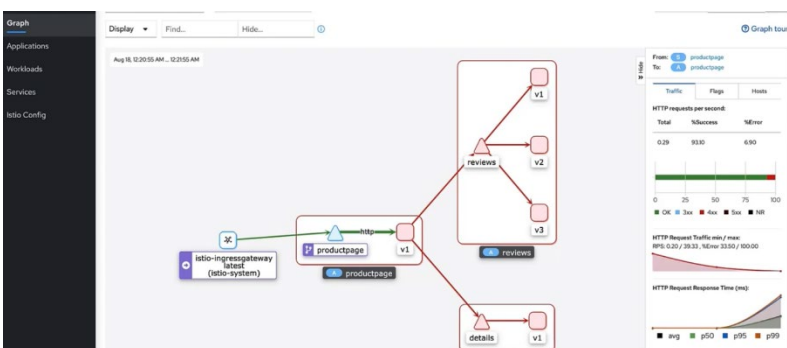
1 السماح للـ productpage

النتيجة:

الصفحة ظهرت جزئياً

لكن باقي services لسه مش شغالة

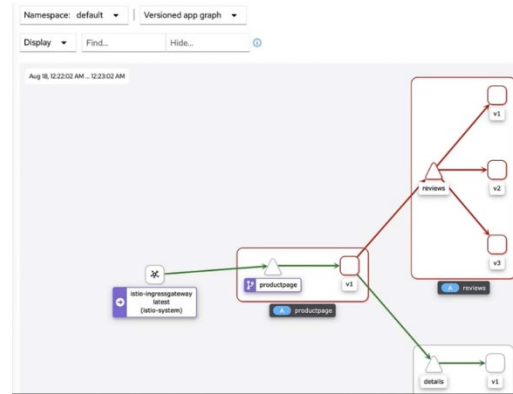
```
istiotraining@local istio-1.10.3 $ kubectl apply -f - <<EOF
> apiVersion: security.istio.io/v1beta1
> kind: AuthorizationPolicy
> metadata:
>   name: "productpage-viewer"
>   namespace: default
> spec:
>   selector:
>     matchLabels:
>       app: productpage
>   action: ALLOW
>   rules:
>   - to:
>     - operation:
>       methods: ["GET"]
> EOF
```



2 السماح لـ details

يعني:

productpage بس هو اللي يقدر يكلم details



3 السماح لـ reviews

4 السماح لـ ratings

نفس الفكرة

كده النظام كله رجع يشتغل

أهم Concept في الدرس

Default Deny Strategy

بدل ما تقول:

• اسمح لكل حاجة وامنع شوية

• بتقول:

• امنع كل حاجة واسمح بس للي محتاجه

• وده:

○ أكثر أماناً

○ أقل risk

○ مناسب production

1. Apply deny-all policy
2. Allow productpage → GET
3. Allow productpage → details
4. Allow productpage → reviews
5. Allow reviews → ratings

الفلو الكامل

⚠ ملاحظة مهمة جدًا

AuthorizationPolicy is additive (ALLOW-based)

يعني:

○ أي rule ALLOW = يفتح access

○ غير كده = blocked

- ❖ كله أحمر ← deny all
- قبعدين بدأ يتحول أخضر تدريجي
- 🚦 كل ما تضيف policy ← traffic يرجع وهكذا

(Interview Level) Insight

- ❖ ليه نعمل كده؟
- 🔒 Least Privilege Principle ← عشان نحقق:
- ❖ وبالتالي كل service :
- تكلم بس اللي محتاجاه
- تستخدم methods محددة

مثال real-life

- ❖ بدل : Any service → reviews (ALL METHODS)
- ❖ تخليها : productpage → reviews (GET only)

الخلاصة

- ❖ قبل:
- كل services مفتوحة على بعض
- Risk عالي
- ❖ بعد:
- كل حاجة مقفولة
- وانت بتفتح بس اللي محتاجه

العلاقة مع باقي Istio

دوره	Feature
يثبت الهوية	PeerAuthentication
يتحكم في الوصول	AuthorizationPolicy
يحدد routing	VirtualService

Certificate Management

الفكرة الأساسية

كل Service في Istio لازم يكون لها:

- Identity
- Certificate
- Private Key

عشان تقدر : Service A ↔ Service B (mTLS secure communication)

الفلو الكامل (خطوة بخطوة)

1 Service تبدأ

- Pod يشتغل
- Sidecar (Envoy + Istio Agent) يبدأ معاه

2 إنشاء الهوية (CSR)

Istio Agent يعمل : Private Key + CSR (Certificate Signing Request)
CSR = طلب للحصول على Certificate

3 إرسال الطلب لـ Istiod

Agent ← Istiod (CA)

4 التحقق (Validation)

- Istiod (CA) يتأكد:
- هل الـ service دي legit ؟
- هل جاية من Kubernetes فعلاً؟
- هل الـ ServiceAccount صح؟

5 توقيع الشهادة

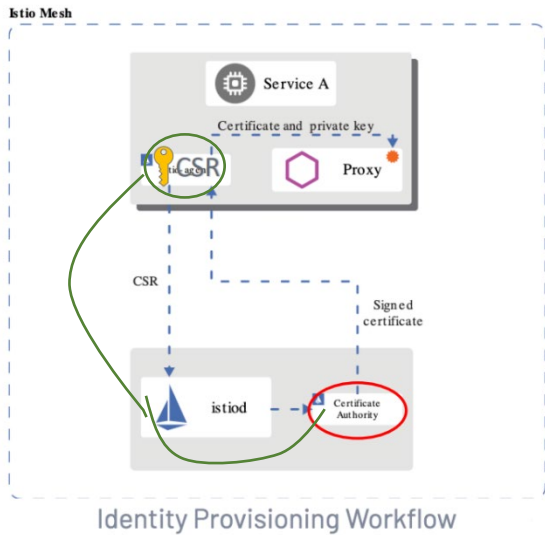
لو كل حاجة تمام:
Istiod → Sign CSR → Generate Certificate

6 إرسال الشهادة

- Istiod → Agent → Envoy Proxy
- دلو قتي Envoy عنده:
- Certificate
- Private Key

7 استخدام mTLS

أي communication بين services : Envoy ↔ Envoy (TLS handshake + identity verification)
يعني كل service بتأكد من الثانية



8 Rotation (مهم جدًا)

❖ الشهادات:

• لها expiration

❖ Istio Agent :

▪ يراقب ← يحدد ← يعيد نفس العملية وبدون downtime 🚦

🔥 ليه ده مهم؟ ▽

❖ Zero Trust Security

▪ كل request لازم:

○ يتوثق

○ يتشفّر

❖ Automatic

▪ مش محتاج:

○ تكتب SSL في الكود

○ تدير certs بنفسك

❖ Transparent

▪ التطبيق نفسه:

○ مش هاحس بأي حاجة

⚠ نقطة Production مهمة جدًا

❖ Istio built-in CA كويسة لـ:

• dev

• testing

❖ لكن في production :

▪ لازم تستخدم:

○ HashiCorp Vault

○ أو CA خارجي (PKI system)

🚦 ليه؟

• أمان أعلى

• تحكم أفضل في certificates

• إمكانية audit

• تخزين مفاتيح خارج الكلاستر

الفرق عن Kubernetes TLS العادي

Istio	Kubernetes TLS
Automatic	Manual غالبًا
Rotating certs	Static certs
Proxy-level	App-level

🎯 Insight مهم (Interview)

❖ إزاي Istio بيوفر Identity لكل service ؟

• باستخدام certificates مربوطة بـ ServiceAccount

• وبتوزع تلقائيًا من Istiod CA

Demo – Certificate Management

أولاً : Istio Certificate Management في Istio

□ الفكرة الأساسية:

أي Service في الـ mesh لازم يكون لها Identity علشان تقدر تتكلم مع باقي الخدمات بشكل آمن (mTLS)

□ الفلو بيحصل كالتالي:

1. الـ Pod يبدأ

2. الـ Istio Agent جوه الـ sidecar :

○ يعمل Private Key

○ يعمل CSR (Certificate Signing Request)

3. يبيعت الـ CSR لـ Istiod (CA)

4. الـ CA :

○ تتحقق من الهوية

○ تمضي (sign) الـ certificate

5. يرجع :

○ Certificate + Key ← الـ Envoy Proxy

6. يحصل :

○ Rotation تلقائي قبل انتهاء الصلاحية

🔗 النتيجة : كل Service عندها شهادة ← تقدر تعمل mTLS بشكل أوتوماتيك

🧠 نقطة مهمة جدًا

❖ Istio بيبقى فيه Built-in CA

• لكن في Production مش مفضل تستخدمه

❖ الأفضل تستخدم :

○ Vault

○ أو External PKI

🚩 ثانياً : استخدام Custom Root Certificate

□ إحنا هنا بنقول لـ Istio:

" أنا مش عايزك تستخدم الـ CA الافتراضي بتاعك... أنا هديك Certificates بتاعتي أنت تشتغل بيها"

□ وده مهم جدًا في الـ Production علشان:

• تتحكم في الـ Security

• تستخدم CA موثوق (زي Vault)

• أو حتى CA داخلي خاص بشركتك

□ هنا أنت بتتحكم في الـ CA بنفسك بدل Istio default

الجزء 1: إنشاء الـ Root Certificate

عمل فولدر:

- ده المكان اللي هنحط فيه كل الشهادات

Create a folder for our cert, in our istio root directory "ca-certs"

إنشاء Root Certificate

بيطلعك 4 ملفات:

الملف	معناه
root-ca.conf	OpenSSL J config
root-cert.csr	طلب إنشاء شهادة
root-cert.pem	الشهادة نفسها
root-key.pem	المفتاح السري

To generate a root certificate

root-ca.conf >> is the configuration for OpenSSL to generate the root cert

root-cert.csr >> is the generated CSR for the root certificate

root-cert.pem >> is the generated root certificate

root-key.pem >> is the generated root key

- مهم جداً:

الـ Root Cert = أخطر حاجة و مينفعش تستخدمها مباشرة ← و لالا ازم تعمل منها Intermediate

الجزء 2: إنشاء Intermediate Certificates

في الصورة برضه:

- بدل مال الـ Root يوقع الشهادات مباشرة

- نخلي الـ Intermediate CA هو اللي

يعمل signing

(ما تستخدمش الـ root مباشرة (Security Best Practice)

To create our immediate certs

الجزء 3: تنظيف Istio القديم

عشان:

- ❖ Istio بيكون already مستخدم الـ CA بتاعه
- ❖ وبالتالي لازم تمسحه عشان نقوله استخدم الجديد

الحل:

kubectl delete ns istio-system

أو تبدأ Cluster جديد

تنظيف الكلاستر
و عمل Clean-up
عشان نبدأ علي
نضافة

```
istiotraining@local istio-1.10.3 $ mkdir ca-certs
istiotraining@local istio-1.10.3 $ cd ca-certs/
istiotraining@local ca-certs $ make -f ../tools/certs/Makefile.selfsigned.mk root-ca
generating root-key.pem
Generating RSA private key, 4096 bit long modulus
.....++
.....++
e is 65537 (0x10001)
generating root-cert.csr
generating root-cert.pem
Signature ok
subject=/O=Istio/CN=Root CA
Getting Private key
istiotraining@local ca-certs $ ls
root-ca.conf  root-cert.csr  root-cert.pem  root-key.pem
istiotraining@local ca-certs $
```

```
istiotraining@local ca-certs $ make -f ../tools/certs/Makefile.selfsigned.mk localcluster-cacerts
generating localcluster/ca-key.pem
Generating RSA private key, 4096 bit long modulus
.....++
.....++
e is 65537 (0x10001)
generating localcluster/cluster-ca.csr
generating localcluster/ca-cert.pem
Signature ok
subject=/O=Istio/CN=Intermediate CA/L=localcluster
Getting CA Private Key
generating localcluster/cert-chain.pem
Intermediate inputs stored in localcluster/
done
rm localcluster/cluster-ca.csr localcluster/intermediate.conf
istiotraining@local ca-certs $
```

```
istiotraining@local ca-certs $ cd localcluster
istiotraining@local localcluster $ ls
-bash: ls: command not found
istiotraining@local localcluster $ ls
ca-cert.pem  ca-key.pem  cert-chain.pem  root-cert.pem
istiotraining@local localcluster $
```

```
istiotraining@local localcluster $ kubectl delete namespace istio-system
namespace "istio-system" deleted
istiotraining@local localcluster $ cd ..
istiotraining@local ca-certs $ cd ..
istiotraining@local istio-1.10.3 $ ./samples/bookinfo/platform/kube/cleanup.sh
namespace ? [default]
using NAMESPACE=default
destinationrule.networking.istio.io "details" deleted
destinationrule.networking.istio.io "productpage" deleted
destinationrule.networking.istio.io "ratings" deleted
destinationrule.networking.istio.io "reviews" deleted
virtualservice.networking.istio.io "bookinfo" deleted
gateway.networking.istio.io "bookinfo-gateway" deleted
Application cleanup may take up to one minute
service "details" deleted
serviceaccount "bookinfo-details" deleted
deployment.apps "details-v1" deleted
service "ratings" deleted
serviceaccount "bookinfo-ratings" deleted
deployment.apps "ratings-v1" deleted
service "reviews" deleted
serviceaccount "bookinfo-reviews" deleted
deployment.apps "reviews-v1" deleted
deployment.apps "reviews-v2" deleted
deployment.apps "reviews-v3" deleted
service "productpage" deleted
serviceaccount "bookinfo-productpage" deleted
deployment.apps "productpage-v1" deleted
Application cleanup successful
istiotraining@local istio-1.10.3 $ kubectl create namespace istio-system
namespace/istio-system created
istiotraining@local istio-1.10.3 $
```

الجزء 4: إنشاء Secret فيه الشهادات

هنا بتقول لـ Istio :

دي الشهادات اللي هتستخدمها بدل الافتراضي

Create istio namespace again and create a secret including all these certs

الجزء 5: إعادة تثبيت Istio

اللي بيحصل:

- Istiod يقرأ الشهادات من الـ Secret
- يبدأ يستخدمها كـ CA

كمان:

- بيرجعك:

Kiali

Prometheus

Grafana

Will install istio back again so that istio certificate authority will read these certs and keys from the secret mount files

Also we get our Kiali, Grafana, Prometheus add-ons back

الجزء 6: تشغيل التطبيق (Bookinfo)

Now will get our book-info app and some traffic default rules from the samples folder

الجزء 7: تفعيل mTLS

معناها:

- كل الترافيك لازم يكون مشفر (mTLS)

Deploy a policy for our workloads to only accept mutual TLS traffic

```
istiotraining@local istio-1.10.3 $ kubectl apply -f - <<EOF
> apiVersion: security.istio.io/v1beta1
> kind: PeerAuthentication
> metadata:
>   name: "default"
> spec:
>   mtls:
>     mode: STRICT
> EOF
peerauthentication.security.istio.io/default created
istiotraining@local istio-1.10.3 $
```

To check if the workloads are signed with the exact same certs that we just plugged in

```
istiotraining@local istio-1.10.3 $ kubectl exec "$(kubectl get pod -l app=details -o jsonpath={.items..metadata.name})" -c istio-proxy -- openssl s_client -showcerts -connect productpage:9080 > httpbin-proxy-cert.txt
140109205299648:error:0200206F:system library:connect:Connection refused:../crypto/bio/b_sock2.c:110:
140109205299648:error:2008A067:BIORoutines:BIOR_connect:connect error:../crypto/bio/b_sock2.c:111:
connect:errno=111
command terminated with exit code 1
istiotraining@local istio-1.10.3 $
```

```
istiotraining@local istio-1.10.3 $ kubectl exec "$(kubectl get pod -l app=details -o jsonpath={.items..metadata.name})" -c istio-proxy -- openssl s_client -showcerts -connect productpage:9080 > httpbin-proxy-cert.txt
depth=2 0 = Istio, CN = Root CA
verify error:num=19:self signed certificate in certificate chain
DONE
istiotraining@local istio-1.10.3 $
```

هنا ظهر الخطأ !

يعني إيه error=111

- ❖ ببساطة : فيه محاولة اتصال حصلت .. لكن الطرف الثاني رفض الإتصال
- ❖ يعني في السيناريو بتاعنا احنا كنا بنحاول نعمل اتصال بـ service ثانية من داخل الـ Envoy sidecar بس الإتصال ده إترفض
- الأسباب بتاع الـ Error ده ان ساعات الخدمة بتكون لسا مش جاهزة والأفضل تستتي 15 ثانية كدا عما :
 - الـ pods تقوم
 - أو الـ sidecar يأخذ الـ config من الـ istiod

يعني إيه error 19

ببساطة : OpenSSL بيقولك:

"أنا شايف شهادة Self-Signed ومش واثق فيها"

ليه طبيعي؟

- لأنك عامل CA بنفسك (مش public)
- فـ OpenSSL مش بيد trust فيها

وده Expected Behavior وليس مش Error حقيقي

معناها	الحالة
مفیش اتصال أصلاً	errno=111
الاتصال تم بس الشهادة self-signed	verify error 19

الفرق المهم جدًا

الجزء 9: تنظيف الشهادة من الـ Output

بيعمل:

- يشيل الـ Logs
- يسيب الشهادات بس

```
istiotraining@local istio-1.10.3 $ sed -ne '/-----BEGIN CERTIFICATE-----/,/-----END CERTIFICATE-----/p' httpbin-proxy-cert.txt | sed 's/\s*/>' > certs.pem
```

الجزء 10: تقسيم الشهادات

بقي عندك:

- root
- intermediate
- workload

```
istiotraining@local istio-1.10.3 $ split -p "-----BEGIN CERTIFICATE-----" certs.pem proxy-cert-
```


الجزء 11: مقارنة الشهادات

الهدف:

نتأكد إن:

- الشهادة اللي في الترافيك = نفس اللي انت عملتها

النتيجة: إن الـ istio فعلاً بيستخدم الشهادات بتاعتك

```
~istio-1.10.3 -- -bash
istiotraining@local istio-1.10.3 $ openssl x509 -in ca-certs/localcluster/root-cert.pem -text -noout > /tmp/root-cert.crt.txt
istiotraining@local istio-1.10.3 $ openssl x509 -in ./proxy-cert-3.pem -text -noout > /tmp/pod-root-cert.crt.txt
istiotraining@local istio-1.10.3 $ diff -s /tmp/root-cert.crt.txt /tmp/pod-root-cert.crt.txt
Files /tmp/root-cert.crt.txt and /tmp/pod-root-cert.crt.txt are identical
istiotraining@local istio-1.10.3 $
```

```
~istio-1.10.3 -- -bash
istiotraining@local istio-1.10.3 $ openssl x509 -in ca-certs/localcluster/ca-cert.pem -text -noout > /tmp/ca-cert.crt.txt
istiotraining@local istio-1.10.3 $ openssl x509 -in ./proxy-cert-2.pem -text -noout > /tmp/pod-cert-chain-ca.crt.txt
istiotraining@local istio-1.10.3 $ diff -s /tmp/ca-cert.crt.txt /tmp/pod-cert-chain-ca.crt.txt
Files /tmp/ca-cert.crt.txt and /tmp/pod-cert-chain-ca.crt.txt are identical
istiotraining@local istio-1.10.3 $
```

الجزء 12: التحقق من الـ Chain

```
~istio-1.10.3 -- -bash
istiotraining@local istio-1.10.3 $ openssl verify -CAfile <(cat ca-certs/localcluster/ca-cert.pem ca-certs/localcluster/root-cert.pem) ./proxy-cert-1.pem
./proxy-cert-1.pem: OK
istiotraining@local istio-1.10.3 $
```

الهدف :

← يعني كل حاجة مربوطة صح

OK

← النتيجة

Root → Intermediate → Workload

الخلاصة (مهم جداً)

أنت عملت:

1. أنشأت Root CA
2. عملت Intermediate CA
3. خليت Istio يستخدمهم
4. فعلت mTLS
5. تأكدت إن الترافيك فعلاً مت Signed بنفس الشهادات
6. فهمت ليه Error 19 طبيعي

الصورة الكبيرة بقى

بدل ما Istio يعمل كده:



⚠️ليه Root CA ما تستخدمش مباشرة؟

- لو اتسربت = انتهى الأمان كله
- لذلك :
- Root → Offline
- Intermediate ← اللي يوقع الشهادات

💡 الخلاصة (أهم 3 أفكار)

1. Istio بيدير الشهادات أوتوماتيك (CSR + Signing + Rotation)
2. كل Service عندها Identity ← أساس mTLS
3. في Production :
 - استخدم Custom CA
 - مش الـ built-in

🔗 ربط ده بكل اللي قبله (مهم جدًا)

- Authentication (mTLS) ← بيعتمد على الشهادات دي
- Authorization ← بيستخدم الهوية الناتجة من الشهادات
- Security كله في Istio مبني على الجزء ده. 🚦

Demo Visualizing Metrics with Prometheus and Grafana

أولاً: الفكرة العامة

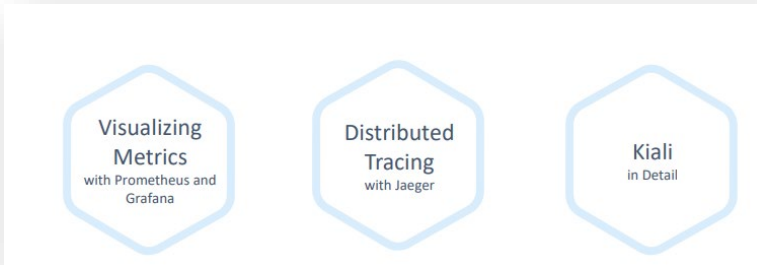
Metrics Istio

❖ يعني Istio ببشغل داخل الـ cluster وبيجمع بيانات زي:

- عدد requests
- latency (زمن الاستجابة)
- success/failure rates
- traffic بين الخدمات

❑ في Istio عندك 3 أدوات أساسية للمراقبة:

- Prometheus ← يجمع Metrics
- Grafana ← يعرض Dashboards
- Kiali ← يوريك topology + traffic



Prometheus بيعمل إيه؟

Prometheus

- يجمع Metrics من:

○ Istio Control Plane

○ Envoy (Data Plane)

○ Applications

- وبعدين بيخزنها كـ time-series data

- وبيخليها جاهزة للاستعلام (queries)

✚ يعني هو "database للمراقبة"

✚ مثال مهم جداً:

■ `istio_requests_total`

✓ ده بيقولك : عدد كل الـ requests اللي عدت في الـ mesh

تقدر تعمل Filtering (ودي أهم نقطة)

❖ حسب service : `istio_requests_total{destination_service="productpage"}`

✓ تشوف requests اللي رايحة لـ productpage بس

❖ حسب (subset) version ← `destination_version="v3"`

✓ تشوف traffic على version معين

■ دي قوية جداً في:

○ A/B Testing

○ Canary Deployments

```
istiotraining@local istio-1.10.3 $ istioctl dashboard prometheus
http://localhost:9090
```

To open Prometheus

```
istiotraining@local istio-1.10.3 $ kubectl get svc prometheus -n istio-system
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
prometheus    ClusterIP     10.98.236.105 <none>         9090/TCP   85m

istiotraining@local istio-1.10.3 $ kubectl get svc grafana -n istio-system
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
grafana       ClusterIP     10.96.79.92   <none>         3000/TCP   85m

istiotraining@local istio-1.10.3 $
```

Make sure Prometheus & Grafana are running in our cluster

istio_requests_total{destination_service="reviews.default.svc.cluster.local", destination_version="v3"}

```
istiotraining@local istio-1.10.3 $ istioctl dashboard grafana  
http://localhost:3000
```

To open Grafana

الخلاصة

- Istio = يولد metrics من حركة microservices
- Prometheus = يجمع ويخزن البيانات
- Grafana = يعرضها في dashboards سهلة الفهم

Grafana (Visualization)

❖ Prometheus لوحده :

❌ مش مريح بصرياً

✓ أرقام + queries

▽ هنا بييجي دور Grafana :

❖ بيأخذ البيانات من Prometheus ويعرضها في شكل:

- dashboards
- graphs
- charts

🔗 يعني هو "واجهة عرض جميلة للـ metrics"

أهم Dashboards في Istio

1. Control Plane Dashboard

- CPU / Memory
- Errors
- Config sync

2. Mesh Dashboard

• نظرة عامة على :

- Services ○
- Workloads ○
- Success Rate ○
- Errors ○

🔗 ده أهم Dashboard عملياً

3. Service Dashboard

- Requests
- Latency
- Errors

4. Workload Dashboard

- inbound / outbound traffic
- dependencies بين services

5. Performance Dashboard

- latency
- throughput

🔗 الربط مع Kiali

- Kiali ← بيوريك شكل الترافيك (graph)
- Prometheus ← بيوريك الأرقام
- Grafana ← بيوريك التحليل

🔗 التلاتة مع بعض = Observability كاملة

1. Debugging

- service بطيئة؟
- ← شوف latency في Grafana
- ← شوف requests في Prometheus

2. Monitoring

- في Errors ؟
- success rate نازل ؟

3. Traffic Analysis

- مين بيكلم مين؟
- انهبي version واخذ traffic ؟

4. Capacity Planning

- هل الخدمة تستحمل load أكثر؟

⚠ نقطة مهمة جدًا

❖ Metrics في Istio فيها Dimensions (Labels)

❖ زي:

- source
- destination
- version
- response_code
- ده اللي بيخليها powerful جدًا 🚀

🌿 الخلاصة

- Prometheus = يجمع الداتا
- Grafana = يعرضها
- Kiali = يدك الصورة الكبيرة

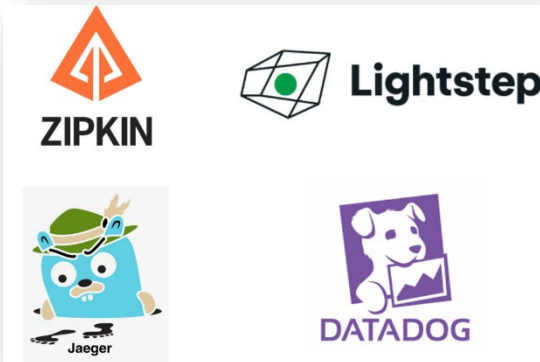
🚀 وكل ده شغال automatically بسبب Envoy sidecars

Demo Distributed Tracing with Jaeger

الفكرة الأساسية

عشان تفهم الـ behavior بتاع services في الـ Service Mesh فلازم تراقب كل request وهو ماشي بين الخدمات

✓ وده اسمه: Distributed Tracing



يعني إيه Distributed Tracing ؟

❖ هو إنك:

- تمسك الـ request من أول ما يدخل السيستم
- وتتبع رحلته وهو بيعدي على كل Service

مثال: User → Product Service → Reviews → Ratings

▪ بدل ما تبص على Logs لكل Service لوحدها ← فبتشوف الرحلة كاملة في Trace واحد

فأيدته إيه؟

1. تفهم dependencies

تعرف:

- مين بينادي مين
- ترتيب الخدمات

2. تكتشف latency

تعرف:

- فين التأخير اللي حصل
- أي Service هي السبب

مثال:

- Product سريع وتمام
- Reviews سريع وتمام
- Ratings بياخد 3 ثواني ← اذا المشكلة هنا

Istio بيعمل ده إزاي؟

❖ Istio بيستخدم: Envoy Proxy (Sidecar)

❖ كل request :

- يعدي على Envoy

❖ Envoy يسجل معلومات عنه (Trace)

الخلاصة

• Distributed Tracing

Tools اللي بيدعمها

❖ Istio يقدر بيعت الـ tracing data لـ tools زي:

- Jaeger ← الأشهر في الكورس

- Zipkin - Lightstep - Datadog

في microservices

لو عندك request واحد بيعدي على كذا service : User → productpage → details → reviews → ratings

المشكلة: ← لو في delay أو error .. فمين السبب وازاي اعرفه؟!

هنا بييجي دور Jaeger

```
istiotraining@local istio-1.10.3 $ istioctl dashboard jaeger
http://localhost:16686
```

To Open Jaeger

المفاهيم الأساسية

1. Trace
 - رحلة request كاملة بين كل services
 2. Span
 - خطوة واحدة جوه الـ trace
 - كل service = span
- مثال:
- productpage → span
 - details → span
 - reviews → span

السيناريو اللي حصل :

- الحالة الطبيعية
- كل الـ spans : سريعة ومفيش أي Errors

- بعد ما عمل (delay) باستخدام Fault Injection
- يعني details service بقى بياخر الرد

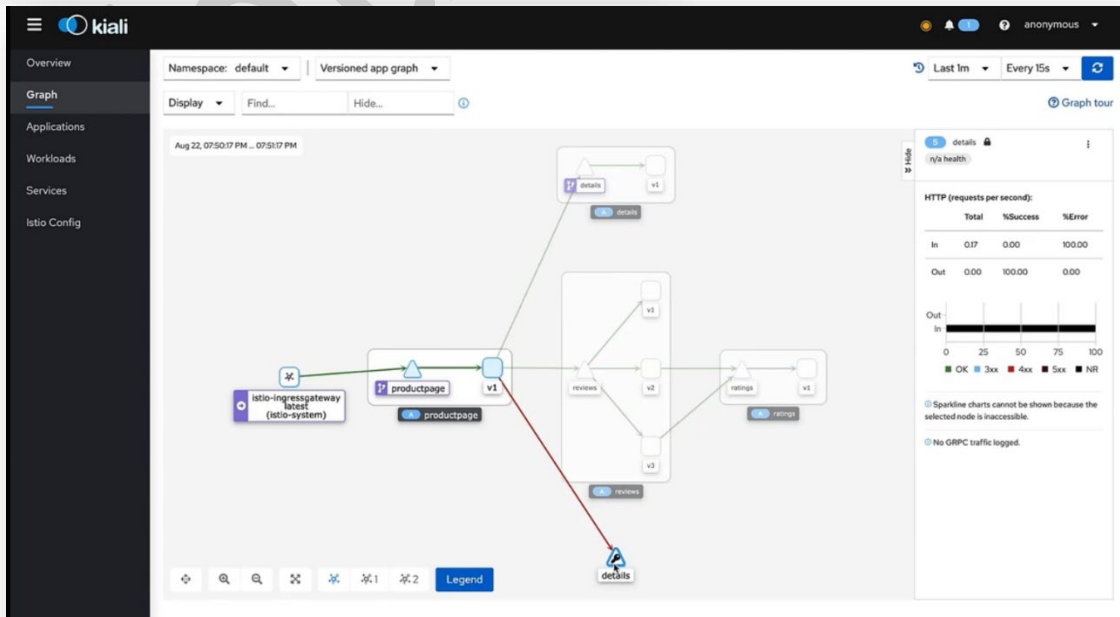
النتيجة :

- Productpage

يستنى 3 ثواني

- وبعدها يعمل timeout

```
istiotraining@local istio-1.10.3 $ kubectl apply -f - <<EOF
> apiVersion: networking.istio.io/v1alpha3
> kind: VirtualService
> metadata:
>   name: details
> spec:
>   hosts:
>   - details
>   http:
>   - fault:
>     delay:
>       percentage:
>         value: 70.0
>       fixedDelay: 7s
>     route:
>     - destination:
>       host: details
>       subset: v1
> EOF
```



Jaeger كشف المشكلة إزاي؟

1. Filtering

○ قدر يعمل:

- filter على service (زي productpage)
- أو reviews

2. فتح Trace

○ شاف:

- إن ال spans مترتبة .. وكم ان كل span خد وقت قد إيه

3. اكتشاف المشكلة

○ لقي:

- span بتاع details :

○ بطيء جدًا

- span بتاع productpage :

○ انتهى ب error بعد 3s

🔗 وبالتالي فاله RCA واضح: وهو إن :

details هو السبب

⚠ نقطة ذكية جدًا

- مش كل requests فيها error

• ليه؟

- لأنه عامل: delay على 70% من الترافيك

🔗 ذلك:

- بعض traces فيها error

- بعض traces تمام

💡 ده realistic behavior

🔗 الربط مع باقى الأدوات

- مع Kiali

• بيقولك:

- في مشكلة في (red edge) details

- مع Prometheus

• يقولك:

- latency عالي errors / زادت

- مع Grafana

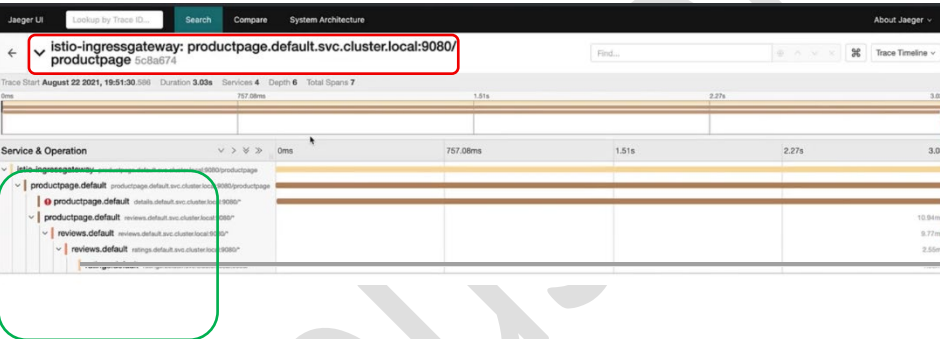
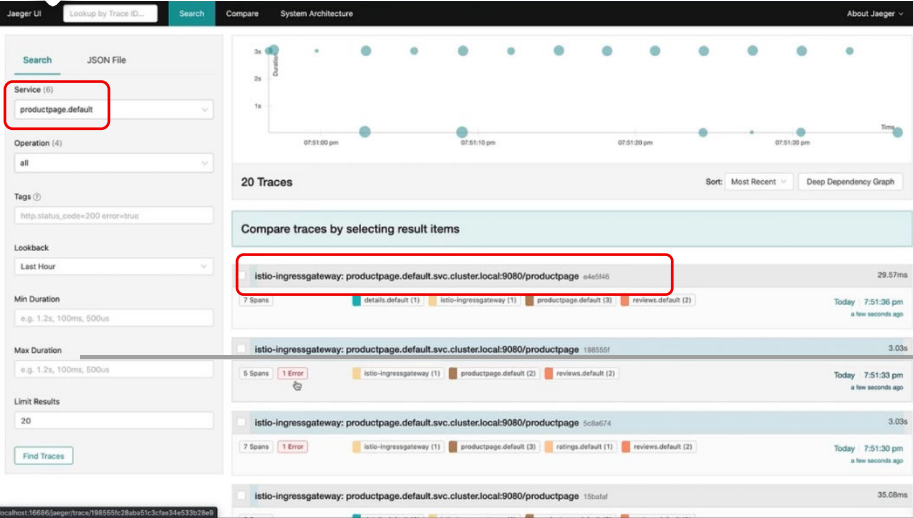
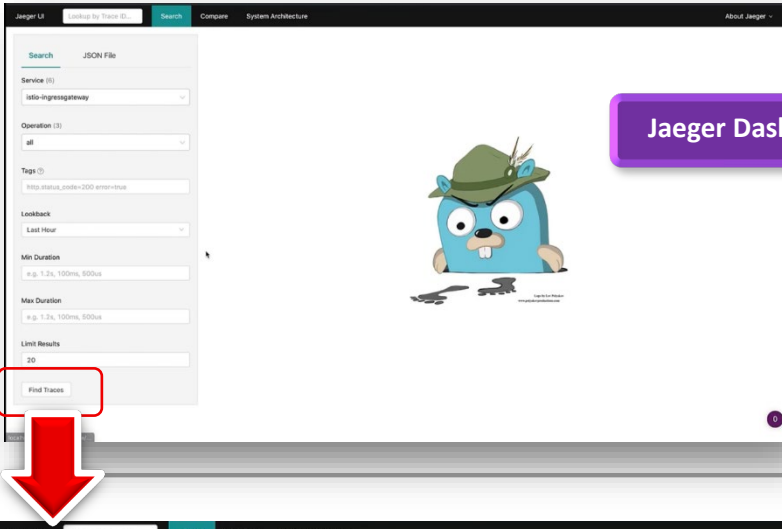
• يعرض charts

- مع Jaeger

• يقولك:

- " المشكلة حصلت في الحنة دي بالظبط "

Jaeger Dashboard



Note that each line here is representing a span

🌟 سيناريو Debug كامل (مهم جداً في الشغل)

1. user اشتكى: "السيستم بطيء"
 2. تفتح Kiali :
 - تلاقى service لونها أحمر
 3. تفتح Grafana :
 - تلاقى latency عالي
 4. تفتح Jaeger :
 - تحدد trace
 - تشوف span البطيء
- 🚀 ونقوم مديها واحدة Here we go خلاص شكرا عرفنا السبب

💡 أهم Use Cases لـ Jaeger

- Root cause analysis
- Performance bottlenecks
- Debugging timeouts
- فهم dependencies بين services

⚠️ ملاحظات مهمة

- Jaeger يستخدم sampling ← يعني مش كل requests بتتسجل
- عشان كده قال:
50 traces ≈ كام ثانية بس

🚀 الخلاصة

- request = Trace كامل
- Span = خطوة جوه الرحلة
- Jaeger = يوريك الرحلة دي بالتفصيل

🚀 ودي أقوى أداة تفهم: "ليه السيستم باظ؟"

Demo Kiali in Details

أولاً: إيه هو Kiali؟

Kiali هو UI Tool يبيشغل مع Istio

وبيستخدم عشان:

- تشوف الـ Service Mesh بشكل Visual
- تراقب الترافيك بين الخدمات
- تكتشف المشاكل (Errors / Latency)
- وتعمل Configuration من غير YAML

يعني بدل ما تبص على Logs بس ← عندك Dashboard فاهم كل حاجة

أهم Features

1. Overview

- بيديك نظرة عامة على:

- Services
- Workloads
- Namespaces

- يمكن تشوف:

- Health (سليم ولا فيه مشاكل)
- mTLS status

ده أول مكان تروحله لو في Incident

2. Graph (أهم Feature)

- دي أهم حاجة في Kiali:
- بتشوف:

مين بيكلم مين (Service → Service)

نسبة الترافيك

Errors (بالأحمر)

Latency

- يمكن تضيف عليها:

Request Rate

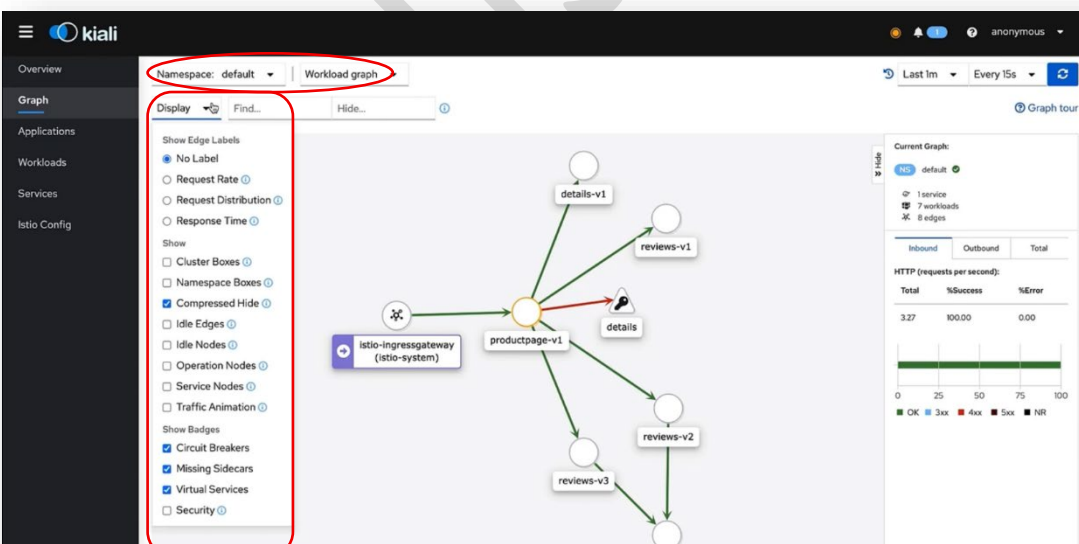
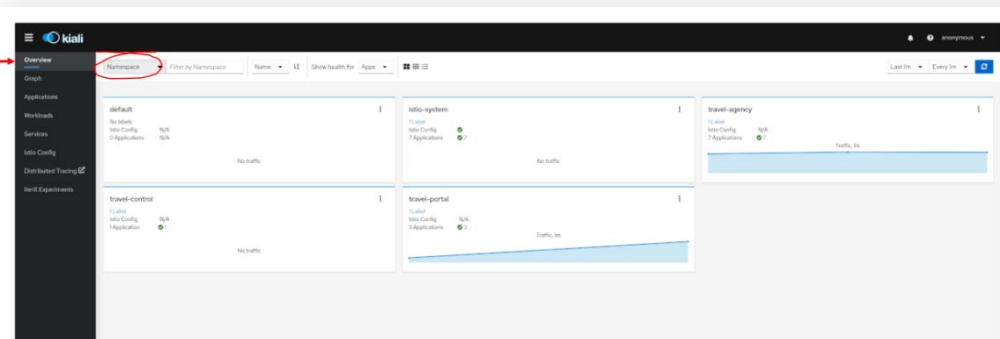
Response Time

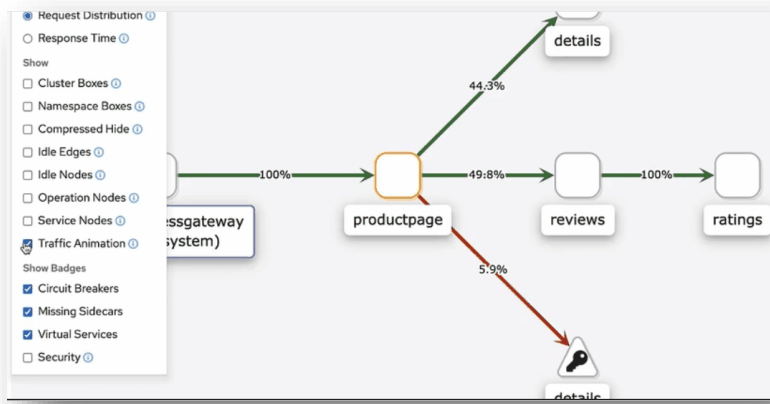
Traffic Distribution (%)

مثال:

لو Service لونها أحمر ← غالباً:

- timeout
- أو service down
- أو auth issue



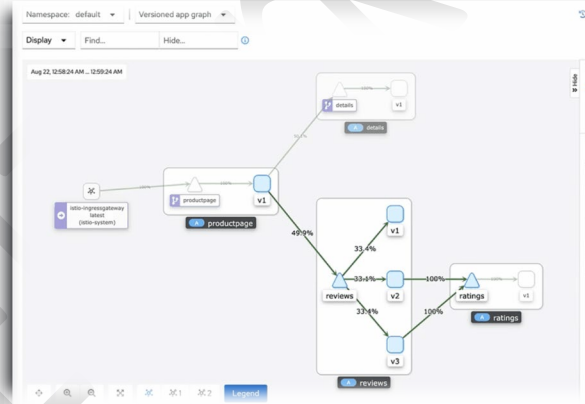
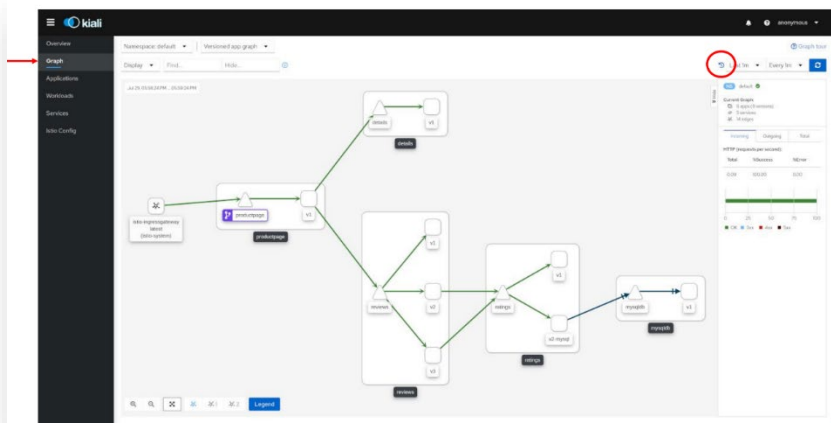


Traffic Animation وكم ان ممكن تصنيف

- بيخلي الترافيك يتحرك قدامك Live
- مفيد جدًا عشان :
 - تفهم الـ flow
 - تلاحظ bottlenecks

Replay Mode .3

- ترجع تشوف الترافيك في وقت سابق
- دي قوية جدًا في debugging ← بدل ما المشكلة تكون راحت وأنت مش فاهم حصل إيه وإيه سببها



Service Drill-down .4

- لما تدوس على Service :
 - هتلاقي نفسك داخل على تفاصيل زي :
 - Inbound / Outbound traffic
 - Metrics
 - Logs
 - (integrated with Jaeger) Traces

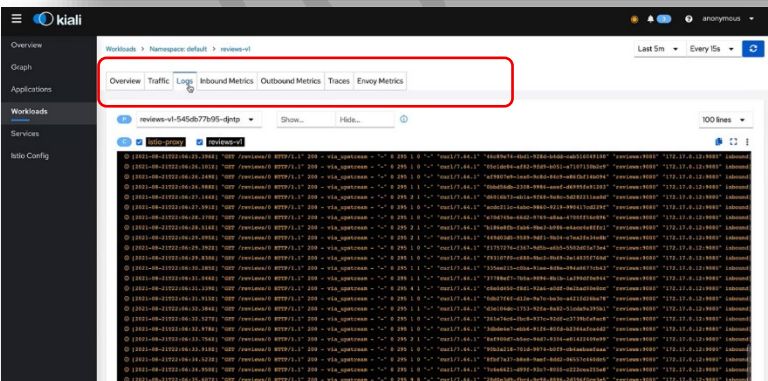
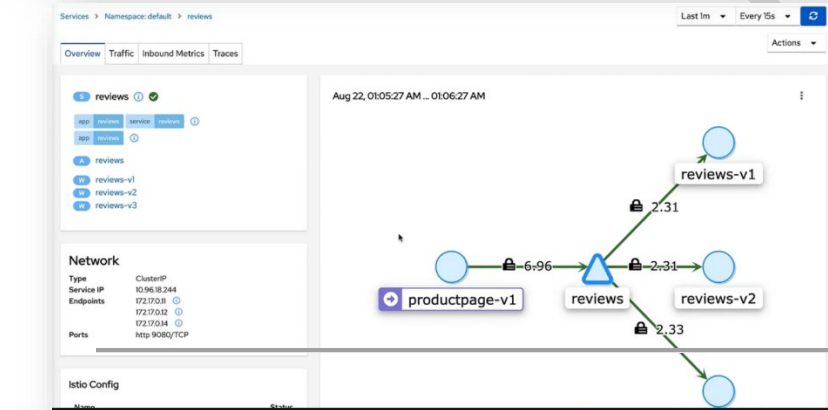
Workload View .5

- تفاصيل Pod :

- Logs (App + Envoy)
- Metrics
- Requests
- مفيدة جدًا في RCA

Istio Config Management .6

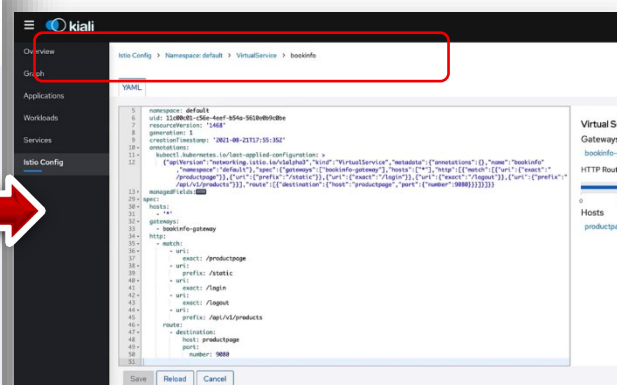
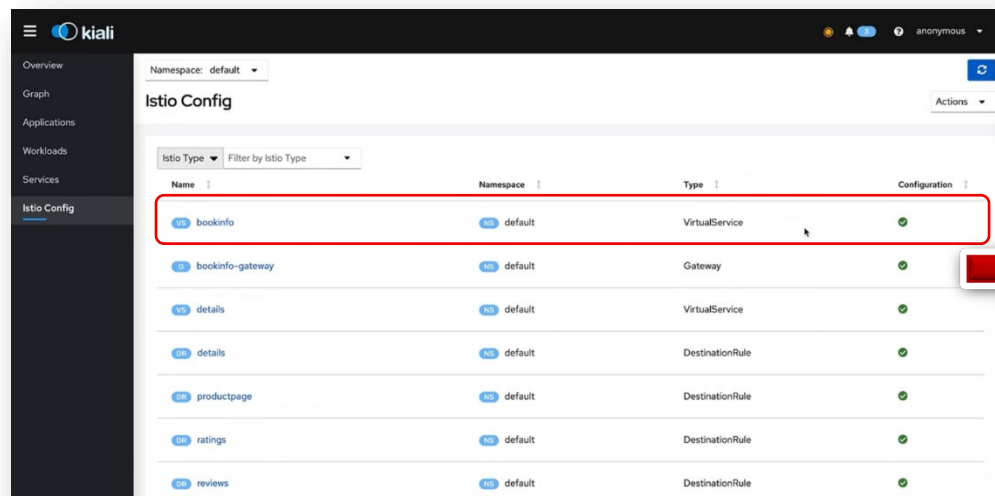
- تقدر تشوف وتعديل :
 - VirtualServices
 - Gateways
 - Policies



كمان: ⚡

• فيه Validation (يحدرك لو config غلط)

• Wizard بدل YAML



7. Istio Wizard

❖ بتخليك تعمل Configurations بتاعة Istio (زي VirtualService, Gateway...) من غير ما تكتب YAML بايديك

❖ Wizard بيطلع ايه؟

• بيفتحلك UI زي Form كدا ويسألك:

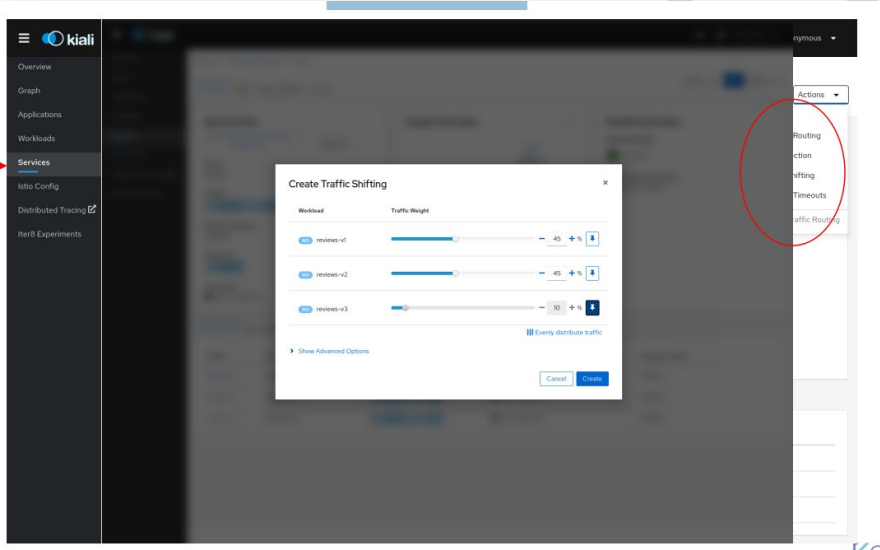
○ عايز تعمل ايه؟ (Routing / Fault Injection / mTLS)

○ اسم الـ service؟

○ عايز traffic يروح فين؟

○ النسب قد ايه؟

🌈 وانت بس تختار الـ Options وهو يولدلك الـ YAML



⚠ سيناريو عملي

○ المشكلة:

• حذف Service (reviews)

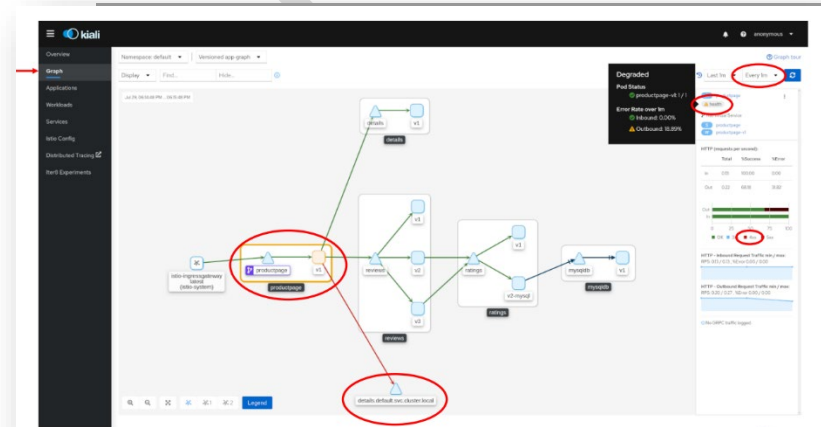
○ Kiali عمل ايه؟

• ظهر Errors في الـ Graph

• الترافيك بقى أحمر

• عرفنا إن المشكلة في reviews

🌈 ودا بالظبط اللي بتحتاجه في Production



Kiali يجمعهم بشكل بصري سهل

وظيفته	Tool
Metrics	Prometheus
Dashboards	Grafana
Tracing	Jaeger
Visualization + Control	Kiali

الخلاصة المهمة ليك كـ DevOps

- Kiali ← يعني عينك على الـ Service Mesh وشايفها قدامك
- أسرع طريقة تعمل :
 - Debugging
 - Root Cause Analysis
 - Understanding Traffic

Tip مهم

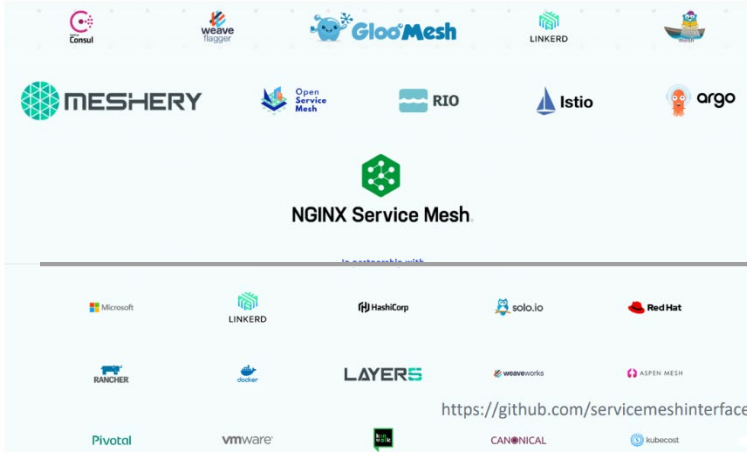
- لو اشتغلت في Production :
- دايماً شغل :
 - mTLS
 - Tracing
 - Metrics

عشان Kiali يدريك Full visibility

Service Mesh Interface (SMI)

إيه هو Service Mesh Interface (SMI) ؟

Service Mesh Interface



Service Mesh Interface هو Standard اتعمل عشان:

يخلي كل Service Mesh Tools تتكلم نفس اللغة

يعني بدل ما تتعلم:

Istio بطريقة

Tool ثاني بطريقة مختلفة

فيبقى عندك Interface موحد تشتغل بيه على أي Service Mesh

المشكلة اللي SMI بيحلها

مع انتشار Service Mesh :

كل Vendor بيضيف Features بطريقته

APIs مختلفة

Configs مختلفة

النتيجة: Vendor Lock-in

(لو اتعلمت Tool ← صعب تنقل لواحدة ثانية)

الحل SMI :

اتقدم من Microsoft سنة 2019

وييعلن: Abstraction Layer (يعني طبقة فوق كل Service Mesh)

تخليك:

تكتب Config مرة واحدة

يشتغل على أكثر من Tool

SMI بيغطي إيه؟

1. Traffic Policy

مين يقدر يكلم مين (زي Authorization)

2. Traffic Telemetry

Metrics / Monitoring

زي :

request rate

errors

latency

3. Traffic Management

Routing

A/B testing

Canary releases

- Istio واحد من الـ implementations التي يدعم SMI
- يعني:
 - تقدّر تستخدم Istio
 - أو تغيره بغيره
- بدون ما تعيد كتابة كل حاجة (نظرياً)

⚠ نقطة مهمة (واقعية شوية)

- ❖ الفكرة حلوة جداً... بس:
- مش كل Features في Istio موجودة في SMI
- SMI بيغطي basic/common features بس
- 👉 يعني:
- لو استخدمت Advanced Features في Istio ← هترجع تاني لـ Vendor Lock-in

🔗 الخلاصة ليك كـ DevOps

- SMI = Standard لتجنب الـ Lock-in
- مفيد في:
 - Multi-cloud
 - شركات بتغير Tools كثير
- 👉 لكن:
- لو عايز تستفيد بأقصى قوة من Istio ← غالبا هاتستخدم Features خارج SMI

🧠 Tip مهمة

- ❖ خلي فكرك دايماً كده:
- SMI → portability
- Istio native → power
- ❖ وأنت تختار حسب:
- هل المشروع محتاج flexibility ؟
- ولا محتاج advanced control ؟